

Received 6 February 2025, accepted 22 March 2025, date of publication 1 April 2025, date of current version 8 April 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3556187

 SURVEY

A Survey of Ransomware Detection Methods

SALEH ALZHRANI^{1,2}, YANG XIAO¹, (Fellow, IEEE), SULTAN ASIRI^{2,3},
JIANYING ZHENG⁴, AND TIESHAN LI⁵, (Senior Member, IEEE)

¹Department of Computer Science, The University of Alabama, Tuscaloosa, AL 35487, USA

²Department of Computer Science, Applied College, King Khalid University, Muhayil 61913, Saudi Arabia

³Center for Artificial Intelligence (CAI), King Khalid University, Abha 61421, Saudi Arabia

⁴School of Intelligent Manufacturing and Smart Transportation, Suzhou City University, Suzhou 215104, China

⁵School of Automation Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China

Corresponding authors: Yang Xiao (yangxiao@ieee.org) and Jianying Zheng (zjy@szcu.edu.cn)

The work of Jianying Zheng was supported by the Young and Middle-Aged Academic Leader Qing Lan Project of Jiangsu Province.

ABSTRACT Ransomware attacks continue to pose a significant challenge to cybersecurity, causing substantial financial and reputational damage to individuals and organizations. These attacks typically encrypt user data and demand a ransom for its release. There is a growing need for more effective and dynamic detection methods, especially for zero-day and unknown ransomware variants. This survey focuses on ransomware-detecting methods published from 2019 to 2025. One hundred thirty-five papers were reviewed and filtered based on their scope and publication venue. For in-depth analysis, this survey selected 45 papers focusing on ransomware detection for Windows or Android operating systems. This paper aims to comprehensively review existing ransomware detection methods, focusing on their effectiveness, limitations, and applicability. The detection methods are categorized into machine-learning-based and non-machine-learning-based methods, discussing the advantages and drawbacks of each. The paper also highlights ransomware-as-a-service, explaining what it is, how it works, and how it affects the increasing number of ransomware attacks in recent years. It also studies the datasets used in the reviewed literature, listing their structures and limitations. This survey identifies gaps in current research and suggests future directions for developing more robust ransomware detection systems.

INDEX TERMS Ransomware, detection methods, static analysis, dynamic analysis, machine learning.

I. INTRODUCTION

Ransomware, unlike traditional malware, openly takes over system resources. Traditional malware usually tries to stay hidden on the host system. Crypto-ransomware encrypts victims' files and demands a ransom for decryption [1]. Ransoms are often paid in cryptocurrencies for anonymity [2], [3]. The threat from this malware has grown since 2012 due to inadequate security systems [4], [5].

The concept of ransomware was theoretically envisioned as early as 1996, with practical manifestations around 2005, marking the beginning of a new era of cyber extortion [6], [7], [8].

In 2006, ransomware became a real method for cyber extortion, with malware creators using the increasing number of internet users and e-commerce platforms to increase their illegal profits [9]. By 2008, ransomware had adopted

encryption as a primary means of extortion, demonstrating a significant evolution in its modus operandi. The sophistication and effectiveness of encryption-based extortion marked a notable shift in the threat landscape [7], [10]. With the ubiquity of mobile devices, ransomware authors extended their reach into the mobile domain, encapsulating a broader spectrum of potential victims.

As predicted by research labs [11] and the Federal Bureau of Investigation (FBI) [12] in 2015, the ransomware trend has escalated, and it remains among the top five most dangerous threats in the cybersecurity landscape as of 2023 according to Forrester [13].

Ransomware as a Service (RaaS) mirrors Software as a Service (SaaS), allowing individuals to launch attacks using pre-built ransomware tools and taking advantage of existing vulnerabilities in some software running on old or outdated and unpatched systems. RaaS affiliates earn a portion of successful ransom payments [14], [15]. Notable ransomware like Ryuk and Conti operate within the RaaS framework, with

The associate editor coordinating the review of this manuscript and approving it for publication was Md. Moinul Hossain¹.

Conti targeting various organizations globally since 2019 [16].

Current approaches to ransomware detection employ various methods, ranging from traditional signature-based detection to advanced machine learning (ML) models [17]. However, these techniques come with their own sets of challenges. Signature-based methods are often ineffective against new variants, and ML approaches require a comprehensive and continually updated dataset for training. Moreover, existing solutions are primarily reactive, relying on known indicators like file hashes or Internet Protocol (IP) addresses. Such a strategy often results in data loss and system damage before the ransomware is effectively neutralized. Conversely, proactive approaches like Cyber Threat Hunting (CTH) are still in their infancy and have yet to prove their comprehensive efficacy against ransomware.

This survey reviews work published between 2019 and 2025. A search was conducted for all papers containing the words “ransomware” AND “detection” in the title, resulting in 140 papers being selected and screened based on their relevance and publication source. Non-English and non-technical papers were filtered out. From these, 45 papers were included for further analysis addressing ransomware detection for Windows [18], [19] or Android operating systems. The objective is to thoroughly assess current ransomware detection techniques, emphasizing their effectiveness, limitations, and practicality. Fig. 1 illustrates the survey’s flow.

In light of the existing literature and ongoing challenges, this survey aims to make several contributions:

- Present a taxonomy of existing ransomware, classifying its various types and outlining the analysis techniques and detection methods used to understand and counter it.
- Explain RaaS, detailing its functionality, significant impact on the threat landscape, and implications for the evolution of cybercrime.
- Offer a critical analysis of state-of-the-art ransomware detection methods, highlighting their advantages, limitations, and applicability to different platforms.
- Compile all accessible datasets for ransomware research, providing a comprehensive overview. Additionally, discuss the limitations of existing datasets and propose solutions to improve data collection and analysis.
- Identify current gaps in ransomware research and outline future directions to advance the ransomware detection and mitigation field.

This survey provides an extensive analysis of the current state of ransomware detection methods and future challenges, offering a comprehensive resource for researchers and practitioners alike to devise more effective and resilient countermeasures against the ever-evolving ransomware threat.

The rest of the paper is organized as follows. Section II lists some related work, explaining how this paper differs and the

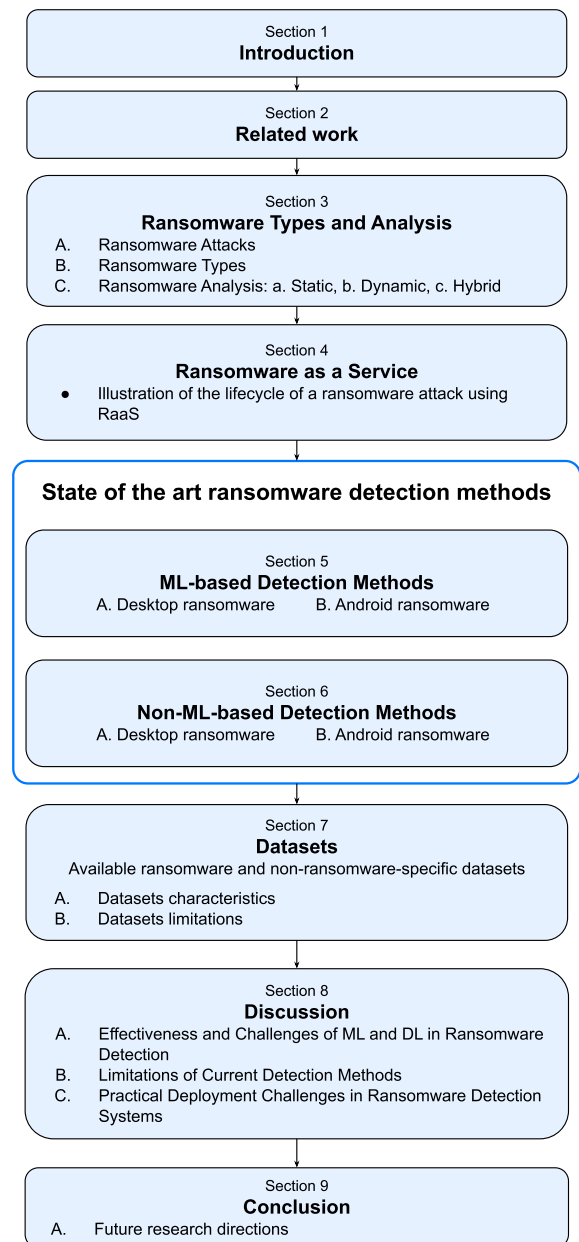


FIGURE 1. Representation of the flow among survey sections.

gaps it plans to fill. In Section III, recent ransomware attacks, types, and analysis techniques are presented. In Section IV, RaaS is explored, providing an overview of its nature and discussing its impact on the recent surge in ransomware attacks. Section V presents state-of-the-art ML-based ransomware detection methods. In Section VI, state-of-the-art non-ML-based ransomware detection methods are presented. In Section VII, the limitations of available ransomware datasets are discussed, along with some suggestions to overcome those limitations. Section VIII discusses the limitations of state-of-the-art ransomware detection methods. Finally, the paper is concluded in Section IX, and future directions are discussed.

II. RELATED WORK

Understanding the evolution of ransomware is crucial for developing effective detection and prevention techniques. Many survey papers study ransomware and its detection methods, each focusing on a specific approach or providing a comprehensive overview of all detection approaches. In this section, their summaries and limitations are listed as follows:

The authors [20] survey Android ransomware, covering its various types and feature extraction, analysis, and detection methods. They emphasize using ML algorithms for detection and provide a review of Android ransomware research from 2015 to 2020. They discuss various methods for detecting Android ransomware using ML techniques. They explain how to use hidden Markov Models (HMM), k-nearest Neighbors (kNN), Decision Trees (DT), Ensemble Learning, Logistic Regression (LR), Support Vector Machines (SVM), and Artificial Neural Networks (ANN) for this purpose. The methods are employed to analyze various features and behaviors of Android apps to differentiate between malicious and benign ones. One limitation of the paper is that it focuses exclusively on Android ransomware without drawing broader conclusions applicable to the wider field of ransomware detection. It also lacks a comparative analysis of the effectiveness of Android ransomware detection methods compared to those used for other platforms or types of malware.

The complexity and diversity of Internet of Things (IoT) devices introduce additional challenges in ransomware detection [21]. The authors [22] discuss the limitations of IoT devices' heterogeneity and lack of security design. They also explore the potential of Artificial Intelligence (AI), Blockchain, and Software Defined Networks (SDN) as solutions to detect ransomware attacks in IoT environments, indicating the necessity for future research in this domain. On the other hand, in [23], the authors examine the detection techniques for the encryption activities carried out by crypto-ransomware, suggesting a hybrid approach combining multiple detection methods and ML.

One reason for the increase in ransomware attacks is the rise of RaaS, a business model used by cybercriminals where ransomware creators lease their malware to affiliates. These affiliates use the ransomware to conduct attacks and, in return, share a portion of the profits with the creators. This model makes it easier for individuals with limited technical skills to launch ransomware attacks.

The authors [24] provide insights into the RaaS threat landscape, discuss how RaaS works, provide statistics on ransomware cases, and offer various strategies for mitigating the impact of ransomware attacks. However, the paper lacks a discussion of detection methods for ransomware, which is a significant limitation. It primarily focuses on prevention and containment strategies, leaving a coverage gap. Without addressing detection methods, the paper does not offer a comprehensive view of the entire lifecycle of ransomware defense.

The authors [25] conduct a survey focused on ML-based ransomware detection methods, such as Random Forest (RF), SVM, Naïve Bayes (NB), kNN, Stochastic Gradient Descent (SGD), and LR. The paper lacks a detailed investigation of each method, showcasing its advantages and limitations.

The authors [26] systematically reviewed ransomware countermeasures, focusing on Windows-based ransomware. They defined four stages of the malware attack (Delivery, Deployment, Destruction, and Dealing) and assigned corresponding countermeasures for each phase. They also detailed the taxonomy of ransomware workflow, from the initial infection means to the measures taken by attackers to bypass detection solutions. The paper also provides an overview of ransomware detection techniques developed by the research community between 2014 and 2021. However, it is limited because it does not thoroughly analyze each detection method in detail, including its strengths, weaknesses, performance, and comparisons with other detection methods.

Understanding the modus operandi of ransomware attacks is crucial for developing effective detection mechanisms. The authors [27] provide a survey on ransomware intrusion and infection vectors, as well as anti-analysis and evasion techniques employed by ransomware. Their work calls on the research community to address these technical challenges, suggesting the need for innovative algorithms and procedures for effective detection.

The authors [28] present an overview of the ransomware threat landscape. They discuss the prevalence of ransomware attacks, categorize different types of ransomware, dissect how these malware intrude and spread, and analyze the evasion techniques used by ransomware. The paper covers ransomware detection methods from 2019 to 2021, which may limit the comprehensiveness of the survey. While the paper discusses various detection models and methods, it may need a more detailed analysis of the practical implementation challenges and considerations for deploying these detection techniques in real-world scenarios. The paper should have extensively evaluated and validated the proposed ransomware detection models. It is crucial to thoroughly assess the effectiveness and robustness of these models in real-world environments.

The authors [29] discuss different detection methods for crypto-ransomware attack detection. They categorize detection methods into event-based detection and ML-based detection. Event-based detection involves searching for specific gauges that indicate an impending crypto-ransomware attack by tracking specific events such as Command-and-Control (C&C) traffic, Domain Generating Algorithms (DGA) used by ransomware, and data shared between infected malware and its distant C&C server. ML-based detection utilizes ML techniques to model the behavior of crypto-ransomware attack patterns. It involves using single-based classifiers such as SVM, LR, and DT and ensemble-based classifiers such as Bagging, AdaBoost (AB), and RF. Their work offers valuable insights into using ML for ransomware detection.

However, it would benefit from exploring non-ML detection methods for a balanced view. Analyzing traditional methods, such as signature-based detection, heuristic approaches, and behavior-based techniques, would highlight each one's strengths and weaknesses. This comparison could identify scenarios where ML techniques excel, or non-ML methods might be more suitable, providing a clearer understanding of ransomware detection.

The authors [30] present a survey on ransomware and ransomware defense research concerning Personal Computers (PCs)/workstations, mobile devices, and IoT/Cyber-Physical Systems (CPS) platforms. It covers 137 studies throughout 1990-2020. They cover some ransomware detection methods, including rule-based detection, ML-based detection, formal methods-based detection, nature-inspired computing-based detection, and hybrid detection. The paper lacks in-depth technical details for each ransomware detection and defense method. The paper provides an extensive list of studies and their techniques, but it does not delve deeply into the technical aspects of each method.

The authors [31] investigate using Cyber Threat Intelligence (CTI) and CTH techniques, highlighting the limitations and gaps in existing approaches. Their work focuses on CTI and CTH techniques but lacks other reactive ransomware detection methods; it also needs more technical details for each method covered.

The authors [23] focus on the cyber kill chain and its encryption phase, highlighting three main detection methods: Application Programming Interface (API) and system calls, Input/output (I/O) monitoring, and file system activities monitoring, all of which utilize ML. One area for improvement in the paper is focusing on delayed detection methods without significantly emphasizing early detection methods. The authors [14] highlight the strategies employed at different stages of the attack chain, such as infection, installation, communication, execution, extortion, and emancipation. Their work summarizes each surveyed paper without comprehensive details of each covered detection method.

The authors [32] survey ML-based techniques for ransomware detection and static and dynamic analysis. From 2017 to 2022, they conducted specific research studies on ML techniques for ransomware detection. Their work focuses on ML-based techniques, with little coverage of other detection approaches.

The authors [33] trace the origins and evolution of ransomware, propose a taxonomy categorizing different types of ransomware, and review existing research on detection, prevention, mitigation, and prediction techniques. They identify a lack of emphasis on predicting ransomware and highlight under-researched areas in the field, such as adversarial ML exploitation and concept drift. While their survey is comprehensive, they did not provide technical details for covered detection methods, especially ML-based detection approaches.

Most studies identify future directions and open challenges; for example, the authors [34] call for continuous

efforts to develop effective security defense mechanisms, the authors [33] identify shortcomings related to real-time protection and zero-day ransomware detection and the paper in [28] emphasize the need for more research in dynamic analysis for all targeted platforms.

ML and Deep Learning (DL) have shown promise in enhancing detection capabilities [35], [36]. However, the dynamic nature of ransomware calls for continuous innovation, hybrid approaches, and proactive detection strategies. While the current literature offers valuable insights into the various aspects of ransomware detection, gaps remain. The challenges of early detection, the need for comprehensive feature engineering, the limitations of current detection mechanisms in IoT, and the call for continuous research efforts are common threads across the reviewed works.

In Table 1, a comparison of this work to existing survey studies is presented. The comparison is based on whether the study paper covered specific topics such as RaaS, ransomware analysis, ML detection, non-ML detection methods, and datasets. "Yes" indicates that the study covered the topic, "No" indicates that it did not cover the topic, and "Partial" indicates that the paper partially covered the topic. If the study does not specify a target platform, "Not Specified (N/S)" is used.

This survey also focuses on RaaS, a vital driver of the recent surge in ransomware attacks. It explores ransomware analysis techniques and their limitations, exploring both ML and non-ML methods for detection. It examines the challenges of finding suitable datasets for training and testing these models and provides suggestions for overcoming those limitations.

III. RANSOMWARE TYPES AND ANALYSIS

This section explores recent ransomware attacks, dissecting their outcomes and categorizing the different types of ransomware and their analysis techniques. Fig. 2 presents a taxonomy of ransomware types, analysis techniques, and detection methods.

A. RANSOMWARE ATTACKS

Ransomware has significantly evolved in complexity and potency, particularly accentuated by the COVID-19 pandemic, which has increased reliance on digital platforms and remote work. High-profile cases, such as the May 2021 attack on Colonial Pipelines [37], the most extensive oil pipeline in the United States of America (USA) and the similar attack on José Batista Sobrinho (JBS) [37], the world's largest meat processor, have not only escalated ransomware attacks as a national security concern but also broadened their impact to include a wide range of sectors, from healthcare and education to critical infrastructure as shown in Table 2. A staggering increase in incidents, from 37% of companies experiencing ransomware attacks in 2020 to 66% in 2021 [38], indicates the growing severity of this threat.

By analyzing the recent ransomware attacks, we have observed the following:

TABLE 1. Comparison of survey research papers published between 2019 and 2023 with our work.

Ref	Year	Platform	RaaS	Analysis	ML methods	Non-ML methods	Datasets
[20]	2019	Android	No	Yes	Yes	Yes	Partial
[22]	2020	N/S	No	No	Yes	No	No
[24]	2021	N/S	Yes	No	No	No	No
[26]	2021	Windows	No	Yes	Partial	Yes	No
[27]	2021	N/S	No	Partial	No	Yes	No
[25]	2022	N/S	No	No	Yes	No	Partial
[28]	2022	N/S	No	Yes	Yes	Partial	Yes
[29]	2022	N/S	No	No	Partial	Partial	No
[30]	2022	N/S	No	Yes	Partial	Partial	Partial
[31]	2022	N/S	Partial	No	Partial	Partial	Partial
[23]	2023	N/S	No	No	No	Yes	No
[32]	2023	N/S	No	Yes	Yes	No	No
[33]	2023	N/S	Partial	Yes	Yes	Yes	No
[14]	2023	N/S	No	No	No	Yes	No
This work	2025	Desktop/Android	Yes	Yes	Yes	Yes	Yes

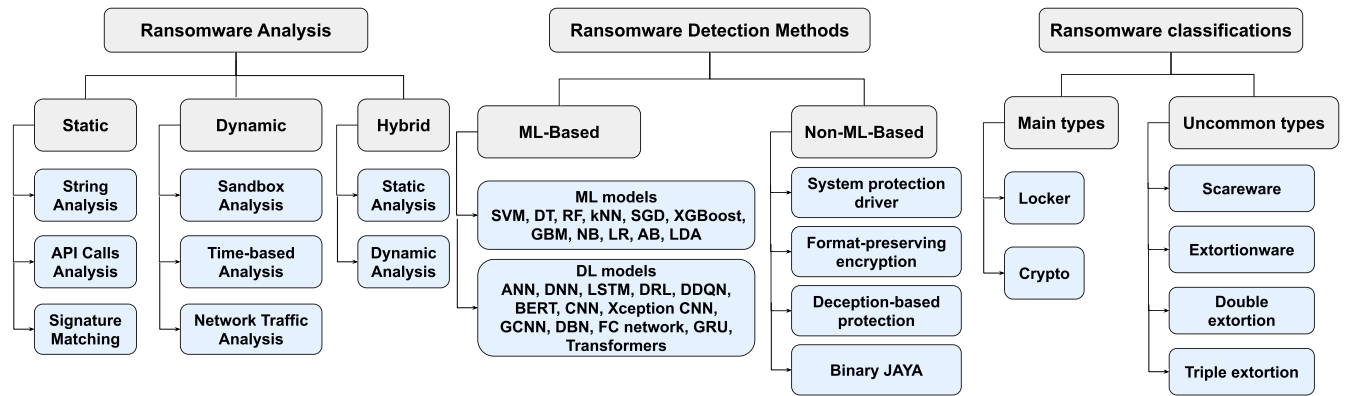


FIGURE 2. Taxonomy of ransomware types, analysis techniques, and detection methods.

- **Attack Frequency:** Numerous attacks occurred throughout 2023, indicating that ransomware remains a persistent and frequent threat. We expect this trend to increase in 2025 upward due to many factors, such as:
 - The proliferation of easy-to-use RaaS models.
 - The rise of cryptocurrency and other anonymous payment methods makes ransom payments harder to trace.
 - Increasing reliance on digital infrastructure across all sectors.
 - The potential for high financial rewards for successful attacks.
 - Ongoing geopolitical tensions are leading to state-sponsored cyber activities.
 - The expanding attack surface due to the growth of IoT devices.
 - Challenges in maintaining up-to-date cybersecurity measures across large organizations.
 - The continued evolution of sophisticated evasion techniques by attackers.
 - Potential exploitation of emerging technologies like AI for more targeted attacks.
- **Healthcare Vulnerability:** Healthcare organizations are prime targets for ransomware attacks due to their critical data, immediate operational impact on patient care, and often outdated or complex IT infrastructure. Limited cybersecurity resources, regulatory pressures, and the continuous nature of healthcare services compound the sector's vulnerability. These factors make healthcare providers particularly attractive to attackers and underscore the urgent need for improved cybersecurity measures in this critical sector [41], [44], [47], [48].
- **Target Diversity:** Attackers are targeting a wide range of organizations, including healthcare providers, government entities, educational institutions, manufacturing companies, and entertainment businesses [39], [40], [41], [42], [43], [46], [50], [54].
- **Geographical Scope:** The attacks affect organizations in multiple countries, including the USA, UK, Australia, Taiwan, Denmark, and China, highlighting the global nature of the threat [39], [40], [49], [50], [52], [59].

TABLE 2. The most impactful ransomware attacks 2023 were organized without adherence to chronological order.

Target	Date	Ransomware Group	Ransom Demand	Response/Outcome
Royal Mail - United Kingdom (UK)	January, 2023	LockBit	\$80 million	Data leaked after not paying ransom [39].
United States Marshals Service - (USA)	February, 2023	Unknown	Unknown	Most critical tools were restored within 30 days of the breach discovery [40].
Managed Care of North America (MCNA) Dental	March, 2023	LockBit	\$10 million	Leaked data posted on the dark web; an extensive list of healthcare providers impacted [41].
Minneapolis Public Schools	March, 2023	Medusa	\$1 million	Confidential information leaked; ransom not paid [42].
Capita (UK)	March, 2023	BlackBasta	Unknown	The share price dropped 12% after the attack. The attack will incur expenses of up to £20 million. [43].
Lehigh Valley Health Network - (USA)	March, 2023	ALPHV (BlackCat)	Unknown	Healthcare providers faced lawsuits; data leaked after refusing to pay ransom [44].
MOVEit	May, 2023	Clop	\$70 million	Ongoing impact and fallout; some victims publicly announced their involvement [45].
City of Dallas - (USA)	May, 2023	Royal ransomware	Unknown	\$8.6 million approved for services; identity theft reported by some city employees [46].
Barts Health National Health Service (NHS) Trust - (UK)	June, 2023	ALPHV (BlackCat)	Unknown	Ongoing investigation into the scope of the attack [47].
St Margaret's Health - (USA)	June, 2023	Unknown	Unknown	Closure due to ransomware attack; other factors also contributed [48].
Hillyer, Wormald & Leake (HWL) Ebsworth - (Australia)	June, 2023	BlackCat	\$5 million Australian Dollar(AUSD)	1.4 terabyte (TB) of exfiltrated data publicly released; ransom not paid [49].
Taiwan Semiconductor Manufacturing Company (TSMC) - (Taiwan)	June, 2023	Bassterlord (LockBit)	\$70 million	TSMC reported no breach; an attack on IT hardware supplier acknowledged [50].
Tampa General Hospital - (USA)	July, 2023	Unknown	Unknown	Class-action lawsuit filed; delayed notification [51].
CloudNordic - (Denmark)	August, 2023	Unknown	Six Bitcoins (\$157,000)	Data loss; financial demands not met [52].
Prospect Medical Holdings - (USA)	August, 2023	Unknown	Unknown	Substantial impact on operations; ongoing fallout reported in the news [53].
Caesars Entertainment - (USA)	September, 2023	Scattered Spider	\$15 million	Ransom payment made; specific access to loyalty database reported [54].
MGM Resorts - (USA)	September, 2023	Scattered Spider	Unknown	Downtime costs and financial losses reported; lack of communication from attackers [55].
Johnson Controls International - (Global)	September, 2023	Dark Angels	\$51 million	Operations impacted; stolen data may contain sensitive Department of Homeland Security (DHS) data [56].
KNP Logistics - (UK)	September, 2023	Akira	Unknown	The company was declared insolvent due to financial impact; employees were made redundant [57].
Ardent Health Services - (USA)	November, 2023	Unknown	Unknown	Investigations ongoing; patient care impacted, no ransomware group identified [58].
Industrial and Commercial Bank of China (ICBC) - (China)	November, 2023	LockBit	Unknown	Fallout in Wall Street and Beijing; Federal Reserve involved in resolving trading rates [59].

- **Ransom Variability:** The ransom amounts range from \$1 million to \$80 million. This indicates that attackers adjust their demands based on the target's perceived ability to pay [39], [41], [42], [45], [49], [50], [56].
- **Attack Consequences:** The outcomes often involve data leaks, financial losses, operational disruptions, and reputational damage. Organizations sometimes faced lawsuits or had to make employees redundant [39], [41], [42], [43], [45], [51], [55], [57].
- **Long-term Impact:** Many attacks had ongoing fallout, investigations, or recovery efforts lasting well beyond the initial incident [45], [47], [53], [58].
- **Threat Actor Landscape:** The ransomware landscape is highly fragmented, with numerous active groups, including LockBit, BlackCat, Clop, Conti, REvil, and others. This diversity indicates a competitive cybercriminal ecosystem, with various actors employing distinct tactics and targeting different sectors, underscoring the complex and evolving nature of the ransomware threat.
- **Tactical Evolution:** Attackers are increasingly targeting critical IT infrastructure and valuable databases, suggesting a shift towards more strategic and sophisticated approaches and a deeper understanding of organizational vulnerabilities and high-value assets [50], [54].

As the digital landscape continues to evolve, so too do the tactics employed by cybercriminals in their ransomware operations. This section explores four cutting-edge ransomware attack strategies that are at the forefront of this ever-changing threat landscape. These emerging approaches demonstrate the increasing sophistication, adaptability, and potential for widespread impact that characterizes modern ransomware attacks.

- **Big Game Hunting strategies**

Big Game Hunting targets large organizations, corporations, or critical infrastructure to extract higher ransom payments. This approach is characterized by extensive reconnaissance of target networks, where attackers spend considerable time understanding the architecture and vulnerabilities of the victim's systems. Manual, hands-on hacking techniques are employed to gain access and maintain persistence within the network. Once inside, attackers move laterally across the network to exfiltrate sensitive data before deploying encryption. The ransom demands are typically customized based on the financial capacity of the victim, making the attack more lucrative. Notable examples of Big Game Hunting include the attacks on Colonial Pipeline, JBS Foods, and Garmin, which resulted in significant operational [37].

- **Automated and AI-assisted attacks**

These attacks leverage AI and ML to enhance cyberattack efficiency and evasion capabilities. These attacks are characterized by automated vulnerability scanning and exploitation, where AI-driven tools can identify and exploit weaknesses in target systems faster than ever before. AI-driven target selection allows attackers to focus on victims with the highest potential for ransom payment. At the same time, dynamic malware generation ensures that the malicious code can evade traditional detection methods. Automated lateral movement within networks further complicates the defense, as AI can quickly navigate and compromise multiple systems. The potential capabilities of such attacks are vast. Self-propagating ransomware that adapts to network defenses could become a reality, making it harder for defenders to stop an ongoing attack. AI-powered social engineering techniques could be used to gain initial access to systems with greater precision. In contrast, automated negotiations using natural language processing could streamline ransom demands, reducing the need for human intervention on the attacker's side. For defenders, these developments pose significant challenges, including the increased speed and scale of attacks, more sophisticated evasion techniques, and the potential for attacks to learn and adapt in real time, making traditional security measures less effective.

- **Cloud-targeted ransomware (Ransomcloud)**

Cloud-targeted ransomware, often called Ransomcloud, is specifically designed to attack cloud-based services and infrastructure [60]. These attacks primar-

ily target cloud storage providers like Dropbox and Google Drive, SaaS platforms, Infrastructure-as-a-Service (IaaS) providers, and cloud-based email services. The attack vectors for Ransomcloud often involve exploiting misconfigurations in cloud services, compromising cloud service provider accounts, leveraging OAuth tokens for persistent access, and attacking APIs and containerized environments.

One of the unique challenges of Ransomcloud attacks is the rapid encryption of vast amounts of data, which can lead to widespread disruption due to the shared nature of cloud infrastructure. The complexity of incident response is heightened by the shared responsibility models in cloud environments, where both the service provider and the customer have roles in security. This makes it difficult to determine the exact failure point and respond to and recover from such incidents effectively.

- **Crypto-Jacking Combined with Ransomware**

Crypto-jacking combined with ransomware represents a hybrid attack strategy where cryptocurrency mining malware is paired with ransomware. The initial infection is designed to focus on crypto-mining, quietly using the victim's computing resources to generate revenue for the attacker. After a period of stealthy mining operations or in response to a specific trigger, the ransomware payload is deployed, encrypting the victim's data and demanding a ransom.

This dual-pronged approach offers several benefits for attackers. It diversifies their income streams, allowing them to earn from mining and ransom payments. The multi-stage nature of the attack increases its difficulty to detect, as security teams may focus on one aspect of the threat while the other persists. Additionally, if the initial infection is not remediated, attackers can repeatedly exploit the compromised systems, making it a long-term threat.

The challenges are significant for victims. The resource drain from crypto-mining can severely affect system performance, leading to operational inefficiencies and increased costs. Due to the layered nature of the attack, detecting and removing the threat is more complex, and there is a higher risk of repeated ransom demands if the initial infection is not thoroughly eliminated.

B. RANSOMWARE TYPES

Ransomware has evolved significantly over the years, developing into various types with distinct characteristics and attack methodologies. While crypto-ransomware and locker ransomware remain the two main types, several other variants have emerged, each employing unique tactics to extort victims.

1) CRYPTO-RANSOMWARE

Crypto-ransomware, the most prevalent type, encrypts users' files and demands a ransom for the decryption key [1]. This type of ransomware has gained notoriety due to

its potential for widespread damage and financial impact. Crypto-ransomware is characterized by:

- Higher prevalence compared to other ransomware types
- Ability to target local computers, network drives, and cloud storage
- Generally more challenging to remove
- Higher technical complexity
- Lower potential for data recovery without backups

a: WANNACRY EXAMPLE

The “WannaCry” ransomware, which emerged in 2017, is a prime example of crypto-ransomware [61]. Its modus operandi includes:

- Encrypting files on infected computers
- Demanding payment in Bitcoin for file release
- Exploiting vulnerabilities in older Microsoft Windows versions
- Capability to encrypt files on connected network drives

WannaCry’s rapid spread and significant impact on organizations worldwide highlighted the potential devastation of crypto-ransomware attacks.

2) LOCKER RANSOMWARE

Locker ransomware operates by locking users out of their systems and displaying a ransom note [62]. While less sophisticated than crypto-ransomware, it can still cause significant disruption to individuals and organizations. Locker ransomware is characterized by:

- Lower prevalence compared to crypto-ransomware
- Primarily targeting local computers
- Relatively easier to remove
- Lower technical complexity
- Higher potential for data recovery

a: REVETON EXAMPLE

The “Reveton” ransomware exemplifies the locker ransomware category [63]. Its key features include:

- Displaying fake law enforcement messages
- Accusing users of illegal activities
- Demanding payment of a “fine” to regain system access

Reveton’s use of social engineering tactics, particularly impersonating law enforcement, adds a layer of psychological manipulation to the attack.

3) EMERGING RANSOMWARE VARIANTS

Beyond the two primary categories, several less common but equally concerning ransomware types have emerged:

a: SCAREWARE

Scareware employs psychological manipulation tactics, displaying alarming pop-up messages that create a false sense of urgency [64]. Key characteristics include:

- Fake virus detection alerts
- Pressure to purchase fraudulent security software

- Exploitation of user fear and lack of technical knowledge

b: EXTORTIONWARE (LEAKWARE/DOXWARE)

Extortionware represents a shift in ransomware tactics, focusing on data theft and the threat of exposure [65]. Its features include:

- Exfiltration of sensitive data
- Threats to publish stolen information
- Potential for reputational damage to victims

c: DOUBLE EXTORTION RANSOMWARE

This variant combines elements of crypto-ransomware and extortionware, creating a two-pronged attack:

- Encryption of victim’s files
- Exfiltration of sensitive data
- Dual ransom demands for decryption and non-disclosure

d: TRIPLE EXTORTION RANSOMWARE

Triple extortion ransomware adds another layer of complexity to the attack, incorporating:

- File encryption
- Data exfiltration
- Distributed Denial-of-Service (DDoS) attacks

This multi-faceted approach increases pressure on victims and potentially accelerates ransom payments.

4) IMPLICATIONS FOR CYBERSECURITY

The diversification of ransomware types presents significant challenges for cybersecurity professionals and organizations. Each variant requires specific detection, prevention, and mitigation strategies. As ransomware evolves, a comprehensive and adaptive approach to cybersecurity becomes increasingly crucial.

C. RANSOMWARE ANALYSIS

Ransomware analysis is a process of examining ransomware samples to identify their characteristics and behaviors. This information can be used to develop countermeasures to protect against ransomware attacks and track and disrupt ransomware operations. There are three main types of ransomware analysis: static, dynamic, and hybrid.

1) STATIC ANALYSIS

It is a signature-based analysis technique that analyzes ransomware files without executing them. It can extract strings, opcode sequences, control flow graphs, and import functions from the Portable Executable (PE) file [66], which can provide helpful information about the ransomware’s capabilities. Here is a list of methods used in static analysis:

- **Signature Matching:** This method compares the ransomware sample against known malware signatures [67].

Normal code	Dead code insertion	Code transposition	Instruction substitution
mov eax, 5 add eax, 7	mov eax, 5 test ecx, ecx jz skip xor edx, edx skip: mov ebx, 7 nop add eax, ebx	jmp main add_nums: add eax, ebx ret main: mov eax, 5 mov ebx, 7 call add_nums	mov eax, 5 mov ebx, 7 neg eax sub eax, ebx neg eax

FIGURE 3. Assembly code obfuscation techniques demonstrated on a simple addition operation.

- **String Analysis:** Analysts look for specific strings within the code that might hint at ransomware activity, such as encryption algorithms or ransom messages [68].
- **File Properties:** Checking the metadata, such as the file name, size, and type, can provide clues about the ransomware.
- **Disassembly:** Tools like Interactive Disassembler (IDA) convert the binary into assembly code. Analysts can study the assembly code to understand the program's logic [69].
- **Control Flow Graphs:** Analysts create graphs to visualize the program's flow; it helps identify suspicious patterns [70].
- **API Calls:** By checking the API functions that the sample calls, analysts can determine what the ransomware aims to do, such as encrypting files or communicating with a server [71].
- **Static Heuristics:** Algorithms analyze the characteristics of the code to generate a heuristic score, indicating the likelihood of the file being malicious [72].
- **Resource Analysis:** Analysts can gather information by looking at embedded resources like icons, images, or configuration files.
- **Header Analysis:** Studying the executable header provides information on compiler settings, linked libraries, and more [73].
- **Cryptographic Algorithms:** Finding cryptographic functions or constants can indicate encryption-related activities, a common feature in ransomware [74].
- **Data Indicators:** Hash values, IP addresses, or Uniform Resource Locators (URLs) can be extracted and checked against threat intelligence databases [75].

While static analysis can help identify ransomware, it has several limitations.

a: OBFUSCATION

Many ransomware authors use obfuscation techniques to hide their code from static analysis tools. This can make it difficult to identify the ransomware's malicious behavior [76].

Basic block chaining is an obfuscation technique that hides code to make static analysis hard. It links basic blocks or instruction sets without obvious jumps [76]. This confuses analysts about where jumps or calls go. To implement this,

the obfuscator first identifies all basic blocks, then replaces direct jumps and calls with more complex instructions that maintain the program's flow while obscuring the target.

Several obfuscation methods can be employed in conjunction with basic block chaining:

- **Dead-code insertion:** This technique introduces irrelevant instructions or code paths that never execute [77], [78]. For example, a simple addition operation might be obfuscated to include dummy tests and jumps, as shown in Fig. 3. This makes the code more difficult to follow without changing its functionality.
- **Code transposition:** This method rearranges the order of instructions and uses jumps or calls to maintain the original flow [77], [78]. This obfuscation makes it harder to understand the code's structure at a glance.
- **Instruction substitution:** This technique replaces simple instructions with more complex, equivalent operations [77], [78]. For example, an addition could be replaced with a series of negations and subtractions, as shown in Fig. 3. This achieves the same result as a simple addition but in a less obvious manner.

In more advanced implementations, the obfuscator may replace a jump with code that calculates the target using a hidden constant. This value is then pushed onto the stack along with a return address. Subsequently, an x86 ret instruction transfers control to the address at the top of the stack. This method significantly complicates the process of determining a jump's target. For an analyst to deobfuscate such code, they must not only locate the basic block containing the jump but also decipher the hidden constant and understand the complex logic used to calculate the target. This task becomes increasingly challenging when multiple obfuscation techniques are layered together, creating a formidable barrier to reverse engineering and static analysis.

b: DYNAMIC BEHAVIOR

Ransomware often employs sophisticated techniques to evade detection by static analysis tools, primarily by leveraging dynamic behavior patterns. This approach capitalizes on the inherent limitations of static analysis, which examines code without execution, thereby failing to capture runtime behaviors crucial for identifying malicious intent.

Ransomware developers employ two primary techniques to obfuscate malicious activities: API hashing and dynamic API loading. These methods effectively conceal the ransomware's functionality from static analysis tools, significantly complicating detection and analysis efforts.

API hashing, API obfuscation, or import address table (IAT) obfuscation is a technique in which ransomware stores hashed versions of API names in its binary instead of plaintext strings. This approach circumvents traditional static analysis methods that rely on identifying suspicious API calls in the PE headers.

The process of dynamic API resolution in ransomware typically involves the following steps:

- 1) **Loading Required Libraries:** The ransomware utilizes the *LoadLibraryA* or *LoadLibraryW* function from the *kernel32.dll* to dynamically load necessary system libraries (e.g., *user32.dll*, *advapi32.dll*).
- 2) **Iterating Through APIs:** The ransomware enumerates its exported functions once a library is loaded. This is typically done using a combination of *GetModuleHandle* and *GetProcAddress* functions or by parsing the PE header structures directly.
- 3) **API Name Hashing and Comparison:** For each exported function, the ransomware computes a hash of the function name using the same algorithm employed during the initial hashing phase. This computed hash is compared against the precomputed hashes stored in the ransomware’s code.
- 4) **API Address Resolution:** Upon finding a match, the ransomware uses *GetProcAddress* to obtain the memory address of the corresponding API function. This address is typically stored in a data structure or global variable for later use.
- 5) **Function Pointer Typecasting:** To invoke the resolved API functions, the ransomware often employs function pointer typecasting. This allows it to call the API without direct reference to its name.

This dynamic API resolution technique presents significant challenges for static analysis tools. The absence of visible API calls in the PE headers and the runtime-only resolution of these functions render traditional static analysis methods largely ineffective. Furthermore, this approach complicates reverse engineering efforts, as analysts must first decipher the hashing algorithm and then map the hashed values to their corresponding API functions.

2) DYNAMIC ANALYSIS

It is behavior-based and involves executing the ransomware sample in a controlled environment. This allows analysts to observe the ransomware’s behavior in real-time and to identify its malicious activities, such as data encryption, file deletion, network communication, and what system resources it uses [79]. Here are some common techniques used for dynamic analysis:

- **Sandbox Analysis:** Analysts use isolated environments to run and observe the ransomware. This helps to identify its behavior, such as file encryption or network communication [80].
- **System Monitoring:** Tools like Process Monitor capture system calls, file system activity, and registry changes made by the ransomware [81].
- **Network Traffic Analysis:** Analysts monitor network packets to identify any remote servers the ransomware communicates with [82].
- **API Call Tracking:** Observing the API calls made during the execution helps to understand what the ransomware aims to do [83].

C++ Code	Assembly Code
<pre>if (IsDebuggerPresent()) ExitProcess(-1);</pre>	<pre>start: call IsDebuggerPresent test eax, eax jz no_debugger push dword -1 call ExitProcess no_debugger: ; Continue with the rest of the code</pre>

FIGURE 4. A simple code example demonstrates how the PEB structure can be utilized to detect debugging attempts. If a debugger is detected, the process is terminated immediately.

- **Memory Dump:** Extracting and analyzing the memory dump can reveal more details about the ransomware’s activities [84].
- **Behavioral Heuristics:** Like static heuristics, algorithms analyze the code’s behavior during execution to generate a heuristic score, indicating the malicious file’s likelihood.
- **Time-based Analysis:** Observing the ransomware over a specific time frame can show delayed or scheduled actions.

Combining these techniques improves the chances of accurately identifying and understanding the ransomware. However, there are several limitations to dynamic ransomware analysis:

a: ANTI-DEBUGGING

Ransomware authors continually refine their techniques to impede analysis and detection. A prevalent approach is the implementation of anti-debugging mechanisms, which aim to identify and subvert attempts at reverse engineering through debuggers.

- **Process Environment Block (PEB) Exploitation:** Ransomware frequently leverages the PEB structure to detect debugging attempts using two primary methods:
 - Examination of the *BeingDebugged* field using an API call such as *IsDebuggerPresent* from the *Kernel32* library to determine if the current process is being debugged, as shown in Fig. 4. It is also possible to check if a different process is being debugged using *CheckRemoteDebuggerPresent* from the same *Kernel32* library.
 - Inspection of the *ProcessHeap* flag to determine if the initial heap was created in debug mode.
- **System Residue Analysis:** Malicious code may search for indicators of debugging tools on the system, including:
 - Specific registry keys associated with debugger configurations.
 - Presence of debugger executables or processes using WinAPI functions like *FindWindow* or *OpenProcess*.

- **Breakpoint Detection:** Sophisticated ransomware implements methods to identify debugger-inserted breakpoints:
 - Scanning for the '0xCC' instruction.
 - Runtime calculation of code checksums or hashes to detect modifications.
- **Debugger Environment Recognition:** Several techniques are employed to identify the presence of a debugger:
 - Exploitation of the *OutputDebugString* function and subsequent error analysis.
 - Timing checks to detect execution delays indicative of single-stepping or breakpoints. When a debugger pauses code execution for inspection, the ransomware can detect this temporal discrepancy. Upon identifying such a delay, the ransomware may alter its behavior or terminate execution entirely, significantly complicating analysis efforts [85]. This technique presents a substantial challenge to researchers attempting to study and understand the ransomware's operational mechanisms.
 - Exploitation of known debugger vulnerabilities.
- **Image File Execution Options (IFEO) Inspection:** Ransomware may examine IFEO registry keys to identify configured debuggers or abuse this mechanism for malicious code execution.
- **Thread Local Storage (TLS) Callback Utilization:** Advanced ransomware leverages TLS callbacks to execute code before the PE entry point, potentially bypassing debugger initialization.

b: MODULARITY

Modular ransomware samples are composed of multiple independent modules; they can be loaded and unloaded at runtime, as seen in Conti ransomware source codes [86]. This makes tracking the ransomware's behavior and identifying its capabilities difficult [87], [88].

3) HYBRID ANALYSIS

This analysis technique merges static and dynamic analysis strengths, offering a more thorough understanding of ransomware samples than either method alone. By integrating these approaches, analysts can uncover deeper insights into ransomware behavior and capabilities. Here is how this hybrid strategy can be applied, with examples:

- **Memory-Centric Hybrid Analysis**
This method analyzes the ransomware's behavior in system memory, combining static memory dump analysis with dynamic memory monitoring. This approach can reveal encryption keys, identify process injection techniques, and uncover fileless ransomware that operates primarily in memory.
- **Network-Behavior Hybrid Analysis:**
This strategy combines network traffic analysis with system behavior analysis to provide a holistic view of ransomware operations. By correlating network communications with system-level activities, analysts

can identify command and control infrastructure, detect data exfiltration attempts, and understand the ransomware's propagation mechanisms. This approach is particularly useful for analyzing ransomware that spreads across networks or communicates with external servers.

- **Multi-Platform Hybrid Analysis:**
This approach involves analyzing ransomware across different operating systems and device types to understand its cross-platform capabilities and variations. This approach is crucial for understanding sophisticated ransomware that targets multiple types of systems within an organization.
- **Time-Series Hybrid Analysis:**
This method combines traditional analysis techniques with temporal analysis to understand the evolution of ransomware over time. By analyzing multiple versions or variants of ransomware in chronological order, researchers can track the development of new features, changes in behavior, and improvements in evasion techniques. This approach helps predict future trends and prepare for emerging threats.

The limitations of hybrid ransomware analysis include the following:

- **It can be time-consuming and resource-intensive.** Analyzing ransomware samples using a hybrid approach requires running them in a sandbox environment, which can take significant time. Additionally, analyzing ransomware samples using static and dynamic analysis techniques can require significant resources.
- **It can be challenging to identify all of the malicious components of a ransomware sample.** Some ransomware samples are designed to evade detection by using obfuscation techniques or by only executing their malicious code at a later time. Hybrid analysis can help to identify some of these malicious components, but it is not always possible to identify all of them.
- **It can be difficult to determine the full impact of a ransomware attack.** Hybrid analysis can help to identify the malicious components of a ransomware sample, but it cannot always determine the full impact of an attack. For example, hybrid analysis cannot determine whether a ransomware sample contains encrypted or deleted files.

Ransomware analysis is a time-consuming and resource-intensive process. This can make it difficult for organizations to effectively dedicate the resources necessary to analyze ransomware samples. For example, the WannaCry ransomware outbreak significantly strained the resources of organizations trying to analyze the ransomware sample. Despite these challenges, ransomware analysis is critical for understanding the threat landscape and developing effective countermeasures. Analysts can develop tools and techniques to detect and prevent ransomware attacks by understanding how ransomware works.

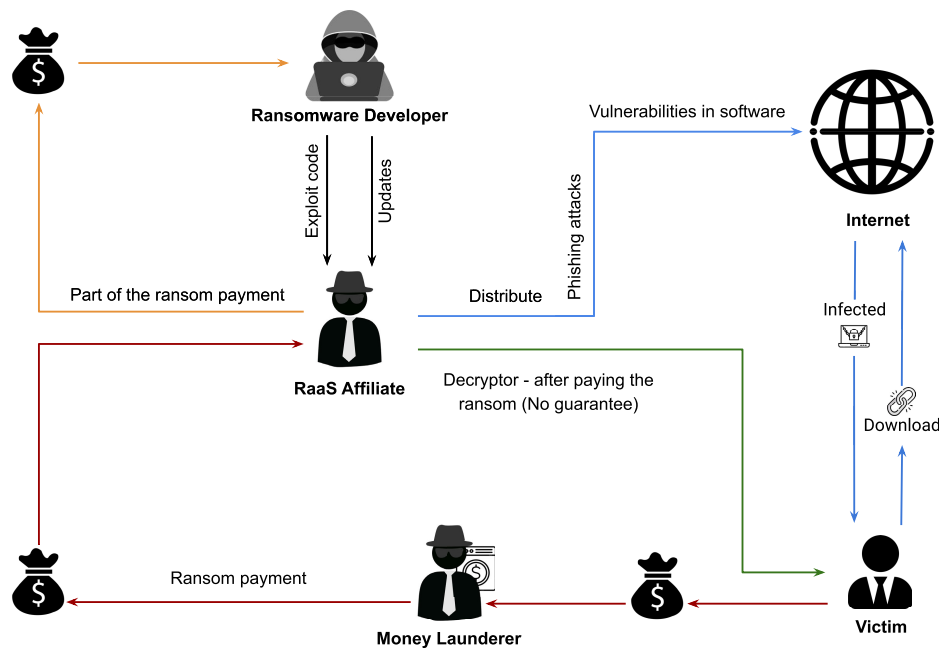


FIGURE 5. The process of a ransomware attack leveraging the RaaS business model.

IV. RANSOMWARE AS A SERVICE

RaaS has emerged as a new trend in cybercrime, allowing individuals to launch ransomware attacks and profit from them easily. This service model provides access to tools and resources for hacking, enabling anyone to become an attacker without requiring advanced programming skills or hacking techniques [66]. As a result, the scale of ransomware attacks and ransom requests has drastically increased, making cybercrimes more accessible and widespread [66]. These attacks have also evolved to include supply chain attacks, targeting vulnerable software components to extort multiple companies or customers [89].

Table 3 compiles a list of the primary vulnerabilities exploited in ransomware attacks. It provides details such as their associated Common Vulnerabilities and Exposures (CVE) numbers, the affected software, and the ransomware that exploits them. Numerous RaaS operations leverage these vulnerabilities. This is primarily attributed to many users neglecting to update their systems, leaving them susceptible to ransomware attacks even after patches for these vulnerabilities have been released.

Fig. 5 illustrates the comprehensive lifecycle of a ransomware attack utilizing the RaaS model. The process begins with a developer or hacker creating sophisticated ransomware code to exploit specific software or system vulnerabilities [90]. This malicious code is crafted to lock a victim's device or encrypt its files, subsequently demanding a ransom for their release.

Once the ransomware code is finalized and tested, it is uploaded to a specialized distribution platform, often hosted on the dark web, to maintain anonymity and evade law

enforcement [91]. Affiliates of the RaaS platform, who may be less technically skilled but adept at social engineering, purchase or lease this code and begin identifying potential targets. These affiliates commonly employ phishing attacks [92], exploit kits, or compromised software updates to deploy the ransomware on target devices.

Upon successful execution on a victim's system, the ransomware initiates its primary function: encrypting the victim's data to render it inaccessible. In more advanced attacks, the ransomware may also exfiltrate a copy of the victim's data, providing the attacker additional leverage in double extortion.

The victim is then presented with a ransom note, typically demanding payment in cryptocurrencies like Bitcoin to ensure anonymity in transactions [93]. If the victim chooses to pay the ransom, the funds are directed to the RaaS affiliate who initiated the attack. The RaaS platform usually takes a percentage of this payment as their service fee.

After confirming the payment, the victim may receive a decryption key, ostensibly enabling them to decrypt their files and regain access to their data. However, it is crucial to note that there is no guarantee that all victims will receive a functional decryption key, even after paying the ransom, highlighting the risks involved in complying with attackers' demands.

To further obfuscate the trail of illicit funds, the ransom payment often undergoes a series of complex transactions or changes hands among various intermediaries. Specialists in money laundering handle these operations, employing techniques such as cryptocurrency mixing services, chain hopping, and conversion to private coins. These methods

TABLE 3. Top vulnerabilities exploited and used in ransomware attacks.

CVE Number	CVE Explained	Affected Software	Utilized by Ransomware
CVE-2021-34527 [95]	Remote Code Execution (RCE) in Windows Print Spooler	Windows	REvil, LockBit, Conti, BlackCat
CVE-2021-44228 [96]	RCE in Apache Log4j logging library	Apache Log4j	Conti, BlackCat, AvosLocker, Vice Society
CVE-2020-1472 [97]	Elevation of Privilege (EoP) vulnerability in Microsoft Netlogon	Microsoft Netlogon	Ryuk, Conti, REvil, Maze
CVE-2019-1182 [98]	RCE in Remote Desktop Services (RDS)	Windows	WannaCry, NotPetya, REvil, GandCrab
CVE-2018-13379 [99]	Arbitrary File Upload vulnerability in Fortinet FortiOS Secure Sockets Layer (SSL) VPN	Fortinet FortiOS SSL VPN	REvil, Conti, Clop, Maze
CVE-2017-0144 [100]	RCE in Microsoft Server Message Block (SMBv1)	Windows	WannaCry, NotPetya, Bad Rabbit
CVE-2019-11510 [101]	RCE in Pulse Secure VPN	Pulse Secure VPN	Sodinokibi (REvil), RansomExx, Maze
CVE-2020-0688 [102]	RCE in Microsoft Exchange Server	Microsoft Exchange Server	DearCry, Black Kingdom, RansomExx, DoppelPaymer
CVE-2022-22954 [103]	RCE in VMware Workspace ONE Access	VMware Workspace ONE Access	Conti, LockBit
CVE-2022-22965 [104]	RCE in Spring Framework	Spring Framework	LockBit, Hive, BlackCat (ALPHV)

make it extremely challenging for law enforcement agencies to trace the funds back to the initial ransomware attack and its perpetrators [94].

Many ransomware nowadays use RaaS business models such as:

- Conti - One of the most dangerous RaaS-based ransomware, having targeted healthcare sectors, first responders, and law enforcement agencies in the US, along with over 400 global organizations since its inception in 2019 [86].
- Ransom32 - Offers a platform for individuals to implement ransomware attacks and earn money without physical presence [105].
- Ryuk – Credited to the hacking group WIZARD SPI-DER, Ryuk ranks among the most lucrative ransomware strains. By the close of 2020, it reportedly brought in around \$150 million [33].
- Maze – This ransomware was the pioneer in adopting the double extortion method. Ransomware cybercriminals groups encrypt data and threaten to make it public if not paid a ransom. Though Maze is no longer active, its tactics live on in similar ransomware like Egregor, which uses the RaaS business model [33].
- Lockbit – Introduced in late 2019, Lockbit stands out for its speed in encrypting the systems of large enterprises. This quick action limits the window for security teams to neutralize the threat [33].
- REvil - Also known by the name Sodinokibi, REvil is infamous for demanding one of the highest ransoms ever recorded, a staggering \$10 million. The ransomware spreads through various techniques, including exploiting vulnerabilities in Citrix and Pulse Secure Virtual Private Networks (VPNs) [33].

RaaS platforms create a separation between the ransomware creators and the individuals using the service,

exposing the latter to potential risks. At the same time, the former profit from their activities [106].

Research has shown that the RaaS threat is less severe than often portrayed in the media and security reports. There are relatively few RaaS items available for sale in darknet marketplaces, and the authenticity of many of these items could be better [107]. However, experienced cybercriminals still pose a significant threat, and darknet forums may serve as recruitment grounds for their organizations [107]. Understanding the value chains behind the underground ransomware economy can help mitigate this cyber threat [107].

The anonymity and privacy provided by decentralized technologies, including smart contracts and the InterPlanetary File System (IPFS), have enabled the launch of ransomware operations. By leveraging blockchain and IPFS, ransomware authors can minimize their online presence, hide their identities, and automate payments [108]. This poses a new challenge for researchers to find solutions to counter the emerging threat of ransomware facilitated by blockchain and IPFS technologies [108].

RaaS has become a significant concern in the field of cybersecurity. It has made ransomware attacks more accessible to individuals without advanced technical skills, increasing the scale and frequency of such attacks. Understanding the dynamics of the underground economy behind RaaS and exploring countermeasures against emerging threats facilitated by decentralized technologies are crucial for mitigating the risks associated with ransomware attacks.

V. ML-BASED DETECTION METHODS

Based on the literature review, ransomware detection methods can be classified into two main types: ML-based and non-ML-based detection methods. ML plays a vital role in ransomware detection by excelling in identifying complex attack patterns, adapting to evolving tactics, and efficiently processing large-scale data. Its strengths in anomaly detection, behavioral analysis, and risk assessment reduce

false positives, while adaptive learning ensures continuous improvement. ML's role extends to automated decision-making, enabling swift responses and minimizing the impact of ransomware incidents. The literature surveyed primarily focuses on two platforms: Desktop and Android.

A. DESKTOP PLATFORM

Table 4 lists some papers we reviewed for ransomware detection on desktop platforms. The table includes a variety of methods, such as ML, DL, and hybrid approaches, along with their corresponding datasets and performance metrics. We use “Ran” in the dataset column to denote ransomware samples to save space. The datasets are discussed in detail in the Ransomware Datasets section VII. The table shows that accuracy is a key metric, with several studies achieving values above 99%. For instance, the SVM, DT, RF, KNN, SGD, ANN model reported an accuracy of 99.83%, while CNN Transformers achieved 99.1%. However, many studies do not provide additional metrics like precision, recall, F1-score, or FPR, leaving gaps in comparative analysis. We added additional information about the results in the Comments column when needed. Notable advancements include using novel architectures like zero-shot learning and transformer-based models, which show potential in tackling complex ransomware detection challenges. These papers are analyzed in greater detail in this and subsequent sections, providing insights into their methodologies, results, and contributions to ransomware detection.

Singh et al. [109] introduce an approach to ransomware detection using a neural network-based method. The scheme consists of several phases: dataset acquisition, log file processing, and pre-processing. The authors collected network traffic data from multiple sources, including IEEE Dataport, for dataset acquisition. They used a Virtual Machine (VM) running Zeek software to generate log files from network captures, as shown in Fig. 6. The dataset included traffic from various ransomware families.

The authors captured network traffic from machines performing routine web browsing and emailing to obtain benign data. The resulting dataset comprised conn.log, ssl.log, and x509.log files generated by Zeek. These log files contained diverse information about network connections, SSL traffic, and X.509 certificates. The authors combined and processed these log files, creating separate Comma-Separated Values (CSV) files for each ransomware family and benign traffic. They assigned labels to the samples and added them to a combined CSV file for ransomware detection. Regarding data pre-processing, the authors implemented various techniques to prepare the dataset for analysis. This involved eliminating redundant and invalid values, handling missing data, and scaling the input features to a normalized range. The pre-processing also included generating new features from the log files and applying techniques like linear interpolation, one-hot level encoding, and outlier removal. Furthermore, the authors employed the Synthetic Minority Oversampling Technique (SMOTE) to address imbalanced datasets. In the

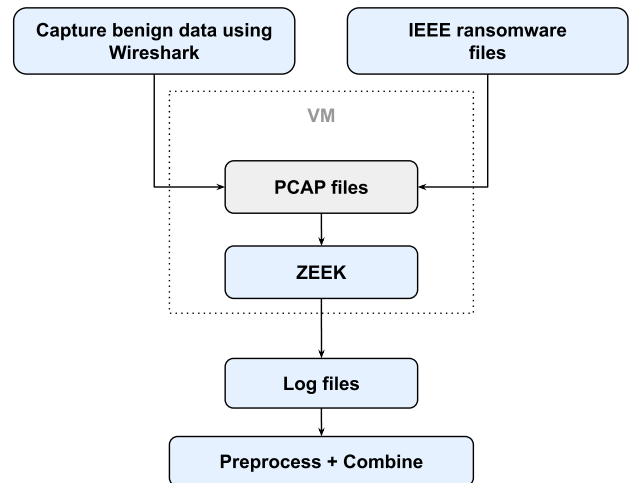


FIGURE 6. Dataset collection flow for SINN-RD scheme [109].

comparative study, the authors evaluated the performance of their proposed Spline Interpolation envisioned Neural network-based Ransomware Detection (SINN-RD) scheme against other existing schemes. The results demonstrated that SINN-RD achieved a higher accuracy of 99.83% compared to other similar schemes, including those proposed in [132], [133], [134], [135], and [136].

Thummapudi et al. [123] collected data for selected processor and disk I/O events for the entire VM from the host machine. They utilized the perf tool to capture Hardware Performance Counters (HPC) values for processor, memory, and system events and the virsh tool with the dombkstats option to collect disk I/O stats of the virtual disk allocated to the VM. Fig. 7 shows their HPC and disk I/O data collection flow. They start to launch either a benign or ransomware sample. Once a sample executes, they gather HPC and I/O data from the host machine, focusing on the VM's process for approximately 60 seconds. They evaluated five ML models, RF, SVM, DT, kNN, and eXtreme Gradient Boosting (XGBoost), and two DL classifiers, Deep Neural Network (DNN) and Long Short-Term Memory (LSTM), for their suitability for detecting ransomware based on the HPC and disk I/O data. They developed three models for each classifier: one that uses HPC data only, a second that uses disk I/O data only, and a third integrated model that uses both HPC and I/O data. They then used Python library modules such as scikit-learn, XGBoost, autokeras, and Keras with TensorFlow backend for their analysis. The authors aim to identify ransomware in action, considering it a significant threat when it encrypts files, and they aim to detect this activity promptly. They explored an alternative method to assess the detection speed by calculating the number of instances (time to detect) for the initial optimistic prediction during ransomware operation. In a 21-fold cross-validation totaling 5544 ransomware traces, they compute the time to detect ransomware for each trace.

Results for best-performing RF-integrated model:

TABLE 4. Quantitative Comparison of State-of-the-Art Papers on Ransomware Detection Targeting Desktop Platforms.

Ref	Year	Method	Models	Dataset	Accuracy	Precision	Recall	F1	FPR	Comments
[109]	2023	ML / DL	SVM, DT, RF, kNN, SGD, ANN	[110]	99.83%	98.82%	99.83%	99.83%	N/S	-
[111]	2023	DL	DRL	[112]	97.9%	N/S	N/S	N/S	2.1%	-
[113]	2023	DL	BERT, LSTM	N/S	95%	N/S	N/S	N/S	N/S	-
[114]	2023	ML	RF	Ran: 183 Benign: 183	98.68%	N/S	N/S	N/S	N/S	Five HPC register values in 2s
[115]	2023	DL / ML	RF, DT, NB, kNN, LR, SGD, SVM, Xception CNN	[116]	98.20%	N/S	N/S	N/S	N/S	Accuracy for Dataset 1 is 93.73%
[117]	2023	ML	DT, RF	[118]	97%	92%	86%	N/S	N/S	Within 120 seconds
[119]	2023	DL	GCNN DBN	Ran: 420 Benign: 420	99.64%	N/S	N/S	N/S	N/S	-
[120]	2023	DL	GRU	Ran: 420 Benign: 420	97.31%	N/S	N/S	N/S	N/S	-
[121]	2023	DL	CNN Transformers	[122]	99.1%	99.5%	98.5%	97.64%	N/S	Attained a training accuracy: 99.6%
[123]	2023	ML / DL	RF, SVM DT, kNN XGBoost, DNN LSTM	[112]	N/S	N/S	N/S	N/S	N/S	RF: Within 400 milliseconds 0.98 probability ransomware detection
[124]	2024	ML with Nilsimsa fingerprinting	RF, LR AdaBoost	[125]	98.23%	99.44%	97.74%	98.59%	N/S	-
[126]	2024	UA-DES -DBN	RF, LR AdaBoost VGG16-PSO DBN-IDS DBN	[112]	98%	N/S	N/S	N/S	0.10%	-
[127]	2024	AGNN	CNN RNN	Ran: 582 Benign: 942 Families: 11	96.6%	N/S	91.4%	N/S	N/S	-
[128]	2024	zero-shot learning	SA-CNN	Ran: 582 Benign: 942 Families: 11	96.02%	91.49%	98.47%	96.31%	N/S	-
[129]	2025	DL	BERT, RoBERTa, XLNet, GPT-2	Ran: 1,414 Benign: 657 Families: 18	99.12%	98.88%	100%	99.44%	N/S	-
[130]	2025	DL	RNN	[131]	93.45	94.92	93.77	93.34	N/S	The authors created another dataset consist of 6881 ransomware from 30 families

- 95% detection in the first instance (200ms)
- 98% detection in 400ms (two instances)
- False positive rate: 0.03
- False negative rate: 0.12

21-fold cross-validation with 5544 ransomware traces. Threshold adjustment to 0.3:

- Improved false positive rate: 0.01
- Increased false negative rate: 0.18
- 90% detection in 200ms, > 95% in 400ms

While this method shows promise for ransomware detection, it has several limitations:

- **Time sensitivity:** Even 200-400ms detection time might be too slow for preventing file encryption.

- **Limited scope:** Only focuses on VM environments, potentially missing bare-metal attacks.
- **False positives:** Even with low rates, frequent false alarms could lead to alert fatigue.
- **Resource intensity:** Continuous monitoring of HPC and I/O data may impact system performance [137].
- **Evasion techniques:** Advanced ransomware might mimic benign behavior to avoid detection. They also can detect VM environments.

The authors [111] presented a new static analysis framework for the early detection of ransomware using Deep Reinforcement Learning (DRL), specifically focusing on the portable executable header (PE header). They utilized the PE header as the primary feature for identifying ransomware

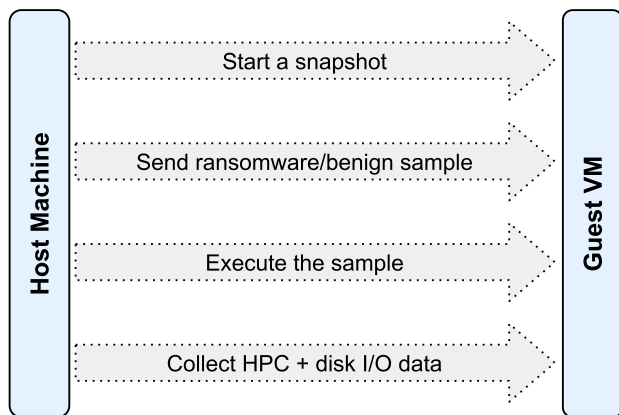


FIGURE 7. HPC and disk I/O data collection flow presented in [123].

and employed YARA rules to extract Bitcoin addresses from ransomware samples. These PE header details were then used to create feature vectors. The proposed approach involved training a DRL detection policy, with the agent interacting with the environment and utilizing the Manhattan distance for model training. The experiments were carried out using two real-world datasets. Dataset 1 comprised 62,485 PE files, while Dataset 2 comprised 688 ransomware samples from 12 families. The proposed method attained a notable level of accuracy of 97.9% and a false detection rate of 2.1% without executing the samples. The detection process involved extracting 14 PE header information and applying the ransomware detection policy to detect unseen samples. The method exhibited excellent accuracy in detecting unseen samples, achieving a 99.3% accuracy rate on Dataset 2.

In [113], the authors proposed two ransomware detection models using BERT and LSTM based on API calls gathered from Cuckoo Sandbox reports. The BERT model used a base model for pre-training, trained for five epochs with a batch size of 4, and achieved around 95% accuracy. The LSTM model, trained for 5 and 10 epochs with a batch size of 4, outperformed BERT, achieving approximately 97% accuracy at five epochs and 99% at ten epochs. The study concluded that LSTM's ability to handle sequential data with parallel operations made it slightly more effective for ransomware detection, particularly in overcoming memory forgetting issues.

Both models were trained with a small batch size of 4, which may limit their ability to generalize across larger datasets or varied environments. The BERT model, trained for only five epochs, might not have fully optimized its performance. Additionally, the LSTM's high accuracy, particularly after ten epochs, raises concerns about potential overfitting, especially if the dataset lacks diversity. BERT's lower accuracy could be attributed to computational and memory constraints, affecting its effectiveness. Furthermore, deploying these models, especially BERT, in real-world systems might introduce significant performance overhead, potentially impacting system responsiveness and security.

The authors [114] propose a method for early ransomware detection using HPC. They analyzed 183 ransomware and 183 benign samples from diverse sources, covering 27 ransomware families. By examining HPC traces of various applications, they performed a feature importance and correlation analysis to identify the optimal features for detection. The Boruta feature selection algorithm, an extension of Random Forest, was used to build a robust classification model. The model was tested on known and newer ransomware variants, showing that a smaller subset of HPC registers monitored over a shorter duration could be effective for early detection.

While the proposed method shows promise, several challenges with implementation arise:

- **Timeframe Sensitivity:** The effectiveness of the proposed detection method is highly dependent on selecting the appropriate timeframe for monitoring HPC registers. This timeframe can vary significantly depending on the environment and the specific ransomware behavior, potentially leading to inconsistent detection results.
- **Operational Load:** Monitoring and analyzing HPC data can introduce substantial overhead, potentially impacting system performance. The need for continuous monitoring of HPC events may lead to resource consumption that could be detrimental in high-performance or resource-constrained environments.

Moreira et al. [115] present a method for detecting ransomware based on static analysis of PE header files. The authors converted these files into color images using a sequential vector pattern as shown in Fig. 8. They classified them using the Xception Convolutional Neural Network (CNN) model, referred to as Xception ColSeq, without transfer learning. The method aimed to identify distinguishing characteristics in the resulting images and differentiate ransomware from benign applications.

The proposed method was evaluated using two datasets. Dataset 1 included 1000 ransomware and 1000 benign samples, while Dataset 2 comprised 1023 ransomware samples (from 25 families) and 1134 benign applications. The method achieved high accuracy and performance metrics: 93.73% accuracy, 92.95% precision, 94.64% recall, and 93.75% F-measure on Dataset 1, and 98.20% accuracy, 97.50% precision, 98.76% recall, and 98.12% F-measure on Dataset 2. It outperformed some existing methods such as [138], [139], and [140].

While this method does not rely on APIs that ransomware can obfuscate, it has several limitations:

- **Loss of Critical Information:** Converting PE headers into color images may overlook essential non-visual data like dynamic behaviors or encrypted payloads.
- **Computational Intensity:** The use of CNNs like Xception is computationally demanding, which can limit real-time application on systems with limited resources.
- **Susceptibility to Adversarial Attacks:** The visual-based approach is vulnerable to minor alterations in

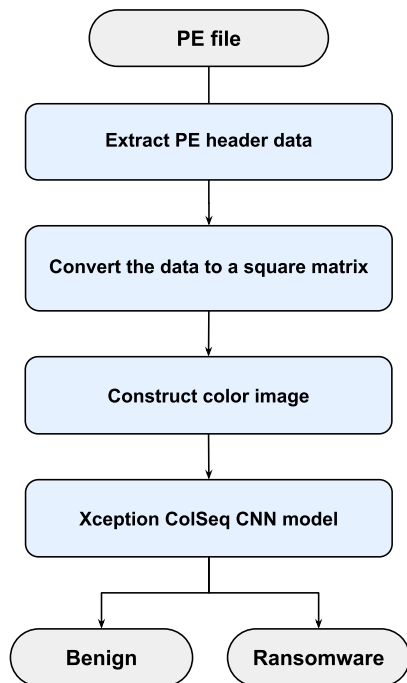


FIGURE 8. Static analysis method utilizing PE header data introduced in [115].

input images, potentially leading to misclassification and evasion of detection.

Ayub et al. [117] analyzed ransomware detection using ML and DL techniques. They focused on dynamic analysis from I/O Request Packet (IRP) logs, examining changes in “registry,” “files,” Windows API calls, and “process” behaviors to identify differences between ransomware and benign processes. They used classifiers like DT, RF, AB, and gradient-boosting for binary classification, reporting metrics such as accuracy, precision, recall, and F1 scores. Features analyzed included “imports,” “function names,” “PE headers,” “PE section names,” “functions,” and “strings,” with Principal Component Analysis (PCA) used for dimensionality reduction. They introduced “RWAarmor,” a static-informed dynamic analysis approach, which integrated static analysis features into dynamic analysis. The RF classifier showed high performance across various metrics. Data was sourced from a large-scale IRP dataset, sandbox environment, SOREL-20M repository, and Any Run platform, with VirusTotal API used for confirming ransomware.

One issue with RWAarmor is feature overlap, where combining static and dynamic features can lead to redundant data, potentially confusing the classifier and reducing performance. Another concern is scalability; models trained on large-scale datasets might not adapt well to new ransomware variants, as ransomware evolves quickly, and models may struggle with novel threats not present in the training data.

The authors [119] proposed an Optimal Graph Convolutional Neural Network-based Ransomware Detection

(OGCNN-RWD) technique for cybersecurity in an IoT environment. A Graph Convolutional Neural Network (GCNN) is a neural network that operates on graph data [141]. The proposed model involves technical details such as using the Learning Enthusiasm-based Teaching Learning-Based Optimization (LETLBO) algorithm for feature subset selection. The GCNN model is used for ransomware classification, and its hyperparameters are optimized using the Harmony Search Algorithm (HSA). The proposed model’s performance was evaluated on a database comprising 840 samples, which included 420 instances of goodware and 420 instances of ransomware. The results of the simulation show that the OGCNN-RWD system achieved an accuracy of 99.64%. The proposed technique was also compared with existing systems, outperforming other models with a maximum accuracy of 99.64%. The comparative outcomes showed that the OGCNN-RWD technique performed superior to existing techniques such as [142] and [143]. The authors utilized a ransomware dataset to evaluate the proposed model, but the paper did not provide specific details regarding the dataset’s characteristics.

Alohali et al. [120] introduced the Sine Cosine Algorithm with the DL-based Ransomware Detection and Classification (SCADL-RWDC) method, which combines the Sine Cosine Algorithm with a Deep Learning-based approach for ransomware detection. They used a Hybrid Grey Wolf Optimizer (HGWO) with a Gated Recurrent Unit (GRU) model to classify ransomware. The GRU, a Recurrent Neural Network (RNN), was applied to a dataset of 840 samples (420 each of goodware and ransomware) for training and testing. The SCADL-RWDC method showed superior accuracy, sensitivity, specificity, F-score, and Area Under the Receiver Operating Characteristic (ROC) Curve (AUC) scores compared to other methods such as [142] and [143]. Key parameters included a learning rate of 0.01, dropout of 0.5, batch size of 5, and 50 epochs. The model’s effectiveness can be further enhanced using hybrid DL models and involves feature engineering and dimensionality reduction for optimal performance. While this method shows promising results, several limitations should be considered:

- **Dataset Size:** With only 840 samples, the dataset may not be sufficient to capture the variability in ransomware, potentially leading to overfitting.
- **Hybrid Model Complexity:** Combining multiple techniques can increase model complexity, making it harder to maintain and tune.
- **Feature Engineering Dependence:** The reliance on feature engineering and dimensionality reduction might miss out on emerging ransomware patterns not well-represented in the engineered features.

Singh et al. [121] developed RANSOMNET+, a hybrid ensemble model combining CNN and pre-trained transformers to classify ransomware attacks on cloud-encrypted data. The model was trained using transfer learning and DL ensemble methods on a diverse dataset of cloud-

encrypted data. RANSOMNET+ was compared to other models like ResNet 50, VGG 16 [120], CNN, ANN [144], LSTM [145], and GRU [28], showing superior accuracy and performance. The model's effectiveness was evaluated using various metrics and visual representations over 30 epochs. Potential issues with this implementation for ransomware detection:

- **Cloud-Centric Approach:** Reliance on cloud-specific features may reduce effectiveness for non-cloud environments.
- **Limited Generalization:** Overfitting to the specific dataset, potentially limiting generalization to new ransomware strains.
- **Performance Bottleneck:** Computational intensity may limit real-time detection capabilities.

Several recent papers have explored other ML-based methods for ransomware detection. Lee et al. [146] address ransomware's ability to bypass entropy-based detection methods using encoding techniques and propose an ML-based approach to counteract this. By employing models like random forest, SVM, and gradient boosting, they achieved approximately 98% accuracy in detecting ransomware-infected files. Cen et al. [128] introduced Zero-Ran Sniff (ZRS), a zero-day ransomware detection method using zero-shot learning and PE header features for early detection. Gaber et al. [129] proposed Pulse, a framework leveraging Transformer models and assembly language analysis from the Peekaboo dynamic binary instrumentation tool to classify zero-day ransomware. Li et al. [127] developed a ransomware detection model based on an adaptive graph neural network (ADC-TextGCN) that learns from co-occurrence information and sensitive API calls.

Gajjar et al. [147] presented RD-FAXID, an FPGA-accelerated XGBoost model for multi-dataset ransomware detection using hardware performance counters, OS calls, and network traffic. Gazzan and Sheldon [148] investigated an Uncertainty-Aware Dynamic Early Stopping (UA-DES) technique for Deep Belief Networks to improve accuracy and reduce false positives. Jaimes et al. [124] enhanced ML for ransomware detection in IoMT using Nilsimsa fingerprinting for network traffic analysis. Davidian et al. [149] compared various machine-learning techniques and API call features for early ransomware detection. The authors [150] developed volGPT, a method leveraging Large Language Models (LLMs) with the Volatility framework to improve ransomware triage in memory forensics. Finally, Ashwini and Nagasundara [151] developed an intelligent ransomware detection and classification system utilizing a dual vision transformer with a Mantis Search Split Attention Network. Alian et al. [152] introduce FREEDOM, a ransomware defense for Windows that uses Alternate Data Streams (ADS) to trick ransomware into encrypting file "shells" instead of real data. Unlike backup-based solutions, it needs no extra storage, has no performance overhead, and ensures seamless protection.

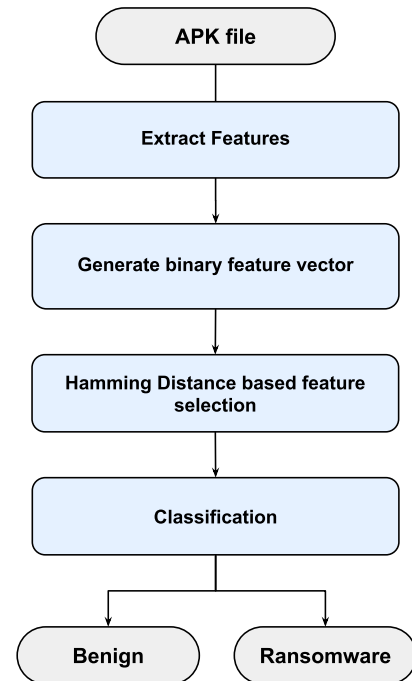


FIGURE 9. Static-based ML ransomware detection system proposed in [154].

B. ANDROID RANSOMWARE

Table 5 lists some papers reviewed for ransomware detection on Android platforms. Various methods, including ML, DL, and hybrid approaches, are presented alongside their corresponding datasets and performance metrics. "Ran" is used in the dataset column to represent ransomware samples to save space. The datasets are described in detail in the Ransomware Datasets section VII.

Accuracy is shown as a critical metric, with several studies reporting values near or above 99%. For instance, the SVM, RF, AB, DT, kNN, NB, LDA, and CNN model achieves 99.5% accuracy, while AlexNet reports an accuracy of 99%. In some cases, additional metrics such as precision, recall, F1-score, or FPR are not provided, making comprehensive comparisons challenging. Notable advancements include techniques like vision-based RD with FT ResNet50 CNN, which achieves 99.5% accuracy, and the use of high-performance, low-resource consumption models such as Honeyfile. These papers are analyzed in greater detail in this section and the following section, with their methodologies, results, and contributions discussed.

Manzil et al. [154] introduced an Android ransomware detection system using a hamming distance-based feature selection technique. The proposed system comprises four primary steps, as shown in Fig. 9.

Initially, static features like permissions and intents are automatically extracted from Android applications using a Python tool to identify distinguishing characteristics between ransomware and benign apps. Subsequently, a binary feature vector is generated from these extracted features using

TABLE 5. Quantitative Comparison of State-of-the-Art Papers on Ransomware Detection Methods Targeting Android Platforms.

Ref	Year	Method	Models	Dataset	Accuracy	Precision	Recall	F1	Comments
[153]	2023	BJAYA	-	Burkardt, Martello Toth, Kulkarni	N/S	N/S	97%	N/S	Gmean: 98.2% SMO-BJAYA-KNN Gmean: 98.2%
[154]	2023	ML / DL	RF, SVM LR, NB, kNN, DT AB, DNN	Ran: 500 Benign: 500	99%	N/S	N/S	N/S	RF, DT: Accuracy: 99% SVM, LR, kNN: Accuracy: 98% with the Adam optimizer: Accuracy: 97.19%
[155]	2023	ML	SVM, RF AB, DT, kNN, NB LDA, LR, CNN	Ran: 500 Benign: 500	99.5%	N/S	N/S	N/S	Static-based RD: AB: Accuracy: 97% Vision-based RD: FT ResNet50 CNN: Accuracy: 99.5%
[156]	2024	SMO-BJAYA-SA-SVM, SVM	-	[157]	98.7%	98.6%	98.7%	98.6%	-
[158]	2024	DL	AlexNet	[159]	99%	N/S	N/S	N/S	-
[160]	2024	DRL	A2C	Ran: 500 Benign: 10,153	89.78%	92.03%	93.03%	91.20%	-
[161]	2024	Honeyfile	-	[162]	N/S	N/S	N/S	N/S	High performance and Low resource consumption
[163]	2024	MITRE ATT&CK	-	[164]	89.82%	N/S	N/S	N/S	At the pre-operational stage
[165]	2025	DRL	Bagging AdaBoost RF, GBM, A2C	[166]	99%	99%	99%	99%	-

a specified algorithm, enabling comprehensive application analysis based on feature presence or absence. Following this, a novel hamming distance-based technique is employed for feature selection, focusing on reducing the feature space by retaining those most pertinent to discriminating ransomware from benign applications. Finally, ML and DL models, such as RF, SVM, LR, NB, kNN, and DT, are utilized for classification. The study, utilizing the CIC-InvesAndMal2019 dataset with forty samples from ten ransomware families, achieved up to 99% accuracy with RF and DT classifiers. However, the approach has limitations:

- **Static Feature Reliance:** The system's dependence on static features like permissions and intents may not account for dynamic behaviors or runtime changes.
- **Limited Detection of New Variants:** This static approach may miss newer or more sophisticated ransomware that employs dynamic tactics and obfuscation techniques.

The authors [155] proposed an End-to-End Ransomware Detection System (E2E-RDS) that utilizes both static-based ML and vision-based DL classifiers.

They used a newly collected balanced dataset with 500 benign and 500 ransomware apps for their investigation. The dataset is publicly available on the Security Engineering Lab (SEL) [157]; more details about the used dataset can be found in [167]. The collection process involved assembling the Android Package Kits (APKs) from various sources, including the Google Play store for benign APKs and several

famous market repositories for ransomware APKs, such as HelDroid, VirusTotal, RansomProper, and Koodous.

Their static-based approach shown in Fig. 10 involves decompiling Android APK files to extract metadata and code representations. This process utilizes APKtool to generate AndroidManifest.xml and SMALI files. Feature extraction is performed by analyzing these files, resulting in a comprehensive set of 778 features, including permissions, API packages, and other attributes. The feature extraction process has two main stages: parsing the manifest file for permissions and analyzing SMALI files for API packages. The extracted features undergo pre-processing to eliminate irrelevant data. Eight ML models train and test the processed features, including SVM, RF, and LR. The models' effectiveness is assessed using various performance metrics such as precision, recall, and accuracy.

Their vision-based approach method transforms binary executable files from benign and ransomware APKs into 2D visual images, either in color or grayscale.

Fine-tuned CNN models analyze these images to classify them as ransomware or benign. This efficient method uses transfer learning and fine-tuning, avoiding data augmentation or complex feature extraction. The static-based approach achieved 97% accuracy, while the vision-based approach reached 99.5% using ResNet50. However, the vision-based approach has potential drawbacks to consider:

- **High Computational Requirements:** Converting files to images and using CNNs demands significant resources, hindering real-time detection in constrained environments.

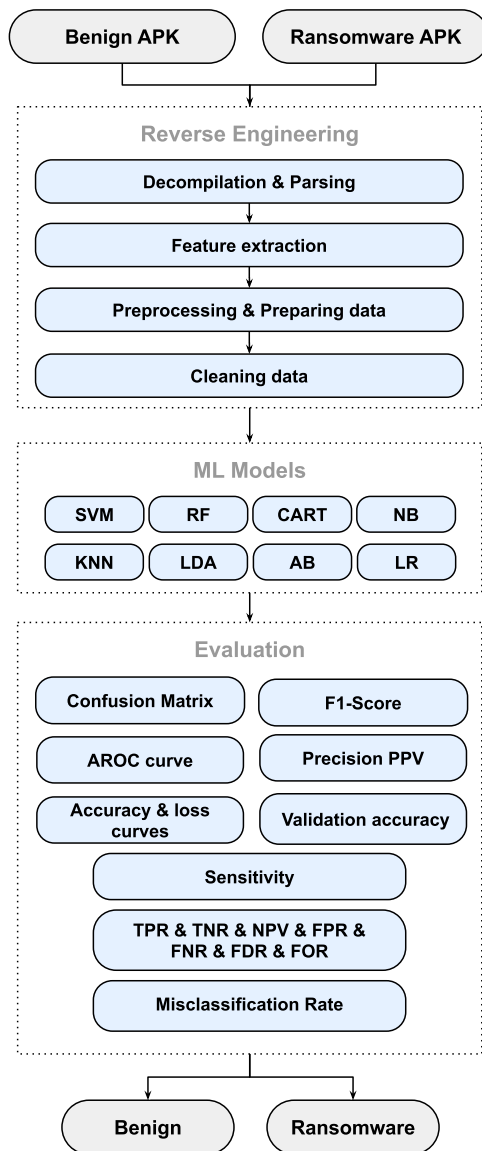


FIGURE 10. Static-based ML ransomware detection system proposed in [155].

- **Scalability Issues:** Large datasets may pose challenges, and fine-tuning for various ransomware types could become complex and time-consuming.
- **Image Representation Limitations:** Converting binary files to images might lose critical information, potentially reducing detection accuracy.

Recent studies have increasingly focused on enhancing the detection of Android ransomware. Alazab et al. [156] proposed a hybrid approach combining a multi-solution binary JAYA algorithm with single-solution simulated annealing, achieving 98.7% accuracy. The authors [158] combined a deep learning model with hybrid cryptography for both detection and secure cloud storage, achieving 99.89% accuracy and outperforming GNN, CNN, and RF. Jeremiah et al. [160] explored DRL models using application permissions and

network traffic attributes, finding the A2C model particularly effective. Hossain et al. [165] developed an ensemble ML approach using network traffic data and achieved near-perfect detection rates exceeding 99% for various ransomware types, demonstrating the power of ensemble methods for enhanced accuracy and robustness. Higuchi and Kobayashi [168] introduced the Real-time Open-File Backup System (ROFBS) to minimize ransomware damage by instantly backing up opened files. They claim to achieve high backup rates with minimal system impact.

VI. NON-ML-BASED DETECTION METHODS

Non-ML-based ransomware detection relies on signature-based methods, comparing file patterns to a known database and heuristic approaches that analyze behavioral anomalies. These methods are effective against recognized threats but may struggle with new or evolving ransomware variants. Behavioral analysis monitors real-time activities for unusual patterns, providing additional layers of defense. However, these techniques may face limitations when dealing with polymorphic strains or novel threats.

A. DESKTOP RANSOMWARE

Alexandrov et al. [169] proposed an approach for defending against ransomware encryption attacks by developing a system protection driver that intercepts file system I/O operations at either the software or hardware level. Their model emphasizes detecting the misbehavior of malicious software broadly rather than targeting specific ransomware families. The proposed protection driver intercepts critical system calls, including “file open,” “file read,” “file write,” and “file close,” and incorporates handlers to monitor for any unauthorized modifications to file metadata. The model utilizes a lookup table of predefined metadata types associated with various file extensions to detect abnormal changes that deviate from the expected behavior of workstations or servers in typical use scenarios. Additionally, the protection driver employs a caching control mechanism that incurs no additional overhead, facilitating the restoration of data that may have been tampered with. The authors also mention the potential implementation of their model as a hardware-level protection device designed to interface with the operating system file system at the hardware level. The authors did not compare their work to others regarding implementation or results.

Despite the innovative nature of this approach, there are several significant limitations:

- **Lack of Empirical Testing:** The paper presents a theoretical model without empirical validation or real-world testing. This absence of practical application raises concerns about the method’s effectiveness and reliability in combating ransomware.
- **Scope Limitation:** The approach focuses exclusively on file metadata changes. Sophisticated ransomware

might not alter metadata in detectable ways, potentially evading this detection method.

- **Performance Impact:** While the authors assert minimal real-time overhead, there is no quantitative assessment of the performance impact associated with intercepting file system I/O operations. This unquantified overhead could affect system efficiency.
- **Dependence on Predefined Metadata:** The reliance on a lookup table of predefined metadata types for different file extensions might limit the system's adaptability to new or uncommon file types, potentially impacting its ability to detect evolving threats.
- **Adaptability Concerns:** The method's effectiveness in the face of emerging ransomware variants or novel attack techniques remains uncertain, necessitating further refinement and extensive testing to ensure robustness.

These issues highlight the need for further investigation and practical validation to assess the proposed method's real-world applicability and effectiveness in detecting and mitigating ransomware threats.

A deception-based detection method uses misleading tactics to protect against unauthorized access or attacks. One challenge of these methods is determining the optimal placement of honey files within the file system. The authors [170] developed R-Sentry to address this by analyzing the file traversal patterns of existing and future ransomware variants. By understanding these patterns, R-Sentry strategically places honey files across various folders to maximize detection efficiency. Empirical evidence shows that ransomware samples following identified traversal patterns are immediately detected. For example, in a test scenario with ten folders and 80 files, only 15 honey files were needed for effective detection. Although this system determines the optimal placement of honey files within the file system, it still suffers from vulnerability to random-access attacks.

Another challenge with deception-based detection methods is avoiding detection during random-access attacks. In these attacks, the attacker randomly accesses different parts of a system's data or files rather than following a predictable or sequential path. This randomness can make it more difficult for security measures, such as deception-based methods, to detect the attack. The unpredictability can reveal inconsistencies or patterns, exposing deceptive measures like honey files.

To overcome the limitation of random-access attacks, the authors [171] created a deception-based protection method for ransomware detection. Their method uses the spectral clustering algorithm to distribute decoy files evenly.

Their experiments used an artificial file system consisting of two identical primary subtrees with 19,426 files and 1,378 subdirectories. The researchers conducted experiments to assess the effectiveness of their proposed method, such as analyzing the ability to divide the file system and

the detection performance of the deception-based module. They use a spectral clustering algorithm to distribute decoy files evenly across the entire file system while preventing their clustering in specific, closely located areas. They also employed the spectral clustering algorithm to partition the entire file system tree into multiple subfile systems. Additionally, they monitored the file I/O activities on the deployed decoy files and promptly took mitigation measures if a decoy file was accessed or tampered with. The authors demonstrated that their deception-based protection method could safeguard a high percentage of files with a small number of decoy files. For instance, with 18 decoy files for three subfile systems, approximately 95% of the files were protected, and with 120 decoy files for 20 partitions, around 99% of the files were protected. They also observed a noticeable decrease in the number of encrypted files as the number of partitions increased. Additionally, the authors reported that the number of encrypted files dropped from 3,493 to 23, and the file system could be efficiently protected using only a small set of decoy files.

While the proposed ransomware detection method shows promise, several limitations warrant consideration. The study's relatively small sample size of 94 ransomware and 100 benign samples may not fully represent the diverse landscape of ransomware and legitimate programs in real-world scenarios. While controlled, they used an artificial file system for experimentation, which may not adequately capture the complexity and variability of actual user environments. The method's reliance on simulated random-access attacks rather than real-world sophisticated ransomware potentially limits its validated effectiveness against advanced threats. The approach's dependence on predictable file access patterns by ransomware may be challenged by highly targeted or sophisticated attacks that could evade decoy files. The method's effectiveness is closely tied to the initial partitioning of the file system, which may vary significantly across different computing environments, potentially impacting its consistent performance.

Rana et al. [172] identify a new ransomware attack vector in cyber-physical systems and propose an automated web defense mechanism; their experiments demonstrate the proposed approach on social media platforms using Selenium automation tools. They tested the defense mechanism on multiple browsers, achieving an 85% success rate in standard testing environments, which improved to 95% accuracy in headless applications. The study also presents a generalizable technique for detecting and preventing malicious web automation. These findings contribute to reducing the attack surface and offer insights into mitigating future ransomware threats.

AlMajali et al. [173] developed an adaptive ransomware detection method using a similarity-preserving hashing approach. Their method improves previous hashing techniques by rapidly dividing files into partitions, selectively hashing them, and dynamically analyzing changes within partitions to detect encryption while minimizing false positives.

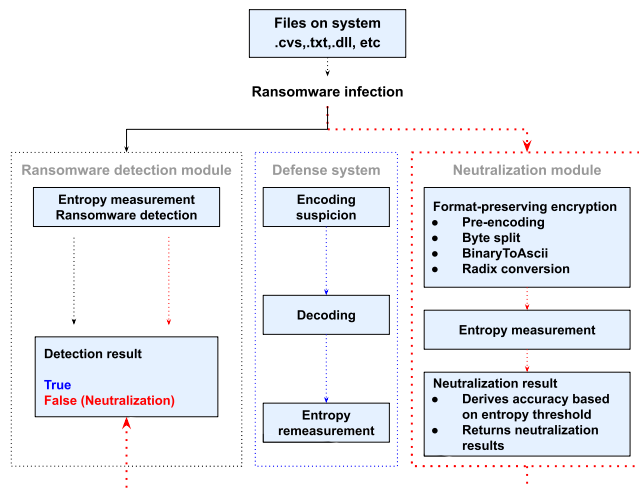


FIGURE 11. The proposed approach for neutralizing ransomware detection through format-preserving encryption is presented in [174].

The neutralization method introduced by Lee et al. [174] leverages format-preserving encryption to enhance ransomware detection capabilities. This approach aims to overcome the limitations identified in previous research, such as those discussed in [175] and [176]. Lee et al. utilized the GovDocs1 dataset for experimental validation in their study. This dataset encompasses a diverse range of file formats, including csv, txt, sys, dll, pdf, doc, docx, ppt, pptx, xls, xlsx, jpg, html, zip, c, and cpp. By testing their method on such a broad spectrum of file types, the authors sought to ensure the robustness and applicability of their technique across different scenarios. To manipulate entropy for ransomware detection, the authors developed three distinct techniques: Byte Split, BinaryToASCII, and Radix Conversion. Each technique offers a unique approach to transforming data, preserving the original format while altering the entropy to facilitate effective neutralization. The proposed method incorporates format-preserving encryption to maintain the integrity of the file formats while performing entropy manipulation. The efficacy of these techniques is illustrated in Fig. 11, which demonstrates the application and impact of each method in the context of ransomware neutralization.

The Radix Conversion technique was found to yield entropy values most closely resembling plaintext across various file formats when the radix was set to the following sequence: 16, 5, 8, 10, 16, 5, 16, 10, 16, 4, 16, 16, 16, and 16. In the results, the proposed neutralization method achieved a 96% improvement in neutralization accuracy, specifically for the PPTX file format. With an entropy threshold set at 0.5, this method demonstrated a 96% increase in neutralization precision compared to the previous approach for the same file format. Furthermore, the proposed method also outperformed earlier techniques in neutralizing various file formats, showing accuracy improvements ranging from 31% to 100% across different file types.

Potential limitations and challenges of this ransomware detection approach:

- **Entropy threshold dependency:** The method's effectiveness is tied to a specific entropy threshold (0.5), which may not be universally applicable across all ransomware variants and file types.
- **File format specificity:** The approach requires adjusting parameters (like radix values) for different file formats, which could be challenging to implement and maintain for a wide range of file types.
- **Potential false positives:** Format-preserving encryption might inadvertently flag legitimate encrypted files as potential ransomware.
- **Adaptability concerns:** Ransomware authors could potentially develop methods to circumvent this detection technique by mimicking the entropy patterns of plaintext files.
- **Performance impact:** The process of applying format-preserving encryption and entropy manipulation techniques may introduce computational overhead, potentially affecting system performance.
- **Limited scope:** The method focuses primarily on entropy-based detection, which may not be effective against ransomware that uses other obfuscation techniques or behavioral patterns.
- **Dataset limitations:** The study relies on the GovDocs1 dataset, which may not represent the full range of file types and structures encountered in real-world scenarios.

B. ANDROID RANSOMWARE

The authors [153] developed and introduced a Binary JAYA (BJAYA) algorithm tailored for detecting ransomware in Android mobile devices, employing an optimization-based approach with the BJAYA algorithm. Algorithm 1 shows the proposed JAYA algorithm.

The proposed JAYA algorithm [153] is designed for optimization tasks like the 0-1 knapsack problem. It follows these main steps:

- **Initialization:** Parameters such as population size and maximum iterations are set. Solutions are initialized randomly, and their fitness is evaluated based on constraints.
- **Modification Procedure:** Each solution in the swarm undergoes modification using JAYA operators. This involves updating the solutions based on the best and current solution positions in the search space, using sensitivity parameters to guide movement.
- **Mapping of Solutions:** Since JAYA is designed for continuous optimization, but the knapsack problem is binary, a mapping process is employed. The sigmoid function is a transfer function to convert continuous values to probabilities (0 or 1), facilitating operation in a discrete search space.

Algorithm 1 JAYA Algorithm [153]

```

1: Set parameters: population_size( $N$ ), and Max_iterations
2: Initialize the solutions ( $S_{i,j}$ )
3: Compute Fitness( $S_i$ ),  $i = 1, 2, \dots, N$ 
4: while iteration  $\leq$  Max_iterations do
5:   Identify the best solution ( $S_{\text{best}}$ )
6:   Identify the worst solution ( $S_{\text{worst}}$ )
7:   for  $i = 1$  to  $N$  do
8:     for  $j = 1$  to  $d$  do
9:       Set random1  $\in [0, 1]$ 
10:      Set random2  $\in [0, 1]$ 
11:       $\bar{S}_{i,j} = S_{i,j} + \text{random}_1 \times (S_{\text{best},j} - |S_{i,j}|) - \text{random}_2 \times$ 
         $(S_{\text{worst},j} - |S_{i,j}|)$ 
12:    end for
13:    if Fitness( $\bar{S}_i$ )  $\leq$  Fitness( $S_i$ ) then
14:       $\bar{S}_i \leq S_i$ 
15:    end if
16:  end for
17:  iteration = iteration + 1
18: end while

```

- **Position Updates:** After modifications, the solutions' fitness is recalculated. The solution is updated if a newly calculated fitness surpasses the previous one.
- **Stopping Criterion:** The modification, mapping, and updating process continues until the maximum iteration limit is reached.

They evaluated the algorithm's effectiveness against the 0–1 knapsack and real ransomware datasets. For the 0–1 knapsack problem, the authors used fifteen 0–1 knapsack dataset instances obtained from citeburkardt, [177], and [178]. The real ransomware dataset contained 500 applications of ransomware and 10,153 benign applications. The authors acknowledge the imbalanced nature of the ransomware dataset used in the study. The dataset contains a significant difference between the number of benign and ransomware applications, making it imbalanced. The authors addressed the imbalance by applying the SMOTE to mitigate the imbalance ratio. The results showed that the BJAYA algorithm surpassed other approaches, especially for small datasets. Moreover, in the context of the ransomware dataset, the BJAYA algorithm demonstrated superior performance compared to conventional classifiers, achieving sensitivity and Gmean values of 97% and 98.2%, respectively. The impact of the imbalanced dataset on the results is not explicitly discussed in the paper.

The authors [179] propose RansomSentry as a runtime detection system for Android to combat lock-screen and crypto ransomware. It instruments sensitive APIs during app installation using a modified Android dex2oat compiler and monitors screen and file operations at runtime to identify malicious activity. A prototype was tested on 2,376 recent Android ransomware samples, the authors claims that, unlike

prior methods, RansomSentry preserves app integrity by avoiding APK or bytecode modifications and resists code obfuscation, ensuring real-time protection.

Research on Android ransomware detection has led to innovative approaches for identifying and mitigating ransomware at various stages. For example, the authors [180] propose a novel method that intercepts communication between ransomware and its C&C server by analyzing the dynamic and static characteristics of Locker.CB!tr ransomware, they were able to impersonate the C&C server, enabling them to reverse the ransomware's disruptive actions, such as screen locking or file encryption.

Similarly, Gómez-Hernández and García-Teodoro [161] introduce a lightweight crypto-ransomware detection tool for Android devices, leveraging honeyfiles to trigger real-time alerts. Their method strategically places “trap” files throughout the file system. It uses Android's FileObserver service to monitor and detect suspicious file access patterns, achieving high detection accuracy without excessive resource usage.

Lastly, studies such as [163], [181], and [182] address the need for early-stage ransomware detection with their proposed “Weapon” mechanism. Instead of relying on post-infection behaviors, this approach identifies ransomware at the pre-operational stage by analyzing permissions and API call patterns in APK files. Using the MITRE ATT&CK framework to correlate observed behavior with known attack patterns, the “Weapon” tool achieved high detection rates, successfully identifying 89.82% of ransomware samples before they became active. This approach highlights the potential of preventive detection mechanisms in reducing ransomware impact on Android systems.

VII. RANSOMWARE DATASETS

High-quality datasets are essential for effectively training ML and DL models to detect ransomware. However, challenges in acquiring such datasets impact the performance of models. The scarcity of up-to-date and well-labeled datasets is a significant difficulty, given the rapid evolution of ransomware. Outdated datasets and those with poor labeling lead to inaccurate model learning, higher false positives, and negative detection rates.

Table 6 presents an overview of available ransomware datasets from various sources. It includes information about the publisher, date, number of samples, data format, and notes on how different models can utilize the datasets for analysis or training purposes. It is worth noting that while the table lists specific models like XGBoost, KNN, CNN, and VGG16 that have been used with these datasets, other models can potentially utilize the datasets, depending on the task and approach.

The Table 6 also highlights that datasets containing raw executable files (exe, dll, etc.) can be used for image-based classification with models like CNN or VGG16 after being converted into a byte sequence or a vector representation by reading the raw bytes of the file and reshaping or padding

TABLE 6. Available ransomware datasets include some with raw binaries, while others only include extracted features.

Source	Name	Publisher	Date	Samples	Data	Format
[131]	RISS Ransomware	RISS	2016	Ransomware: 582 Families: 11 Benign: 942	Cuckoo dynamic analysis	CSV
[183] [184]	ShieldFS	NECSTLab	2016	1.7 billion I/O request packets 2,245 distinct applications	File system interactions	N/S
[185]	RansomProber	WHU	2018	Android ransomware: 2,721 Families: 15	Android ransomware files	APK
[157]	RansIm-DS	SEL	2021	Android ransomware: 500 Android benign: 9500	228 API packages 161 Permissions	N/S
[122]	Ransomware Attacks	Kaggle	2021	Ransomware attacks: 616	Cloud based Ransomware attacks	XLSX
[110]	Open Repository	IEEE Dataport	2022	Ransomware: 70 Strains: 31	DNS requests TCP connections I/O operations	TXT PCAP
[166]	Android Ransomware Detection	Kaggle	2022	Android ransomware: 10 Families: 10 Android benign: 10	Network traffic records	CSV
[186]	Ransomware API calls	IEEE Dataport	2022	N/S	API call counts	JSON CSV
[187]	Dynamic Feature Dataset	MDPI	2022	Ransomware: 20 Benign: 20	Dynamic feature from Cuckoo sandbox	CSV
[188]	RanSAP	Exist on GitHub	2022	Ransomware: 7 Benign: 3	Storage access patterns	CSV
[189] [190] [191] [192]	IO_41Events_5Rounds HPC_5Events_7Rounds HPC_41Events_5Rounds IO_41Events_5Rounds	SyN Lab	2023	Ransomware: 22 Benign: 3	8 block I/O 41 HPC events 1092 block I/O 780 HPC events	TAB
[115]	Ransomware PE Header Feature Dataset	Mendeley	2023	Ransomware: 1023 Families: 25 Benign: 134	Headers of binary executable	CSV
[193]	PE Entropy	IEEE Dataport	2023	N/S	PE sections entropy	CSV
[194]	Android crypto system call	IEEE Dataport	2024	Android ransomware: 213 Families: 6 Android benign: 502	System call logs	LOG
[195] [196]	Ransomware Combined Structural Features	Mendeley	2024	Ransomware: 1023 Families: 25 Benign: 1134	Structural features extracted from binary executable	CSV
[197]	RanSMAP	Exist on GitHub	2025	Ransomware: 6 Families: 6 Benign: 6	Low-level storage and memory access patterns collected using a thin hypervisor	CSV

the byte sequence into a fixed-size 2D or 3D array. However, other models can also analyze them after appropriate feature extraction, such as extracting API calls or other relevant features from the PEs.

The datasets presented in Table 7 are not ransomware-specific, as they encompass various types of malware. These datasets require significant additional effort for proper analysis and filtering to isolate ransomware-specific samples. Furthermore, the composition of these datasets, predominantly containing executable files, presents substantial challenges for researchers. While these files offer valuable insights through dynamic analysis, their execution demands carefully controlled secure environments to prevent accidental malware outbreaks. Collecting and handling ransomware samples involves complex legal and ethical considerations, requiring strict adherence to cybersecurity laws and regulations to ensure responsible research practices.

A. DATASETS CHARACTERISTICS

We analyze the datasets presented in Table 6 to identify the key characteristics of available ransomware resources. These attributes illustrate the diversity and limitations of current datasets, providing insights crucial for effective research and detection efforts. By understanding these characteristics, researchers can make informed choices and address gaps in data collection.

1) DATA TYPES

- **Executable Files:** Several datasets, such as the *Ransomware Combined Structural Feature Dataset* [195], and the *PE Entropy Dataset* [193], provide binary executables and extracted features. These are invaluable for static analysis but require preprocessing.
- **Network Traffic:** The *Open Repository for Evaluating Ransomware Detection Tools* dataset [110] includes

TABLE 7. Available non-ransomware-specific datasets that may require analysis and filtering to extract ransomware samples.

Source	Name	Publisher	Date	Samples	Data	Format
[164]	AndMal 2020	UNB	2020	Android ransomware: 200,000 Families: 191 Android benign: 200,000	Malware/Benign apps	Mixed
[162]	Android Malware 2020	Exist on GitHub	2020	N/S	Android malware	ZIP
[198]	MalwSpecSys	IEEE Dataport	2022	N/S	IoT sensor system calls	TXT LOG
[199]	Malware Repository	theZoo	2023	N/S	Malware files	EXE DLL
[112]	VirusShare malware	VirusShare	2024	Malware: ~72,699,790	Malware files	Mixed
[200]	Hybrid Analysis	Hybrid Analysis	2024	N/S	Malware files	Mixed
[201]	VirusTotal	VirusTotal	2024	N/S	Malware/Benign files	Mixed
[202]	MalwareBazaar	abuse.ch	2024	Malware: ~834,652	Malware files	Mixed
[203]	Open Malware Database	VirusSign	2024	Malware: ~600,000,000	Malware files	Mixed
[118]	Free Malware Reports	ANY.RUN	2024	N/S	Malware files	Mixed
[159]	AndroZoo	Univ. of Luxembourg	2024	Apps: 24,795,935	Android apps	APK

network traffic data in formats like PCAP and TXT, capturing interactions like DNS requests and TCP connections. These are useful for identifying C&C communications.

- **API Calls:** The *Ransomware API Calls* dataset [186] focuses on API call logs in JSON and CSV formats, enabling behavioral analysis during ransomware execution.
- **I/O Request Packets (IRPs):** The *ShieldFS* dataset [183], [184] contains approximately 1.7 billion IRPs, useful for detecting filesystem-level encryption indicative of ransomware.
- **HPC Events:** Datasets from Harvard Dataverse [189], [190], [191], [192] provide low-level HPC event data for analyzing system behavior anomalies.
- **Feature Vectors:** The *RansIm-DS* dataset [157] offers 389 features, including API package counts and permissions, aiding in efficient model training.

2) SAMPLE SIZES

- **Large:** The *Ransomware Combined Structural Feature Dataset* [195] offers 1,023 ransomware and 1,134 benign samples. Such size is beneficial for robust model training.
- **Medium:** The *Android Crypto System Call* dataset [194] provides 213 Android ransomware and 502 benign samples, balancing comprehensiveness and manageability.
- **Small:** Datasets such as *RanSAP* [188] and Harvard Dataverse datasets [189], [190], [191], [192] include fewer samples, allowing for detailed analysis but limiting generalization.
- **Unspecified:** Some datasets, like *PE Entropy* [193], do not specify sample sizes, complicating evaluations.

3) TIME SPANS

- **Recent:** Datasets like *Ransomware Combined Structural Feature Dataset* [195] and *Android Crypto System Call* [194] include 2023 and 2024 samples, ensuring relevance to contemporary ransomware trends.
- **Historical:** The *ShieldFS* dataset [183], [184] and *RISS Ransomware Dataset* [131] cover earlier samples, offering historical perspectives on ransomware evolution.

4) SPECIFICITY

- **Ransomware-Specific:** The *Open Repository for Evaluating Ransomware Detection Tools* [110] focuses on 31 ransomware strains, enabling targeted detection.
- **General Malware:** Datasets like *MalwareBazaar* [202], *TheZoo malware* [199] and *VirusShare* [112] encompass broader malware categories.

5) DATA COLLECTION METHODS

- **Simulated Environments:** Datasets like *Dynamic Feature Dataset* [187] and *RansIm-DS* [157] are likely collected in controlled settings, ensuring experimental precision.
- **Aggregated Sources:** The *ShieldFS* [183], [184] dataset integrates data from real-world environments, capturing authentic system interactions.

6) BALANCE OF DATASETS

- **Balanced:** The *Android Crypto System Call* dataset [194] includes both benign and ransomware samples, enabling balanced classifier training.
- **Malware-Focused:** Datasets such as *PE Entropy* [193] and *ShieldFS* [183], [184] primarily consist of malicious samples, which can inform threat analysis.

7) FEATURE RICHNESS

- **Raw Data:** Datasets like *Ransomware Combined Structural Feature Dataset* [195] and *PE Entropy* [193] provide executable headers and entropy data, necessitating custom feature extraction.
- **Pre-Extracted Features:** The *Ransomware API Calls* dataset [186] offers API call counts, facilitating faster analysis.

B. DATASETS LIMITATIONS

The current ransomware datasets exhibit several critical limitations:

- 1) **Sample Size Disparity:** Many datasets suffer from limited sample sizes. For instance, the RanSMAP dataset [197] only contains 6 ransomware and 6 benign samples, while others like RISS [131] have moderate sizes with 582 ransomware samples. This limitation impacts the generalizability of models trained on these datasets.
- 2) **Feature Representation:** Most datasets provide preprocessed features rather than raw samples. For example, the SEL dataset [157] only provides API packages and permissions, while PE Entropy [193] focuses solely on PE section entropy values. This preprocessing limits researchers' ability to explore novel feature extraction approaches.
- 3) **Family Coverage:** The representation of ransomware families varies significantly. While some datasets like Moreira's [195] cover 25 families, others like RansomProber [185] focus on specific platforms with limited family coverage.
- 4) **Outdated Samples:** Many existing datasets contain outdated ransomware samples that may not reflect current attack techniques and evasion mechanisms. For instance, the RISS dataset [131] from 2016 and ShieldFS [183] may not capture modern ransomware behaviors, especially considering the rapid evolution of ransomware tactics and techniques. This limitation can affect the relevance and effectiveness of detection models trained on these datasets when faced with contemporary ransomware threats.
- 5) **Lack of Raw Files:** Most datasets fail to provide the original raw ransomware files, instead only offering preprocessed features or analysis results. For example, the Ransomware PE Header Feature Dataset [115] and PE Entropy dataset [193] only provide extracted features without access to the original malware binaries. This limitation prevents researchers from:
 - Conducting their analysis of the samples
 - Validating the feature extraction process
 - Exploring alternative feature extraction methods
 - Understanding the complete behavior of the ransomware
- 6) **Missing Extraction Methods:** Many datasets lack detailed documentation about their feature extraction

methodologies and procedures. While datasets like RISS [131] mention using Cuckoo dynamic analysis, they often do not provide the following:

- Specific configuration settings used during analysis
- Scripts or tools used for feature extraction
- Parameters and thresholds used in the extraction process
- Data cleaning and preprocessing steps

This lack of methodology transparency makes reproducing results or comparing findings across different studies difficult.

Based on the limitations identified in current ransomware datasets, it is clear that there is a significant need for a comprehensive and specialized ransomware dataset. This new dataset should include the original raw ransomware files, enabling researchers to extract their processed data and explore novel feature extraction techniques. However, the dataset should also provide preprocessed data to accommodate those who may not have access to VMs or sandboxes for analysis. The dataset must include static and dynamic data, capturing a wide spectrum of ransomware behaviors. Furthermore, the dataset should represent a broad range of ransomware families, covering at least 300 major families. It should encompass both Windows and Android operating systems to ensure it addresses a variety of platforms. In addition to ransomware, the dataset should not only contain ransomware samples but also include ransomware droppers and Trojan ransomware, as these variants play a crucial role in the infection chain and pose significant threats to cybersecurity. To enable supervised training of ML models, the dataset should also include a well-curated benign dataset comprising non-malicious samples from similar contexts. This is vital for building models that effectively distinguish between malicious and benign behaviors. The dataset should be regularly updated with new samples to reflect the evolving nature of ransomware. One possible way to achieve this is by opening it up to the community, allowing researchers and cybersecurity professionals to contribute new samples. Lastly, the dataset should be freely available and easily accessible to foster collaboration and accelerate advancements in ransomware detection and mitigation.

VIII. DISCUSSION

The landscape of ransomware detection has witnessed significant advancements, marked by a surge in research focused on integrating ML and DL models. This trend is observable across multiple studies, with a notable emphasis on the practical application of these models. The current approach often involves theoretical propositions, proposing models and methodologies without explicitly evaluating their detection rates, a crucial aspect of cybersecurity.

Table 4 presents a summary and comparison of state-of-the-art papers that target desktop ransomware to enhance their detection methods. Much research has been dedicated to

desktop ransomware attacks and their detection. Conversely, only a limited number of papers address Android ransomware attacks. This discrepancy can be attributed to ransomware developers' preference for desktop platforms, given their perceived vulnerability compared to mobile platforms.

Furthermore, within mobile platforms, a notable concentration of research is observed in the context of Android, as shown in Table 5, with minimal attention to other operating systems, such as Apple's iPhone Operating System (iOS). This preference arises from the inherent differences between the two ecosystems. A more closed environment characterizes iOS regarding app installation, and Apple maintains strict control over customization. In contrast, Android, developed by Google, is an open system that allows users to download apps from various sources. This potentially exposes users to a higher risk of ransomware attacks.

It is also observed that research papers utilize various datasets, with some creating their own due to limitations in acquiring a suitable ransomware dataset, as explained in Section VII.

Regarding detection methods, many research papers utilize traditional ML models for their systems, such as SVM, DT, RF, kNN, and SGD. This choice is influenced by several factors, including the small dataset available to researchers and the explainability of these models, where their output is predictable and not overly complicated.

A. EFFECTIVENESS AND CHALLENGES OF ML AND DL IN RANSOMWARE DETECTION

ML and DL approaches demonstrate potential in enhancing defenses against ransomware, chiefly owing to their capacity to acquire knowledge and adjust to emerging threats. Recent research has demonstrated the efficacy of ML in detecting ransomware by analyzing patterns in system behavior, network traffic, and file changes. These methods typically involve training classifiers on a dataset of known ransomware samples and benign applications to discern between the two. DL, a subset of ML, uses complex neural networks that can detect subtleties in large and high-dimensional data that traditional ML techniques may miss.

For instance, CNNs have been particularly useful in image recognition tasks and can be adapted for malware classification based on visualizing binary file content. RNNs and LSTMs, on the other hand, are adept at processing sequential data and have been applied successfully to detect ransomware based on sequential system call analysis; Table 8 comprehensively lists all ML and DL models featured in the reviewed literature, detailing their characteristics, advantages, and disadvantages.

1) REINFORCEMENT LEARNING (RL)

The ransomware detection methods primarily rely on supervised learning, which requires extensive labeled data for training. However, obtaining a diverse and representative dataset of ransomware samples can be challenging due to the constantly evolving nature of these threats. Moreover,

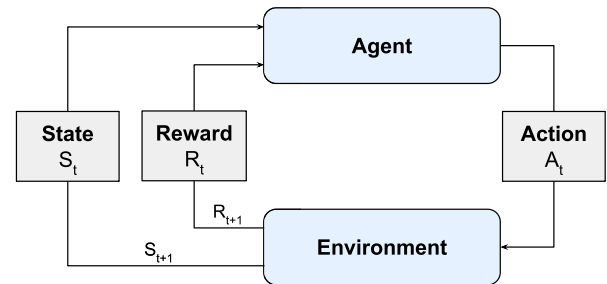


FIGURE 12. Basic framework of RL, illustrating agent-environment interaction through actions, state transitions, and rewards.

supervised learning models may struggle to adapt to new ransomware variants or detect unknown threats that deviate significantly from the training data.

In this context, RL can be a promising approach for ransomware detection and mitigation [111]. RL algorithms learn by interacting with an environment and receiving rewards or penalties based on their actions, allowing them to develop strategies without relying solely on labeled data. This adaptability makes RL particularly attractive for tackling ransomware attacks' dynamic and adversarial nature [204].

Fig. 12 shows a basic RL framework. The agent represents a trained ransomware detection model that observes the current state (S_t) of the system, which could include file activities, network traffic, and other system behaviors or sample data.

Based on this state, the agent takes an action (A_t), such as classifying the activities as benign or ransomware. The environment represents the computer system or network being monitored. Upon the agent's action, the environment transitions to a new state (S_{t+1}), reflecting the updated system activities, and provides a reward (R_{t+1}) to the agent, which could be a positive reward for correctly detecting ransomware or a negative reward for misclassification. This iterative process of observation, action, reward, and state transition allows the agent or the model to continuously learn and improve its detection capabilities through interactions with the environment.

Unfortunately, only two published papers utilize RL for ransomware detection. The first one [111] is analyzed in depth in Section V. The second paper [205] is not focused on ransomware detection methods; instead, the authors explore creating a ransomware simulator using RL to bypass anti-malware defenses and identify potential weaknesses in anti-ransomware solutions.

By reviewing RL applications in other domains [206], [207], some potential challenges that might apply to using RL for ransomware detection can be inferred:

- **Computational Complexity:** Implementing RL for ransomware detection requires significant computational resources to process and analyze large amounts of data in real-time [208], [209].

TABLE 8. Comparison of ML and DL Models for Ransomware Detection.

Model	Characteristics	Advantages	Disadvantages
Supervised, ML			
SVM	Kernel-based binary classification	High accuracy, Efficient for high-dimensional data	Sensitive to feature scaling, Closed box, interpretability can be difficult
DT	DT-based classification	Easy to interpret, Efficient for small datasets	Prone to overfitting, Sensitive to irrelevant features
RF	Ensemble of DT	High accuracy and robustness, Handles missing data well	Less interpretable than DTs, Can be computationally expensive for large datasets
kNN	Case-based reasoning for classification	Easy to implement, No training phase required	Performance degrades with high-dimensional data, Sensitive to noisy data
SGD	Gradient descent for optimization	Efficient for large datasets, Can be adapted to various tasks	Requires careful parameter tuning, Sensitive to data initialization
XGBoost	Gradient boosting framework for regression and classification	High accuracy and performance, Handles missing data well	Complex model, interpretability can be challenging, Requires careful feature engineering
GBM	Gradient Boosting Machine (GBM) for regression and classification	Similar to XGBoost, with efficient boosting algorithms	Can be computationally expensive, Overfitting possible if not tuned properly
NB	Probabilistic model based on Bayes' theorem	Efficient for large datasets, Easy to implement	Assumes independence of features, Not suitable for non-linear relationships
AB	AB for classification	Improves weak learners' performance, Robust to noisy data	Can be computationally expensive, Closed box, interpretability can be difficult
LDA	Linear dimensionality reduction for classification	Efficient for data visualization, Effective for small datasets	Assumes Gaussian distribution of data, Not suitable for complex relationships
DL, Neural Network			
ANN	Artificial neural network for various tasks	Learns complex non-linear relationships, Can handle large amounts of data	Requires large datasets for effective training, Maybe computationally expensive
DNN	Deep artificial neural network with multiple hidden layers	High accuracy and robustness for complex tasks	Requires even more data than ANNs, Prone to overfitting, Debugging, and interpretability can be difficult
LSTM	Recurrent neural network for sequential data	Captures temporal dependencies in data, Effective for analyzing network traffic or log files [149]	Requires careful parameter tuning, May be computationally expensive for long sequences
CNN	Convolutional neural network for image and sequence data	Effective for analyzing file headers or content patterns [149]	Requires large datasets of labeled files, May not be efficient for non-image data
Xception CNN	A variant of CNN for image classification	Improved accuracy and efficiency over standard CNNs	Requires larger amount of data than normal CNNs
GCNN	Graph convolutional neural network for analyzing network graphs	Can analyze relationships between files or hosts	Requires domain-specific knowledge for graph construction, May be computationally expensive for large networks
DBN	Deep Belief Network (DBN) for unsupervised feature learning	Can learn abstract features from unlabeled data	Requires pre-training phase before fine-tuning for specific tasks
FC network	Fully Connected (FC) neural network with one hidden layer	Simpler than deep neural networks, it may be easier to interpret	Less powerful than deep networks for complex tasks
DL, Reinforcement Learning			
DRL	Reinforcement learning for various tasks, often combined with deep learning	Can learn optimal strategies for complex situations, adapt to changing environments	Requires careful reward engineering, Can be computationally expensive and unstable
DDQN	Double Deep Q-Network (DDQN) for reinforcement learning tasks	Improves stability and performance of Deep Q-Network (DQN)	Requires even more data and training time than DQN
DL, Natural language processing (NLP)			
BERT	Pre-trained language model for various NLP tasks	Effective for analyzing ransom notes or communication logs	Requires domain-specific fine-tuning for ransomware detection, May be computationally expensive

- **Nonstationarity:** The behavior of ransomware can change over time, making it difficult for RL models to adapt and remain effective.
- **Observability:** In ransomware detection, not all aspects of the system's state may be observable, making it challenging for the RL model to make accurate decisions.
- **Data Diversity and Distribution Shifts:** The RL model needs to handle diverse types of ransomware and adapt to new variants that might not have been seen during training.

- **Efficiency and Stability:** For ransomware detection, it is crucial to have a stable model that can quickly adapt to new data without compromising performance.

2) HUMAN-IN-THE-LOOP (HITL)

Fig. 13 illustrates the concept of a HITL system. It depicts a process in which an ML model makes predictions or decisions but can incorporate human input or oversight.

The process starts with some input data or information. In the case of ransomware, this could be a set of PE files, API

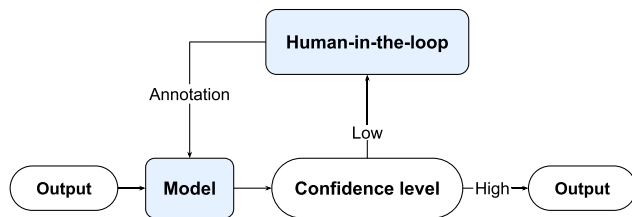


FIGURE 13. A HITL annotation system example, where the model outputs a confidence level that determines whether the decision is automated (high confidence) or requires human intervention (low confidence) for annotation and validation.

calls, network traffic logs, or any other relevant data sources. This input is fed into an ML model trained to detect potential ransomware threats.

The model analyzes the input data and generates an initial output or prediction and a confidence level associated with that prediction. This confidence level represents the model's certainty or uncertainty about its prediction. The confidence level can also be inferred from the model's output, depending on the specific technique used. For example, if the model uses a sigmoid activation function, an output value less than a predetermined threshold (e.g., 0.7) could indicate a low confidence level.

If the model's confidence level is high, meaning it is highly confident about its prediction, the output is considered reliable and can be directly acted upon. This could involve acting appropriately, such as blocking or quarantining the suspected ransomware threat.

However, if the model's confidence level is low, indicating uncertainty or ambiguity in its prediction, the process incorporates a "human-in-the-loop" step. In this step, a human expert or analyst reviews and annotates the input data, using their domain knowledge and expertise to aid decision-making.

The human annotation or input is then used to refine or retrain the ML model, improving its ability to make more accurate predictions in the future. This iterative process of incorporating human feedback and retraining the model aims to enhance the overall accuracy and effectiveness of the ransomware detection system.

The main difference between HITL and RL is that HITL involves explicit human input and guidance to train or refine a model, leveraging human expertise and knowledge. In contrast, RL is an autonomous learning approach where an agent learns by interacting with an environment and receiving reward signals without direct human involvement in the learning process.

However, it is worth noting that HITL and RL are not mutually exclusive. Both approaches can sometimes be combined, where human feedback or annotations are incorporated into an RL framework to improve the agent's learning or guide its exploration [210].

While HITL has been used for cybersecurity applications such as phishing detection [211] and malware classification [212], widespread adoption of HITL systems for

ransomware detection has not yet occurred due to numerous challenges facing HITL in general. Kumar et al. [213] identify HITL challenges as follows: ethical and legal considerations, human factors associated with involving humans in the loop, cost implications, validation of the system's outputs or decisions, overall system complexity, data quality issues, interpretability of the model's reasoning or decisions, and aspects of human-machine interaction related to usability and user experience.

3) HYBRID DEEP LEARNING

Hybrid deep learning combines multiple deep learning architectures to leverage their complementary strengths, offering robust solutions for complex classification tasks. CNNs excel in spatial feature extraction, while recurrent architectures, such as LSTM and GRU networks, capture temporal dependencies in data. By blending these approaches, hybrid models address the limitations of standalone techniques, enhancing accuracy and adaptability.

Although no explicit studies apply hybrid deep learning to ransomware detection, significant progress has been made in using this approach for malware detection, including Android malware. For example, The authors [214] combined CNNs and LSTMs for malware classification by converting malware binaries into grayscale images and applying PCA for dimensionality reduction. This method achieved high accuracy and efficiency in classifying malware families. Similarly, Venkatraman et al. [215] a hybrid deep learning and image-based approach for malware detection, leveraging similarity mining and deep learning architectures to classify obfuscated malware with high accuracy. Their method achieves nearly 99% accuracy while reducing computational cost compared to classical machine learning, and they suggest further improvements through extended training and integration with emerging image analysis techniques.

The authors [216] proposed AI-HydRa, integrating random forest classifiers with a deep learning model, achieving an improved detection rate with minimal false positives through a voting mechanism. The authors [217] introduced a hybrid model for Android malware detection, combining deep belief networks (DBN) for static features and GRU for temporal dynamic features, achieving higher detection accuracy, especially against obfuscated malware. Similarly, the authors [218] developed a hybrid framework, SHLMD, which utilized a merged sparse auto-encoder (MSAE) and stacked denoising auto-encoders (SDAE) for feature representation, achieving over 94% accuracy on Android malware datasets.

Drawing on these approaches, hybrid deep learning could be effectively adapted for ransomware detection. Ransomware's distinct characteristics, such as rapid data encryption and manipulation of file systems, make it well-suited for hybrid architectures. For instance, CNNs could analyze file system snapshots and activity logs for spatial anomalies. At the same time, LSTMs or GRUs could monitor temporal sequences of system calls or network traffic to detect

suspicious patterns indicative of ransomware behavior. Static and dynamic features can be processed separately, with PCA or similar methods enhancing computational efficiency and enabling real-time detection.

Despite its potential, hybrid deep learning for ransomware detection faces several limitations. First, the effectiveness of these models heavily relies on the availability of large, high-quality labeled datasets specific to ransomware, which are often difficult to obtain due to privacy concerns and the evolving nature of ransomware attacks. Additionally, hybrid models, combining components like CNNs and LSTMs, tend to be computationally intensive, requiring significant processing power and memory, which can limit their deployment in resource-constrained environments. Moreover, the interpretability of these models remains a challenge, as the integration of multiple architectures can make it difficult to understand the decision-making process, hindering trust and adoption in critical systems. Finally, ransomware's ability to rapidly evolve and employ obfuscation techniques may render hybrid models less effective against novel or zero-day threats, necessitating frequent retraining and updates to maintain detection accuracy.

4) TRANSFORMERS AND GENERATIVE AI

Transformers are a powerful deep learning architecture originally designed for natural language processing tasks. They leverage self-attention mechanisms to understand contextual relationships within data, enabling them to process input holistically rather than sequentially. Their adaptability has allowed them to excel across various domains, including image processing, time series analysis, and cybersecurity.

To the best of our knowledge, only two research papers specifically address the use of Transformers for ransomware detection: [129] and [151]. Additionally, we found extensive research leveraging Transformers for general malware detection, showcasing their potential to address broader cybersecurity challenges. Furthermore, two papers explore the intersection of generative AI and cybersecurity, one focusing on ransomware attacks [219], and the other discussing malware analysis [220].

In the context of ransomware detection using Transformers, Ashwini et al. [151] introduced an Enhanced Dual Vision Transformer (EDVT) integrated with the Mantis Search Split Attention Network (MSSAN) for identifying and classifying ransomware threats. Their framework incorporates advanced preprocessing and feature selection techniques, resulting in a near-perfect detection accuracy of 99.99%. Similarly, Gaber et al. [129] proposed Pulse, a Transformer-based framework trained on assembly language features captured by a Dynamic Binary Instrumentation tool. By relying on the contextual analysis of assembly instructions, Pulse effectively accurately identifies novel ransomware samples.

Regarding generative AI, Teichmann [219] explored how criminals could misuse generative models for planning and executing ransomware attacks. The study found that generative AI could enable individuals with limited IT exper-

tise to execute complex attacks and help skilled attackers create more effective phishing emails. The implications of these findings highlight the risks associated with the widespread availability of generative AI. On the other hand, Sun et al. [220] demonstrated the potential of generative AI in malware analysis by automating the translation of low-level malware behaviors into human-readable reports. This approach provides technical insights and actionable intelligence for cybersecurity professionals.

While these studies focus on specific applications, they collectively underline the transformative potential of Transformers and generative AI in cybersecurity. To enhance ransomware detection, we recommend further exploration of Transformer-based models combined with generative AI techniques. For example, generative AI could simulate ransomware behaviors, creating diverse datasets to train Transformers. Additionally, Transformer models could be fine-tuned to analyze ransomware behaviors in complex attack scenarios, such as those enabled by generative AI. Integrating these approaches could improve detection accuracy and provide insights into the evolving tactics of ransomware attackers.

5) ADVERSARIAL ML

Adversarial ML refers to a field within ML focused on studying and defending against intentional manipulations of input data designed to deceive ML models. These manipulations, known as adversarial examples, exploit vulnerabilities in ML algorithms, allowing attackers to bypass detection mechanisms or force incorrect predictions. In the context of ransomware detection, adversarial ML poses unique challenges, as adversaries can craft subtle modifications to evade detection systems that rely on ML models.

To the best of our knowledge, only three papers address the role of Adversarial ML in ransomware detection. Ahmed et al. [221] proposed a hybrid ensemble learning approach to mitigate adversarial evasion attacks in ransomware detection. By examining both static and dynamic ransomware features, including permissions, text, memory usage, system calls, and CPU activity, they combined ensemble learning classifiers to achieve robust classification performance. Their results demonstrated the ability of ensemble methods to detect unknown ransomware behavior effectively, highlighting their potential to counteract adversarial evasion techniques. Sewak et al. [204] extended this work by addressing the challenge of metamorphic ransomware, which alters its structure post-infection to evade detection. They proposed a deep counter-adversarial reinforcement learning system that learns to normalize structural transformations of advanced ransomware. To overcome the scarcity of suitable training data, they developed a framework to generate swarms of advanced ransomware, enabling their system to develop effective countermeasures against highly adaptive threats. Building on these ideas, Wang et al. [222] explored adversarial training to enhance deep learning models for ransomware family detection. Their approach combined

Cuckoo sandbox analysis with sequential deep neural networks to capture ransomware behavioral features and classify variants. By employing adversarial examples generated through Carlini and Wagner (CW) attacks during training, they improved the robustness of their models, enabling better detection of zero-day ransomware attacks.

Despite these advancements, several limitations remain. Adversarial ML techniques often require significant computational resources for training and validation, which may hinder their deployment in real-time ransomware detection scenarios. Moreover, the lack of comprehensive datasets for advanced ransomware, especially those exhibiting metamorphic behaviors, continues to pose a challenge. Recommendations for future work include creating standardized and diverse datasets to facilitate better training of adversarially robust models. Additionally, developing lightweight and efficient adversarial ML techniques that can operate in real-time environments is essential. These steps will enable broader adoption of adversarial ML-based approaches for ransomware detection and enhance their effectiveness against increasingly sophisticated threats.

B. LIMITATIONS OF CURRENT DETECTION METHODS

Both non-ML-based and ML-based ransomware detection methods have limitations. Fig. 14 summarizes these limitations and proposed potential solutions. This section thoroughly explores these limitations, examining each approach's challenges in detecting and addressing various issues.

1) NON-ML-BASED DETECTION METHODS LIMITATIONS

Signature-based detection methods can easily identify known threats based on established malicious code patterns. However, they cannot detect new, unknown ransomware (zero-day attacks) and require constant updates to recognize new ransomware variants.

Behavioral analysis can address the limitations of signature-based methods by monitoring system behavior in real-time. However, this can slow down systems, and ransomware might only be detected after it encrypts files, which can already cause damage. Additionally, code obfuscation remains challenging for detection systems that rely on static.

Ransomware can hide its true nature through code obfuscation, making it difficult for static analysis to detect, and it might miss behaviors that only appear during execution. Conti ransomware, for instance, employs dynamic API loading and API hashing to hide its true API usage from static analysis tools. It uses the LoadLibraryA API function from KERNEL32.DLL to load all other libraries dynamically during runtime [86].

Behavioral or dynamic analysis-based detection methods relying on VMs or sandboxes can also fail because ransomware authors can check for various indicators to detect VMs [223]. They look at registry key paths that show VM-specific hardware, drivers, or services and file system paths for VM-specific drivers or services [223]. They also check

for MAC addresses specific to certain VM infrastructures [223]. Virtual hardware can also be a clue, such as outdated network cards like the Intel e1000, which suggests using Quick EMUlator (QEMU) hardware [223]. Additionally, they examine running processes for VM platform-specific services, such as VMware tools [223].

Mitigating VM detection by ransomware is difficult because ransomware creators are familiar with popular virtualization platforms, allowing them easy access to these VMs to test their evasion tactics [223]. Sandboxes often rely on widely used systems such as Kernel-based Virtual Machine (KVM) and QEMU, complicating efforts to thwart this type of evasion.

A potential solution for this issue is to rely on real systems for ransomware dynamic analysis or develop a custom unknown hypervisor to virtualize the environment. However, relying on real systems poses challenges for offline detection, as samples must be transmitted to a remote server. While this approach could offer a solution, it also raises significant privacy concerns.

The deception-based protection method in [171] can overcome some behavioral or dynamic analysis-based detection methods. However, this approach requires creating and maintaining a complex file system with decoy files, which can be labor-intensive and prone to errors. The effectiveness of deception-based detection heavily relies on the proper distribution of decoy files. Misplacement can reduce the detection rate, especially since many ransomware authors are aware of such systems and attempt to avoid targeting decoy files and paths. Additionally, as the number of partitions and decoy files increases, managing and monitoring the system becomes more challenging.

2) ML-BASED DETECTION METHODS LIMITATIONS

Despite their effectiveness, ML and DL methods face inherent limitations in ransomware detection, such as the reliance on large and representative datasets for training. Many ransomware studies rely on limited, or outdated datasets, which challenge the reliability and generalizability of their findings [224]. Existing ransomware datasets have several key limitations:

- Activity-specific data: Many ransomware datasets focus on specific activities, such as API calls or I/O events, which limits the comprehensiveness of the data.
- Limited sample size: The number of samples in these datasets ranges from 30 to 15,000, which is considered low, especially for training DL models. This limitation can result in overfitting, where the model memorizes the dataset instead of learning generalizable patterns.
- Lack of ransomware variety: Many datasets contain samples from specific ransomware families, which reduces the diversity of the dataset and limits the ability to detect and analyze different types of ransomware.
- Lack of raw PE files: Many available ransomware datasets contain feature-extracted data without their

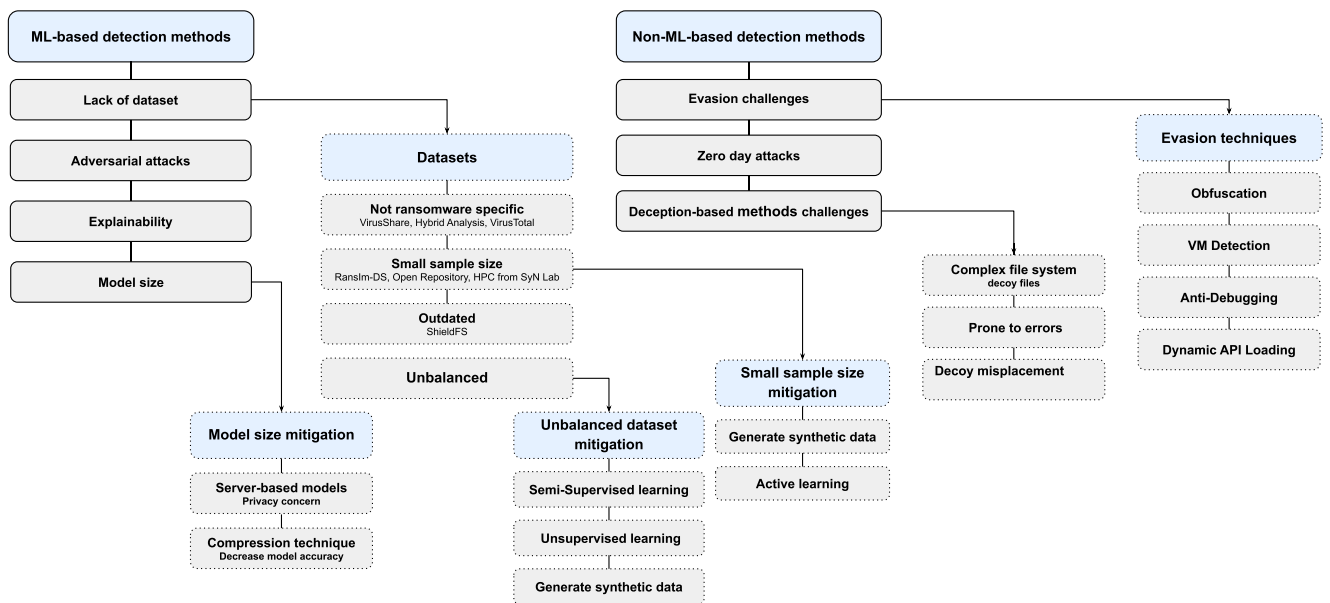


FIGURE 14. Graphical representation of detection methods challenges and limitations.

corresponding PE files, which restricts the ability to perform detailed binary analysis and limits the potential for discovering new features.

- Static and dynamic analysis limitations: As mentioned in the previous section, creating a feature-extracted ransomware dataset using static and dynamic analysis tools encounters the same limitations as non-ML detection methods, potentially missing nuanced or advanced ransomware behaviors.
- Inclusion of other malware: Some datasets that contain a huge number of malware samples, like VirusShare, VirusTotal, and MalwareBazaar, include various types of malware, not just ransomware, which can lead to mixed results and less accurate ransomware-specific analysis.
- Lack of benign dataset: A balanced dataset is crucial for supervised learning detection methods. The absence of a comprehensive benign dataset makes it challenging to train models effectively, leading to potential biases and inaccurate detection results.

To address challenges with datasets, advanced techniques such as Active Learning, Transfer Learning, Semi-Supervised Learning, and Unsupervised Learning can be employed. These methods enhance the exploration of dataset intricacies and improve model performance.

• Active Learning and Transfer Learning

- Active Learning: Efficiently trains models by selectively querying the most informative data points, reducing the need for extensive labeled data, which is crucial in ransomware detection where labeled data is scarce.
- Transfer Learning: Reuses a pre-trained model for a new but related task, saving resources when data is limited or training a model from scratch is computationally expensive.

• Semi-Supervised and Unsupervised Learning

- Semi-Supervised Learning: Bridges the gap between supervised and unsupervised learning by leveraging labeled and unlabeled data, especially useful in contexts like ransomware detection where labeled data is costly.
- Unsupervised Learning: Discovers hidden patterns and structures within data without explicit labels, making it valuable for categorizing and understanding complex datasets.

• Generate Synthetic Data

- Data Simulation: Creates realistic ransomware samples using techniques like GANs.
- Data Augmentation: Expands datasets by introducing variations to enhance model diversity. This includes:
 - * File-based augmentation: Modifies files directly by altering content, metadata, or compression methods.
 - * Network traffic augmentation: Manipulates network packet data to simulate altered traffic, including changes to IP addresses, port numbers, or packet payloads.

Ensuring that modifications align with real-world ransomware behavior is critical to avoid misleading models. Involving domain experts and integrating these techniques with Active Learning, Transfer Learning, and Semi-Supervised Learning is recommended for optimal results.

Collaborative efforts between cybersecurity entities, academia, and industry partners are crucial for compiling comprehensive, current, and well-labeled datasets tailored for ransomware detection. These efforts and synthetic data generation and augmentation techniques can improve data

quality and enhance model robustness and generalizability in real-world scenarios.

Having a comprehensive and diverse ransomware dataset can improve detection rates. However, it can also lead to larger model sizes, which present several challenges. Due to their size and resource requirements, larger models may be impractical to deploy on individual systems for offline use. This issue can be partially mitigated by using model compression techniques, although these may affect the accuracy of the models.

Another solution is to use server-based models, where the client sends sample data to a remote server containing the ML model for analysis. However, this approach raises privacy concerns, such as potentially exposing sensitive information during data transmission.

Another issue facing ML models is that adversaries can employ adversarial ML techniques to craft ransomware that intentionally avoids detection. This creates a cat-and-mouse game between attackers and defenders, as attackers continually adapt to evade detection while defenders strive to enhance their models to catch new threats.

There is also the issue of explainability, wherein complex DL models, often seen as “closed boxes,” make it difficult to interpret the reasons behind their predictions, leading to trust issues among cybersecurity professionals when deploying these solutions.

Furthermore, collecting ransomware samples for training poses ethical and legal challenges, as researchers must handle malicious software carefully to avoid unintended propagation or violation of privacy and legal norms.

While ML and DL have marked a paradigm shift in ransomware detection, offering high levels of effectiveness, significant obstacles remain to be addressed. However, with continued research and development and cross-sector collaboration, these advanced analytical techniques hold great promise for fortifying cybersecurity and adapting to the ever-changing landscape of digital threats.

C. PRACTICAL DEPLOYMENT CHALLENGES IN RANSOMWARE DETECTION SYSTEMS

Despite advancements in ransomware detection methodologies, practical deployment challenges remain understudied in current literature. Many existing papers focus on theoretical accuracy and algorithmic novelty but overlook critical operational constraints. This section synthesizes insights from ransomware detection frameworks and broader malware research to address scalability, latency, integration, and cloud compatibility gaps.

- **Latency and Real-Time Processing Constraints** Real-time detection is paramount in high-velocity environments where ransomware can encrypt critical data within seconds. While the Zero-Space Detection framework achieves sub-2-millisecond processing times through parallelized event-stream engines and multi-phase filtering [225], most ML-based methods (e.g.,

LSTM networks, ensemble models) incur significant latency due to feature extraction and model inference steps [226]. For instance, deep learning architectures like those [226] require extensive computational resources for temporal pattern analysis, which may not scale during peak network traffic. Non-ML methods like behavioral heuristics also struggle with latency when monitoring real-time system-wide activities. Few studies rigorously evaluate detection delays under realistic operational loads, leaving uncertainty about their viability in time-sensitive scenarios.

- **Scalability in Heterogeneous Environments** Scalability challenges arise in distributed systems with diverse file types, network protocols, and endpoint devices. The Zero-Space framework demonstrates adaptability through modular architecture and distributed computational nodes [225]. However, hybrid approaches combining static and dynamic analysis often face bottlenecks when processing large-scale datasets [226], [227]. For example, ML models trained on homogeneous datasets may fail to generalize across heterogeneous environments, such as IoT ecosystems or hybrid cloud infrastructures. Additionally, resource-constrained edge devices struggle to execute complex models like XGBoost or Random Forests, which demand substantial memory and processing power [226].
- **Integration with Legacy and Hybrid Cybersecurity Systems** Retrofitting modern detection frameworks into legacy systems remains a major hurdle. While Zero-Space emphasizes interoperability with existing infrastructures via adaptive configuration layers [225], many ML-based solutions require extensive reconfiguration of security information and event management (SIEM) systems. Though computationally efficient, signature-based tools lack APIs for integrating behavioral or ML-driven alerts, leading to siloed threat intelligence [227]. Furthermore, hybrid environments combining on-premises and cloud workloads often suffer from inconsistent data formats, complicating centralized threat analysis.
- **Resource Overhead in Resource-Constrained Environments** Resource efficiency is critical for deployment in edge devices or low-power systems. The Zero-Space framework reports minimal memory usage and computational overhead [225]. However, many deep learning models (e.g., CNN-LSTM hybrids) consume excessive GPU resources, making them impractical for real-time deployment [226], [228]. Non-ML methods, such as file entropy analysis, also incur overhead when scanning large volumes of data [229]. Few studies quantify trade-offs between detection accuracy and resource consumption, leaving practitioners to guess at hardware requirements.
- **False Alert Fatigue and Adversarial Resilience** High false positive rates plague both ML and non-ML methods. Heuristic approaches generate excessive alerts

due to rigid rule sets [227], while ML models trained on imbalanced datasets misclassify benign activities (e.g., bulk file compression) as malicious [226]. The Zero-Space framework mitigates this through ensemble learning and multi-phase filtering, reducing false positives to <5% [225]. However, adversarial attacks—such as GAN-generated ransomware behaviors that mimic benign patterns—can degrade ML model performance, exacerbating alert fatigue [228], [230]. Current research rarely addresses adversarial robustness in deployment contexts.

- Cloud-Native Compatibility Cloud environments introduce unique challenges, including dynamic scaling, multi-tenancy, and ephemeral workloads. Zero-Space's fault-tolerant design and adaptive configuration layers partially address these issues [225], but most frameworks assume static on-premises setups. For example, containerized applications in Kubernetes clusters require detection systems to operate at the orchestration layer, a scenario seldom tested in existing studies [227]. Additionally, cloud storage systems (e.g., AWS S3, Azure Blob) demand specialized file activity monitoring, which many behavioral models lack [226].

IX. CONCLUSION

The continuously evolving nature of ransomware poses an ever-increasing threat to information security, presenting challenges that demand innovative and effective solutions. This survey explored many ransomware detection methods, emphasizing the significant strides in applying ML and DL techniques. By examining the technologies, methodologies, and strategic approaches employed in recent research, we gain a panoramic view of the current cybersecurity landscape and the ingenious efforts to protect digital assets against ransomware.

This survey has explored traditional approaches and ML techniques for ransomware detection, noting their respective strengths and weaknesses. We noted the adaptability and predictive power of ML and DL methods, which stand at the forefront of identifying and mitigating ransomware attacks. The complex architectures and algorithms, ranging from DT and SVM to complex neural networks and ensemble models, underscore a diverse arsenal available to cybersecurity professionals. As we turn towards the future, it is apparent that the arms race against ransomware will continue to intensify, necessitating a proactive and dynamic approach to defense strategies. The rapidly advancing front of ransomware calls for ceaseless innovation in detection methods. The bridging of ML and DL techniques with traditional cybersecurity measures appears promising. This would harness AI's analytical prowess in conjunction with established defensive infrastructures. Issues of dataset quality, model explainability, and attackers' ongoing development of adversarial techniques compound the battle against these malicious threats.

Staying ahead in this duel will require persistent development and refinement of ML and DL models, vigorous research into new defensive strategies, and an unwavering commitment to collaboration across the cybersecurity community.

Advocacy for more accessible, up-to-date, and comprehensive datasets, promotion of standards for ethical practices in data handling, and encouragement to develop more transparent and interpretable models are essential.

A. FUTURE RESEARCH DIRECTIONS

To overcome current limitations and challenges, future research in ML and DL for ransomware detection is likely to focus on several areas. One important direction is the development of lightweight models that maintain high efficacy while being deployable on devices with limited computing power.

Transfer learning, where models trained on vast datasets can be fine-tuned with smaller, more specialized data, holds the potential to keep detection models up-to-date with minimal retraining. The concept of federated learning, wherein models are trained across multiple decentralized devices holding local data, also presents a promising pathway for enhancing privacy and scalability. Additionally, eXplainable AI (XAI) advancements are anticipated to make DL models more transparent and their decisions more straightforward to interpret.

Recent advances in ML, such as Transformers, adversarial machine learning (AML), and hybrid learning models, are also expected to play a significant role in ransomware detection and mitigation. With their ability to process sequential data efficiently, transformers could enhance behavioral analysis for detecting ransomware patterns in real time. AML techniques will be crucial in improving the robustness of models against adversarial attacks, ensuring that ransomware cannot easily evade detection mechanisms. Hybrid learning models integrating multiple ML paradigms, such as combining supervised and unsupervised learning approaches, may enhance detection accuracy by capturing known and emerging ransomware variants.

Another pressing challenge is the availability of high-quality datasets specifically tailored for ransomware detection. Constructing comprehensive and representative datasets encompassing diverse ransomware families and attack vectors is essential for training robust ML models. Collaborative efforts between academia, industry, and government agencies are necessary to facilitate data sharing and the creation of standardized datasets that reflect real-world ransomware threats.

In the arms race against ransomware, integrating AI with traditional cybersecurity measures, such as firewalls and intrusion detection systems, may offer a more robust defense. Lastly, fostering interdisciplinary collaboration among researchers, practitioners, and policymakers will ensure that ML and DL methodologies remain at the forefront of ransomware detection and response.

REFERENCES

- [1] X. Ding and W. Feng, "Anomaly detection method based on feature mining for wireless sensor networks," *Int. J. Sens. Netw.*, vol. 36, no. 3, pp. 167–173, 2021.
- [2] N. Scaife, H. Carter, P. Traynor, and K. R. B. Butler, "CryptoLock (and drop It): Stopping ransomware attacks on user data," in *Proc. IEEE 36th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jun. 2016, pp. 303–312.
- [3] K. Wang, M. Tong, J. Pang, J. Wang, and W. Han, "XRAD: Ransomware address detection method based on Bitcoin transaction relationships," *ACM Trans. Web*, vol. 18, no. 4, pp. 1–33, Oct. 2024.
- [4] M. M. Ahmadian and H. R. Shahriari, "2entFOX: A framework for high survivable ransomware detection," in *Proc. 13th Int. Iranian Soc. Cryptol. Conf. Inf. Secur. Cryptol. (ISCISC)*, Sep. 2016, pp. 79–84.
- [5] J. Song, R. Paul, J. Yun, H. Kim, and Y. Choi, "CNN-based anomaly detection for packet payloads of industrial control system," *Int. J. Sens. Netw.*, vol. 36, no. 1, pp. 36–49, 2021.
- [6] A. Young and M. Yung, "Cryptovirology: Extortion-based security threats and countermeasures," in *Proc. IEEE Symp. Secur. Privacy*, May 1996, pp. 129–140.
- [7] A. Gazet, "Comparative analysis of various ransomware virii," *J. Comput. Virol.*, vol. 6, no. 1, pp. 77–90, Feb. 2010.
- [8] N. Andronio, S. Zanero, and F. Maggi, "HelDroid: Dissecting and detecting mobile ransomware," in *Research in Attacks, Intrusions, and Defenses*, H. Bos, F. Monroe, and G. Blanc, Eds., Berlin, Germany: Springer, 2015, pp. 382–404.
- [9] B. N. Giri, N. Jyoti, and M. Avert, *The Emergence of Ransomware*. Auckland, New Zealand: AVAR, 2006.
- [10] S. Alzahrani, Y. Xiao, and S. Asiri, "Conti ransomware development evaluation," in *Proc. ACM Southeast Conf.*, Apr. 2023, pp. 39–46.
- [11] B. Cruz, D. Gupta, A. Kapoor, L. Haifei, D. McLean, and F. Moreno. (2014). *McAfee Labs Threats Report*. McAfee, Santa Clara, CA, USA. [Online]. Available: <http://www.mcafee.com/us/resources/reports/rp-quarterlythreat-q1-2014.pdf>
- [12] (Jan. 2015). *Ransomware on the Rise*. Accessed: Oct. 7, 2023. [Online]. Available: <https://www.fbi.gov/news/stories/ransomware-on-the-rise>
- [13] B. Wrozek. (May 2023). *Protecting Against the Top Cybersecurity Threats in 2023 Requires a Balanced Approach*. [Online]. Available: <https://www.forrester.com/blogs/protecting-against-the-top-cybersecurity-threats-in-2023-requires-a-balanced-approach>
- [14] H. Alshaikh, H. Ahmed, and N. Ramadan, "Crypto-ransomware detection and prevention techniques and tools a survey," *Int. J. Comput. Digit. Syst.*, vol. 14, no. 1, pp. 10299–10308, Oct. 2023.
- [15] S. Ghayyad, S. Du, and A. Kurien, "The flaws of Internet of Things (IoT) intrusion detection and prevention schemes," *Int. J. Sensor Netw.*, vol. 38, no. 1, pp. 25–36, 2022.
- [16] (Sep. 2021). *Conti Ransomware*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.cisa.gov/uscert/ncas/alerts/aa21-265a>
- [17] J. Ferdous, R. Islam, A. Mahboubi, and M. Z. Islam, "AI-based ransomware detection: A comprehensive review," *IEEE Access*, vol. 12, pp. 136666–136695, 2024.
- [18] Y. M. Al-Awadi, A. Baydoun, and H. Ur Rehman, "Can windows 11 stop well-known ransomware variants? An examination of its built-in security features," *Appl. Sci.*, vol. 14, no. 8, p. 3520, Apr. 2024.
- [19] M. A. Alvi and Z. Jalil, "RansomGuard: A framework for proactive detection and mitigation of cryptographic windows ransomware," *J. Comput. Virol. Hacking Techn.*, vol. 20, no. 4, pp. 867–884, Sep. 2024.
- [20] S. Sharma, R. Kumar, and C. Rama Krishna, "A survey on analysis and detection of Android ransomware," *Concurrency Comput., Pract. Exper.*, vol. 33, no. 16, p. 6272, Aug. 2021.
- [21] D. K. K. Reddy, J. Nayak, H. S. Behera, V. Shanmuganathan, W. Viriyasitavat, and G. Dhiman, "A systematic literature review on swarm intelligence based intrusion detection system: Past, present and future," *Arch. Comput. Methods Eng.*, vol. 31, no. 5, pp. 2717–2784, Jul. 2024.
- [22] J. H. Park, S. K. Singh, M. M. Salim, A. E. Azzaoui, and J. H. Park, "Ransomware-based cyber attacks: A comprehensive survey," *J. Internet Technol.*, vol. 23, no. 7, pp. 1557–1564, 2022.
- [23] K. Begovic, A. Al-Ali, and Q. Malluhi, "Cryptographic ransomware encryption detection: Survey," *Comput. Secur.*, vol. 132, Sep. 2023, Art. no. 103349.
- [24] A. A. M. A. Alwashali, N. A. A. Rahman, and N. Ismail, "A survey of ransomware as a service (RaaS) and methods to mitigate the attack," in *Proc. 14th Int. Conf. Develop. eSystems Eng. (DeSE)*, Dec. 2021, pp. 92–96.
- [25] N. Rani, S. Dhavale, A. Singh, and A. Mehra, "A survey on machine learning-based ransomware detection," in *Proc. 7th Int. Conf. Math. Comput.*, D. Giri, K.-K. R. Choo, S. Ponnusamy, W. Meng, S. Akleyek, and S. P. Maity, Eds., Singapore: Springer, 2022, pp. 171–186.
- [26] R. Moussaileb, N. Cuppens, J.-L. Lanet, and H. L. Boudier, "A survey on windows-based ransomware taxonomy and detection mechanisms," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–36, Jul. 2021.
- [27] M. N. Olaimat, M. Aizaini Maarof, and B. A. S. Al-Rimy, "Ransomware anti-analysis and evasion techniques: A survey and research directions," in *Proc. 3rd Int. Cyber Resilience Conf. (CRC)*, Jan. 2021, pp. 1–6.
- [28] U. Urooj, B. A. S. Al-Rimy, A. Zainal, F. A. Ghaleb, and M. A. Rassam, "Ransomware detection using the dynamic analysis and machine learning: A survey and research directions," *Appl. Sci.*, vol. 12, no. 1, p. 172, Dec. 2021.
- [29] A. Alqahtani and F. T. Sheldon, "A survey of crypto ransomware attack detection methodologies: An evolving outlook," *Sensors*, vol. 22, no. 5, p. 1837, Feb. 2022.
- [30] H. Oz, A. Aris, A. Levi, and A. S. Uluagac, "A survey on ransomware: Evolution, taxonomy, and defense solutions," *ACM Comput. Surv.*, vol. 54, no. 11, pp. 1–37, Sep. 2022.
- [31] F. Aldauji, O. Batarfi, and M. Bayousef, "Utilizing cyber threat hunting techniques to find ransomware attacks: A survey of the state of the art," *IEEE Access*, vol. 10, pp. 61695–61706, 2022.
- [32] A. Alraizza and A. Algarni, "Ransomware detection using machine learning: A survey," *Big Data Cognit. Comput.*, vol. 7, no. 3, p. 143, Aug. 2023.
- [33] S. Razaulla, C. Fachkha, C. Markarian, A. Gawanmeh, W. Mansoor, B. C. M. Fung, and C. Assi, "The age of ransomware: A survey on the evolution, taxonomy, and research directions," *IEEE Access*, vol. 11, pp. 40698–40723, 2023.
- [34] L. Moujoud, M. Ayache, and A. Belmekki, "A state-of-the-art survey on ransomware detection using machine learning and deep learning," in *Modern Artificial Intelligence and Data Science: Tools, Techniques and Systems*. Berlin, Germany: Springer, 2023, pp. 183–200.
- [35] N. Ahmad, S. Ghosh, and J. K. Rout, "CNN-based hybrid deep learning framework for human activity classification," *Int. J. Sensor Netw.*, vol. 44, no. 2, pp. 74–83, 2024.
- [36] J. Isphahany, M. R. Islam, M. Z. Islam, and M. A. Khan, "Ransomware detection using machine learning: A review, research limitations and future directions," *IEEE Access*, vol. 12, pp. 68785–68813, 2024.
- [37] G. Magazine. (2021). *Held to Ransom*. Accessed: Oct. 7, 2023. [Online]. Available: <https://www.legal500.com/gc-magazine/feature/held-to-ransom>
- [38] Sophos. (2022). *Ransomware hit 66 'State of Ransomware 2022'*. Accessed: Oct. 7, 2023. [Online]. Available: <https://www.sophos.com/en-us/press/press-releases/2022/04/ransomware-hit-66-percent-of-organizations-surveyed-for-sophos-annual-state-of-ransomware-2022>
- [39] R. Davies. (Feb. 2023). *'All We Have Had is Losses': Royal Mail Dismisses 'Absurd' \$80m Ransom Demand*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.theguardian.com/business/2023/feb/15/under-no-circumstances-will-we-pay-that-absurd-amount-royal-mail-tells-hackers>
- [40] NextGov. (May 2023). *US Marshals Unveil Fully Reconstituted System Following Ransomware Attack*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.nextgov.com/cybersecurity/2023/05/us-marshals-unveil-fully-reconstituted-system-following-ransomware-attack/385866>
- [41] B. Computer. (2023). *Mcna Dental Data Breach Impacts 89 Million People After Ransomware Attack*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.bleepingcomputer.com/news/security/mcna-dental-data-breach-impacts-89-million-people-after-ransomware-attack>
- [42] APNews. (2023). *Schools Ransomware Data Breach*. Accessed: Jan. 1, 2024. [Online]. Available: <https://apnews.com/article/schools-ransomware-data-breach-40ebda010158f04a1ef14607bfed9b0>
- [43] T. Record. (2023). *Capita Says Responding to Ransomware Attack Will Cost up to \$20 Million*. Accessed: Jan. 1, 2024. [Online]. Available: <https://therecord.media/capita-ransomware-incident-response-cost>
- [44] H. Journal. (2023). *Lehigh Valley Health Network Sued After Ransomware Gang Publishes Nude Patient Images*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.hipaajournal.com/lehigh-valley-health-network-sued-after-ransomware-gang-publishes-nude-patient-images>
- [45] BlackFog. (2023). *What We Know About the Moveit Exploit and Ransomware Attacks*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.blackfog.com/what-we-know-about-the-moveit-exploit>

- [46] C. News. (2023). *Dallas: 26,000 People Impacted Ransomware Attack*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.cbsnews.com/texas/news/dallas-26000-people-impacted-ransomware-attack>
- [47] C. Page. (Jul. 2023). *UK Battles Hacking Wave as Ransomware Gang Claims 'Biggest Ever' NHS Breach*. Accessed: Jan. 1, 2024. [Online]. Available: <https://techcrunch.com/2023/07/10/uk-hacks-public-sector-nhs-ransomware>
- [48] D. Reading. (2023). *Illinois Hospital Closure Showcases Ransomware's Existential Threat*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.darkreading.com/cyberattacks-data-breaches/illinois-hospital-closure-ransomware-existential-threat>
- [49] A. C. S. Magazine. (2023). *Big Four Bank Data Lost in HWL Ebsworth Cyber-Attack*. Accessed: Jan. 1, 2024. [Online]. Available: <https://australiancybersecuritymagazine.com.au/big-four-bank-data-lost-in-hwl-ebsworth-cyber-attack>
- [50] B. Computer. (2023). *Tsmc Denies Lockbit Hack As Ransomware Gang Demands \$70 Million*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.bleepingcomputer.com/news/security/tsmc-denies-lockbit-hack-as-ransomware-gang-demands-70-million>
- [51] (Aug. 2023). *Tampa General Hospital Attracts a Class Action Lawsuit Over July Data Breach*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.teiss.co.uk/news/tampa-general-hospital-attracts-a-class-action-lawsuit-over-july-data-breach-12682>
- [52] (Aug. 2023). *Cloudnordic, Azerocloud Lose All Data in Huge Ransomware Attack*. Accessed: Jan. 1, 2024. [Online]. Available: <https://techmonitor.ai/technology/cybersecurity/ransomware-attack-on-cloudnordic-azerocloud-loses-all-data>
- [53] (Aug. 2023). *Prospect Medical Holdings Said Cyber Attack Impacted Multiple Hospital Networks*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.teiss.co.uk/news/prospect-medical-holdings-said-cyber-attack-impacted-multiple-hospital-networks-12674>
- [54] A. Murphy. (2023). *Caesars Entertainment Reportedly Paid Ransomware Demand*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.foxbusiness.com/markets/caesars-entertainment-reportedly-paid-ransomware-demand>
- [55] S. R. Kelleher. (Sep. 2023). *Ransomware Attack on Mgm Resorts*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.forbes.com/sites/suzannerowankelleher/2023/09/13/ransomware-attack-mgm-resorts>
- [56] B. Computer. (2023). *Building Automation Giant Johnson Controls Hit By Ransomware Attack*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.bleepingcomputer.com/news/security/building-automation-giant-johnson-controls-hit-by-ransomware-attack>
- [57] A. Martin. (2023). *UK Logistics Firm Blames Ransomware Attack for Insolvency, 730 Redundancies*. Accessed: Jan. 1, 2024. [Online]. Available: <https://therecord.media/knp-logistics-ransomware-insolvency-uk>
- [58] A. Mascellino. (2023). *Ardent Health Services Grapples With Ransomware Disruption*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.infosecurity-magazine.com/news/ardent-health-services-ransomware>
- [59] C. Riotta. (2023). *Report Details Aftermath of ICBC Lockbit Ransomware Attack*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.govinfosecurity.com/report-details-aftermath-icbc-lockbit-ransomware-attack-a-23655>
- [60] R. Y. Seetharamarao, "A unified approach towards security audit and compliance in cloud computing environment," in *Proc. 16th Int. Conf. Develop. eSystems Eng. (DeSE)*, Dec. 2023, pp. 623–629.
- [61] J. Saleem, B. Adebisi, R. Ande, and M. Hammoudeh, "A state of the art survey—Impact of cyber attacks on SME's," in *Proc. Int. Conf. Future Neww. Distrib. Syst.*, Cambridge, U.K., Jul. 2017, Paper 52, doi: [10.1145/3102304.3109812](https://doi.org/10.1145/3102304.3109812).
- [62] Monika, P. Zavarsky, and D. Lindskog, "Experimental analysis of ransomware on windows and Android platforms: Evolution and characterization," *Proc. Comput. Sci.*, vol. 94, pp. 465–472, Jan. 2016.
- [63] N. Sharma and R. Shanker, "Analysis of ransomware attack and their countermeasures: A review," in *Proc. Int. Conf. Electron. Renew. Syst. (ICEARS)*, Mar. 2022, pp. 1877–1883.
- [64] Somya, P. Bansal, and T. Ahmad, "Methods and techniques of intrusion detection: A review," in *Proc. Smart Trends Inf. Technol. Comput. Commun., 1st Int. Conf. SmartCom*, Jaipur, India. Singapore: Springer, Aug. 2016, pp. 518–529, doi: [10.1007/978-981-10-3433-6_62](https://doi.org/10.1007/978-981-10-3433-6_62).
- [65] S. G. Gohwong, "The state of the art of cryptography-based cyber-attacks," *Int. J. Crime, Law Social Issues*, vol. 6, no. 2, pp. 24–34, 2019, doi: [10.2139/ssrn.3546334](https://doi.org/10.2139/ssrn.3546334).
- [66] S.-C. Hsiao and D.-Y. Kao, "The static analysis of WannaCry ransomware," in *Proc. 20th Int. Conf. Adv. Commun. Technol. (ICACT)*, Feb. 2018, pp. 153–158.
- [67] S. N. K., "Ant colony optimization based light weight binary search for efficient signature matching to filter ransomware," *Appl. Soft Comput.*, vol. 111, Nov. 2021, Art. no. 107635.
- [68] B. Sanz, I. Santos, X. Ugarte-Pedrero, C. Laorden, J. Nieves, and P. G. Bringas, "Anomaly detection using string analysis for Android malware detection," in *Proc. Int. Joint Conf. SOCO-CISIS-ICEUTE*, Salamanca, Spain. Cham, Switzerland: Springer, Aug. 2013, pp. 469–478, doi: [10.1007/978-3-319-01854-6_48](https://doi.org/10.1007/978-3-319-01854-6_48).
- [69] Z. Chen, E. Brophy, and T. Ward, "Malware classification using static disassembly and machine learning," 2021, *arXiv:2201.07649*.
- [70] S. Cesare, Y. Xiang, and W. Zhou, "Control flow-based malware VariantDetection," *IEEE Trans. Dependable Secure Comput.*, vol. 11, no. 4, pp. 307–317, Jul. 2014.
- [71] J. Xu, A. H. Sung, P. Chavez, and S. Mukkamala, "Polymorphic malicious executable scanner by API sequence analysis," in *Proc. 4th Int. Conf. Hybrid Intell. Syst.*, Mar. 2005, pp. 378–383.
- [72] J. Aycock, "Anti-virus techniques," in *Computer Viruses and Malware*. Boston, MA, USA: Springer, 2006, pp. 53–95, doi: [10.1007/0-387-34188-9_4](https://doi.org/10.1007/0-387-34188-9_4).
- [73] Y.-S. Choi, I.-K. Kim, J.-T. Oh, and J.-C. Ryou, "PE file header analysis-based packed PE file detection technique (PHAD)," in *Proc. Int. Symp. Comput. Sci. Appl.*, Oct. 2008, pp. 28–31.
- [74] J.-Z. Li, Q. Yin, L.-H. Jiang, and X.-H. Jia, "Analysis of cryptographic algorithms' characters in binary file," in *Proc. 13th Int. Conf. Parallel Distrib. Comput., Appl. Technol.*, Dec. 2012, pp. 219–223.
- [75] X. Zhao, J. Luan, and M. Wolff, "Evaluating randomness in cyber attack textual artifacts," in *Proc. APWG Symp. Electron. Crime Res. (eCrime)*, Jun. 2016, pp. 1–5.
- [76] A. Moser, C. Kruegel, and E. Kirda, "Limits of static analysis for malware detection," in *Proc. 33rd Annu. Comput. Secur. Appl. Conf. (ACSAC)*, Dec. 2007, pp. 421–430.
- [77] H. Rana and M. Stamp, "Hunting for pirated software using metamorphic analysis," *Inf. Secur. Journal: A Global Perspective*, vol. 23, no. 3, pp. 68–85, May 2014.
- [78] A. Jeyashankar. (2022). *Most Common Malware Obfuscation Techniques*. Accessed: Jun. 27, 2024. [Online]. Available: <https://www.socinvestigation.com/most-common-malware-obfuscation-techniques>
- [79] D.-Y. Kao and S.-C. Hsiao, "The dynamic analysis of WannaCry ransomware," in *Proc. 20th Int. Conf. Adv. Commun. Technol. (ICACT)*, Feb. 2018, pp. 159–166.
- [80] K. Nakao, K. Yoshioka, D. Inoue, and M. Eto, "A novel concept of network incident analysis based on multi-layer observations of malware activities," in *Proc. 2nd Joint Workshop Inf. Secur. (JWIS)*, 2007, pp. 267–279.
- [81] W. Cui, R. H. Katz, and W.-T. Tan, "Design and implementation of an extrusion-based break-in detector for personal computers," in *Proc. 21st Annu. Comput. Secur. Appl. Conf. (ACSAC)*, 2005, pp. 361–370.
- [82] W.-J. Li, K. Wang, S. J. Stolfo, and B. Herzog, "Fileprints: Identifying file types by n-gram analysis," in *Proc. 6th Annu. IEEE Syst., Man Cybern. (SMC) Inf. Assurance Workshop*, Jun. 2005, pp. 64–71.
- [83] M. Spreitzenbarth, T. Schreck, F. Echter, D. Arp, and J. Hoffmann, "Mobile-sandbox: Combining static and dynamic analysis with machine-learning techniques," *Int. J. Inf. Secur.*, vol. 14, no. 2, pp. 141–153, Apr. 2015.
- [84] Y. Dai, H. Li, Y. Qian, and X. Lu, "A malware classification method based on memory dump grayscale image," *Digit. Invest.*, vol. 27, pp. 30–37, Dec. 2018.
- [85] M. Sikorski and A. Honig, *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. Burlingame, CA, USA: Starch Press, 2012.
- [86] S. Alzahrani, Y. Xiao, and W. Sun, "An analysis of conti ransomware leaked source codes," *IEEE Access*, vol. 10, pp. 100178–100193, 2022.
- [87] Y.-J. Lee, J. Lee, D. Ryu, H. Park, and D. Shin, "Clop ransomware in action: A comprehensive analysis of its multi-stage tactics," *Electron.*, vol. 13, no. 18, p. 3689, Sep. 2024.
- [88] K. Chimmanee and S. Jantavongso, "Digital forensic of maze ransomware: A case of electricity distributor enterprise in ASEAN," *Exp. Syst. Appl.*, vol. 249, Sep. 2024, Art. no. 123652.
- [89] A. Robinson, C. Corcoran, and J. Waldo, "New risks in ransomware: Supply chain attacks and cryptocurrency," Harvard Kennedy School, Belfer Center Sci. Int. Affairs, Cambridge, MA, USA, Tech. Rep., May 2022. [Online]. Available: <https://www.belfercenter.org/publication/new-risks-ransomware-supply-chain-attacks-and-cryptocurrency>

- [90] H. Kath. (2021). *What is Ransomware-as-a-Service (RaaS)? | Goanywhere MFT*. Accessed: Aug. 24, 2024. [Online]. Available: <https://www.goanywhere.com/blog/what-is-ransomware-as-a-service>
- [91] E. Kost. (2023). *What is Ransomware As a Service (RaaS)? The Dangerous Threat to World Security | Upguard*. Accessed: Jan. 1, 2024. [Online]. Available: <https://www.upguard.com/blog/what-is-ransomware-as-a-service>
- [92] S. Asiri, Y. Xiao, S. Alzahrani, S. Li, and T. Li, "A survey of intelligent detection designs of HTML URL phishing attacks," *IEEE Access*, vol. 11, pp. 6421–6443, 2023.
- [93] M. Paquet-Clouston, B. Haslhofer, and B. Dupont, "Ransomware payments in the Bitcoin ecosystem," *J. Cybersecurity*, vol. 5, no. 1, Jan. 2019, Art. no. tyz003.
- [94] Z. Wang, X. Wu, C. Liu, Q. Liu, and J. Zhang, "RansomTracer: Exploiting cyber deception for ransomware tracing," in *Proc. IEEE 3rd Int. Conf. Data Sci. Cyberspace (DSC)*, Jun. 2018, pp. 227–234.
- [95] M. Corporation. (2021). *CVE-2021-34527*. Accessed: Jan. 24, 2024. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2021-34527>
- [96] M. Corporation. (2021). *CVE-2021-44228*. Accessed: Jan. 24, 2024. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2021-44228>
- [97] M. Corporation. (2020). *CVE-2020-1472*. Accessed: Jan. 24, 2024. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2020-1472>
- [98] M. Corporation. (2019). *CVE-2019-1182*. Accessed: Jan. 24, 2024. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2019-1182>
- [99] M. Corporation. (2019). *CVE-2018-13379*. Accessed: Jan. 24, 2024. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2018-13379>
- [100] M. Corporation. (2017). *CVE-2017-0144*. Accessed: Jan. 24, 2024. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2017-0144>
- [101] M. Corporation. (2019). *CVE-2019-11510*. Accessed: Jan. 24, 2024. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2019-11510>
- [102] M. Corporation. (2020). *CVE-2020-0688*. Accessed: Jan. 24, 2024. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2020-0688>
- [103] M. Corporation. (2022). *CVE-2022-22954*. Accessed: Jan. 24, 2024. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2022-22954>
- [104] M. Corporation. (2022). *CVE-2022-22965*. Accessed: Jan. 24, 2024. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2022-22965>
- [105] R. S. Sajjan and V. R. Ghorpade, "Ransomware attacks: Radical menace for cloud computing," in *Proc. Int. Conf. Wireless Commun., Signal Process. Netw. (WiSPNET)*, Mar. 2017, pp. 1640–1646.
- [106] V. C. Craciun, A. Mogage, and E. Simion, "Trends in design of ransomware viruses," in *Innovative Security Solutions for Information Technology and Communications*, J.-L. Lanet and C. Toma, Eds., Berlin, Germany: Springer, 2019, pp. 259–272.
- [107] P. H. Meland, Y. F. F. Bayoumy, and G. Sindre, "The ransomware-as-a-service economy within the darknet," *Comput. Secur.*, vol. 92, May 2020, Art. no. 101762.
- [108] C. Karapapas, I. Pittaras, N. Fotiou, and G. C. Polyzos, "Ransomware as a service using smart contracts and IPFS," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency (ICBC)*, May 2020, pp. 1–5.
- [109] J. Singh, K. Sharma, M. Wazid, and A. K. Das, "SINN-RD: Spline interpolation-envisioned neural network-based ransomware detection scheme," *Comput. Electr. Eng.*, vol. 106, Mar. 2023, Art. no. 108601.
- [110] E. Berrueta, D. Morató, E. Magaña, and M. Izal, "Open repository for the evaluation of ransomware detection tools," *IEEE DataPort*, 2020, Accessed: Oct. 22, 2023, doi: [10.21227/qynq-q136](https://doi.org/10.21227/qynq-q136). [Online]. Available: <https://iee-dataport.org/open-access/open-repository-evaluation-ransomware-detection-tools>
- [111] X. Deng, M. Cen, M. Jiang, and M. Lu, "Ransomware early detection using deep reinforcement learning on portable executable header," *Cluster Comput.*, vol. 27, no. 2, pp. 1867–1881, Jun. 2023.
- [112] (2023). *Virusshare Malware Samples*. Accessed: Oct. 22, 2023. [Online]. Available: <https://virusshare.com>
- [113] T.-L. Lin, H.-Y. Chang, Y.-Y. Chiang, S.-C. Lin, T.-Y. Yang, C.-J. Zhuang, W.-L. Tseng, and B.-H. Zhang, "Ransomware detection by distinguishing API call sequences through LSTM and BERT models," *Comput. J.*, vol. 67, no. 2, pp. 632–641, Feb. 2024.
- [114] P. M. Anand, P. V. S. Charan, and S. K. Shukla, "HiPeR—Early detection of a ransomware attack using hardware performance counters," *Digit. Threats, Res. Pract.*, vol. 4, no. 3, pp. 1–24, Oct. 2023.
- [115] C. C. Moreira, D. C. Moreira, and C. D. S. D. Sales Jr., "Improving ransomware detection based on portable executable header using exception convolutional neural network," *Comput. Secur.*, vol. 130, Jul. 2023, Art. no. 103265.
- [116] C. Moreira, D. Moreira, and C. S. Jr., "Ransomware PE header feature dataset," Mendeley Data, Tech. Rep., 2024, doi: [10.17632/p3v94dft2y3](https://doi.org/10.17632/p3v94dft2y3). [Online]. Available: <https://data.mendeley.com/datasets/p3v94dft2y3>
- [117] M. A. Ayub, A. Siraj, B. Filar, and M. Gupta, "RWAarmor: A static-informed dynamic analysis approach for early detection of cryptographic windows ransomware," *Int. J. Inf. Secur.*, vol. 23, no. 1, pp. 533–556, Feb. 2024.
- [118] *Free Malware Reports*. Accessed: Nov. 15, 2024. [Online]. Available: <https://app.any.run/submissions>
- [119] H. Khalid Alkahtani, K. Mahmood, M. Khalid, M. Othman, M. Al Duhayyim, A. E. Osman, A. A. Alneil, and A. S. Zamani, "Optimal graph convolutional neural network-based ransomware detection for cybersecurity in IoT environment," *Appl. Sci.*, vol. 13, no. 8, p. 5167, Apr. 2023.
- [120] M. Abdullah Alohali, M. Elsadig, F. N. Al-Wesabi, M. Al Duhayyim, A. Mustafa Hilal, and A. Motwakel, "Optimal deep learning based ransomware detection and classification in the Internet of Things environment," *Comput. Syst. Sci. Eng.*, vol. 46, no. 3, pp. 3087–3102, 2023.
- [121] A. Singh, Z. Mushtaq, H. A. Abosag, S. N. F. Mursal, M. Irfan, and G. Nowakowski, "Enhancing ransomware attack detection using transfer learning and deep learning ensemble models on cloud-encrypted data," *Electronics*, vol. 12, no. 18, p. 3899, Sep. 2023.
- [122] *Ransomware Attacks*. Accessed: Nov. 16, 2024. [Online]. Available: <https://www.kaggle.com/datasets/shivansh002/ransomware-attacks>
- [123] K. Thummapudi, P. Lama, and R. V. Boppana, "Detection of ransomware attacks using processor and disk usage data," *IEEE Access*, vol. 11, pp. 51395–51407, 2023.
- [124] M. L. Hernandez-Jaimes, A. Martínez-Cruz, K. A. Ramírez-Gutiérrez, and E. Guevara-Martínez, "Enhancing machine learning approach based on nilsimsa fingerprinting for ransomware detection in IoMT," *IEEE Access*, vol. 12, pp. 153886–153897, 2024.
- [125] *Intelligent and Dynamic Ransomware Spread Detection and Mitigation in Integrated Clinical Environments*. Accessed: Nov. 17, 2024. [Online]. Available: <http://perception.inf.um.es/ICE-datasets>
- [126] M. Gazzan and F. T. Sheldon, "Novel ransomware detection exploiting uncertainty and calibration quality measures using deep learning," *Information*, vol. 15, no. 5, p. 262, May 2024.
- [127] J. Li, G. Yang, and Y. Shao, "Ransomware detection model based on adaptive graph neural network learning," *Appl. Sci.*, vol. 14, no. 11, p. 4579, May 2024.
- [128] M. Cen, X. Deng, F. Jiang, and R. Doss, "Zero-ran sniff: A zero-day ransomware early detection method based on zero-shot learning," *Comput. Secur.*, vol. 142, Jul. 2024, Art. no. 103849.
- [129] M. Gaber, M. Ahmed, and H. Janicke, "Zero day ransomware detection with pulse: Function classification with transformer models and assembly language," *Comput. Secur.*, vol. 148, Jan. 2025, Art. no. 104167.
- [130] M. Cen, F. Jiang, and R. Doss, "RansoGuard: A RNN-based framework leveraging pre-attack sensitive APIs for early ransomware detection," *Comput. Secur.*, vol. 150, Mar. 2025, Art. no. 104293.
- [131] D. Sgandurra, L. Muñoz-González, R. Mohsen, and E. C. Lupu, "Automated dynamic analysis of ransomware: Benefits, limitations and use for detection," 2016, *arXiv:1609.03020*.
- [132] S. Sharmeen, Y. A. Ahmed, S. Huda, B. S. Koçer, and M. M. Hassan, "Avoiding future digital extortion through robust protection against ransomware threats using deep learning based adaptive approaches," *IEEE Access*, vol. 8, pp. 24522–24534, 2020.
- [133] A. O. Almashhadani, M. Kaiiali, S. Sezer, and P. O'Kane, "A multi-classifier network-based crypto ransomware detection system: A case study of locky ransomware," *IEEE Access*, vol. 7, pp. 47053–47067, 2019.
- [134] R. Vinayakumar, K. P. Soman, K. K. S. Velan, and S. Ganorkar, "Evaluating shallow and deep networks for ransomware detection and classification," in *Proc. Int. Conf. Adv. Comput., Commun. Informat. (ICACCI)*, Sep. 2017, pp. 259–265.
- [135] Y. A. Ahmed, S. Huda, B. A. S. Al-Rimy, N. Alharbi, F. Saeed, F. A. Ghaleb, and I. M. Ali, "A weighted minimum redundancy maximum relevance technique for ransomware early detection in industrial IoT," *Sustainability*, vol. 14, no. 3, p. 1231, Jan. 2022.

- [136] S. Poudyal and D. Dasgupta, "AI-powered ransomware detection framework," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, Dec. 2020, pp. 1154–1161.
- [137] J. E. Hill, T. Owens Walker, J. A. Blanco, R. W. Ives, R. Rakvic, and B. Jacob, "Ransomware classification using hardware performance counters on a non-virtualized system," *IEEE Access*, vol. 12, pp. 63865–63884, 2024.
- [138] F. Manavi and A. Hamzeh, "Static detection of ransomware using LSTM network and PE header," in *Proc. 26th Int. Comput. Conf., Comput. Soc. Iran (CSICC)*, Mar. 2021, pp. 1–5.
- [139] F. Manavi and A. Hamzeh, "Ransomware detection based on PE header using convolutional neural networks," *ISecure*, vol. 14, no. 2, pp. 181–192, 2022, doi: [10.22042/isecure.2021.262846.595](https://doi.org/10.22042/isecure.2021.262846.595).
- [140] F. Manavi and A. Hamzeh, "A novel approach for ransomware detection based on PE header using graph embedding," *J. Comput. Virol. Hacking Techn.*, vol. 18, no. 4, pp. 285–296, Jan. 2022.
- [141] G. Chen, S. Zhang, W. Weng, and W. Yang, "Residual spatial-temporal graph convolutional neural network for on-street parking availability prediction," *Int. J. Sensor Netw.*, vol. 43, no. 4, pp. 246–257, 2023.
- [142] K. A. Alissa, D. H. Elkamchouchi, K. Tarmissi, A. Yafaz, R. Alsini, O. Alghushairy, A. Mohamed, and M. Al Duhayyim, "Dwarf mongoose optimization with machine-learning-driven ransomware detection in Internet of Things environment," *Appl. Sci.*, vol. 12, no. 19, p. 9513, Sep. 2022.
- [143] B. M. Khammas, "Ransomware detection using random forest technique," *ICT Exp.*, vol. 6, no. 4, pp. 325–331, Dec. 2020.
- [144] O. D. Okey, D. C. Melgarejo, M. Saadi, R. L. Rosa, J. H. Kleinschmidt, and D. Z. Rodríguez, "Transfer learning approach to IDS on cloud IoT devices using optimized CNN," *IEEE Access*, vol. 11, pp. 1023–1038, 2023.
- [145] M. Shoaib and M. S. Umar, "Phishing detection model using feline finch optimisation-based LSTM classifier," *Int. J. Sensor Netw.*, vol. 42, no. 4, pp. 205–220, 2023.
- [146] J. Lee, J. Yun, and K. Lee, "A study on countermeasures against neutralizing technology: Encoding algorithm-based ransomware detection methods using machine learning," *Electronics*, vol. 13, no. 6, p. 1030, Mar. 2024.
- [147] A. Gajjar, P. Kashyap, A. Aysu, P. Franzon, Y. Choi, C. Cheng, G. Pedretti, and J. Ignowski, "RD-FAXID: Ransomware detection with FPGA-accelerated XGBoost," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 17, no. 4, pp. 1–33, Dec. 2024.
- [148] M. Gazzan and F. T. Sheldon, "An incremental mutual information-selection technique for early ransomware detection," *Information*, vol. 15, no. 4, p. 194, Mar. 2024.
- [149] M. Davidian, M. Kiperberg, and N. Vanetik, "Early ransomware detection with deep learning models," *Future Internet*, vol. 16, no. 8, p. 291, Aug. 2024.
- [150] D. B. Oh, D. Kim, D. Kim, and H. K. Kim, "VolGPT: Evaluation on triaging ransomware process in memory forensics with large language model," *Forensic Sci. Int., Digit. Invest.*, vol. 49, Jul. 2024, Art. no. 301756.
- [151] A. K. and N. K. B., "An intelligent ransomware attack detection and classification using dual vision transformer with mantis search split attention network," *Comput. Electr. Eng.*, vol. 119, Oct. 2024, Art. no. 109509.
- [152] A. Yu, J. Kang, J. Morris, E. Bertino, and D. Lin, "Fight malware like malware: A new defense method against crypto ransomware," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 5, pp. 4953–4966, Sep. 2024.
- [153] M. Alazab, "Android ransomware detection using binary Jaya optimization algorithm," *Exp. Syst.*, vol. 41, no. 1, p. 13479, Jan. 2024.
- [154] H. H. R. Manzil and S. M. Naik, "Android ransomware detection using a novel Hamming distance based feature selection," *J. Comput. Virol. Hacking Techn.*, vol. 20, no. 1, pp. 71–93, Aug. 2023.
- [155] I. Almomani, A. Alkhayer, and W. El-Shafai, "E2E-RDS: Efficient end-to-end ransomware detection system based on static-based ML and vision-based DL approaches," *Sensors*, vol. 23, no. 9, p. 4467, May 2023.
- [156] M. Alazab, R. A. Khurma, D. Camacho, and A. Martín, "Enhanced Android ransomware detection through hybrid simultaneous swarm-based optimization," *Cognit. Comput.*, vol. 16, no. 5, pp. 2154–2168, Sep. 2024.
- [157] S. E. Lab. (2023). *A Dataset of Android Applications for Ransomware Detection*. Accessed: Oct. 22, 2023. [Online]. Available: https://sel.psu.edu.sa/Research/datasets/2020_RansIm-DS.php
- [158] K. R. Kalphana, S. Aanankumar, M. Surya, M. S. Ramadevi, K. R. Ramela, T. Anitha, N. Nagaprasad, and R. Krishnaraj, "Prediction of Android ransomware with deep learning model using hybrid cryptography," *Sci. Rep.*, vol. 14, no. 1, p. 22351, Sep. 2024.
- [159] K. Allix, T. F. Bissyandé, J. Klein, and Y. L. Traon, "AndroZoo: Collecting millions of Android apps for the research community," in *Proc. IEEE/ACM 13th Work. Conf. Mining Softw. Repositories (MSR)*, May 2016, pp. 468–471.
- [160] S. R. Jeremiah, H. Chen, S. Gritalis, and J. H. Park, "Leveraging application permissions and network traffic attributes for Android ransomware detection," *J. Neww. Comput. Appl.*, vol. 230, Oct. 2024, Art. no. 103950.
- [161] J. A. Gómez-Hernández and P. García-Teodoro, "Lightweight crypto-ransomware detection in Android based on reactive honeyfile monitoring," *Sensors*, vol. 24, no. 9, p. 2679, Apr. 2024.
- [162] *SK3PTRE/Androidmalware_2020*. Accessed: Nov. 16, 2024. [Online]. Available: https://github.com/sk3ptre/AndroidMalware_2020
- [163] N. Singh and S. Tripathy, "It's too late if exfiltrate: Early stage Android ransomware detection," *Comput. Secur.*, vol. 141, Jun. 2024, Art. no. 103819.
- [164] (2020). *Andmal*. Accessed: Nov. 16, 2024. [Online]. Available: <https://www.unb.ca/cic/datasets/andmal2020.html>
- [165] M. A. Hossain, T. Hasan, F. Ahmed, S. H. Chergee, M. H. Kanchan, and M. A. Haque, "Towards superior Android ransomware detection: An ensemble machine learning perspective," *Cyber Secur. Appl.*, vol. 3, Dec. 2025, Art. no. 100076.
- [166] *Android Ransomware Detection*. Accessed: Nov. 16, 2024. [Online]. Available: <https://www.kaggle.com/datasets/subhajournal/android-ransomware-detection>
- [167] I. Almomani, R. Qaddoura, M. Habib, S. Alsoghyer, A. A. Khayer, I. Aljarah, and H. Faris, "Android ransomware detection based on a hybrid evolutionary approach in the context of highly imbalanced data," *IEEE Access*, vol. 9, pp. 57674–57691, 2021.
- [168] K. Higuchi and R. Kobayashi, "Real-time open-file backup system with machine-learning detection model for ransomware," *Int. J. Inf. Secur.*, vol. 24, no. 1, p. 54, Feb. 2025.
- [169] B. Alexandrov, L. Gotsev, M. V. Petkova, and V. V. Petkova, "Heuristic approach to ransomware detection and prevention at software or hardware level," in *Proc. 3rd Int. Conf. Electr., Comput., Commun. Mechatronics Eng. (ICECCME)*, Jul. 2023, pp. 1–6.
- [170] S. Sheen, K. A. Asmitha, and S. Venkatesan, "R-sentry: Deception based ransomware detection using file access patterns," *Comput. Electr. Eng.*, vol. 103, Oct. 2022, Art. no. 108346.
- [171] T. Kim, "Deception-based method for ransomware detection," *J. Internet Services Inf. Secur.*, vol. 13, no. 3, pp. 191–201, Aug. 2023.
- [172] M. U. Rana, M. A. Shah, M. A. Al-Naeem, and C. Maple, "Ransomware attacks in cyber-physical systems: Countermeasure of attack vectors through automated web defenses," *IEEE Access*, vol. 12, pp. 149722–149739, 2024.
- [173] A. AlMajali, A. Elmosalamy, O. Safwat, and H. Abouelela, "Adaptive ransomware detection using similarity-preserving hashing," *Appl. Sci.*, vol. 14, no. 20, p. 9548, Oct. 2024.
- [174] J. Lee, S.-Y. Lee, K. Yim, and K. Lee, "Neutralization method of ransomware detection technology using format preserving encryption," *Sensors*, vol. 23, no. 10, p. 4728, May 2023.
- [175] J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Trans. Inf. Theory*, vol. 37, no. 1, pp. 145–151, Jan. 1991.
- [176] A. Vassilev and T. A. Hall, "The importance of entropy to information security," *Computer*, vol. 47, no. 2, pp. 78–81, Feb. 2014.
- [177] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. Hoboken, NJ, USA: Wiley, 1990.
- [178] A. J. Kulkarni, G. Krishnasamy, A. Abraham, A. J. Kulkarni, G. Krishnasamy, and A. Abraham, "Solution to 0–1 knapsack problem using cohort intelligence algorithm," in *Cohort Intelligence: A Socio-Inspired Optimization Method*. Cham, Switzerland: Springer, 2017, pp. 55–74, doi: [10.1007/978-3-319-44254-9_5](https://doi.org/10.1007/978-3-319-44254-9_5).
- [179] B. Ma, L. Zhou, C. Liao, Y. Zhou, J. Li, and J. Ma, "RansomSentry: Runtime detection of Android ransomware with compiler-based instrumentation," *IEEE Trans. Dependable Secure Comput.*, early access, Jan. 15, 2025, doi: [10.1109/TDSC.2025.3529119](https://doi.org/10.1109/TDSC.2025.3529119).

- [180] K. Drabent, R. Janowski, and J. M. Batalla, "How to circumvent and beat the ransomware in Android operating system—A case study of locker. CB! Tr," *Electronics*, vol. 13, no. 11, p. 2212, 2024.
- [181] L. Albshaier, S. Almarri, and M. M. H. Rahman, "Earlier decision on detection of ransomware identification: A comprehensive systematic literature review," *Information*, vol. 15, no. 8, p. 484, Aug. 2024.
- [182] H. Zhang, L. Zhao, A. Yu, L. Cai, and D. Meng, "Ranker: Early ransomware detection through kernel-level behavioral analysis," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 6113–6127, 2024.
- [183] A. Continella, A. Guagnelli, G. Zingaro, G. De Pasquale, A. Barengi, S. Zanero, and F. Maggi, "ShieldFS: A self-healing, ransomware-aware filesystem," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, Dec. 2016, pp. 336–347.
- [184] A. Continella, A. Guagnelli, G. Zingaro, G. D. Pasquale, A. Barengi, S. Zanero, and F. Maggi. (2023). *ShieLDFS: A Self-healing, Ransomware-Aware Filesystem*. Accessed: Jan. 7, 2024. [Online]. Available: <http://shieldfs.necst.it>
- [185] J. Chen, C. Wang, Z. Zhao, K. Chen, R. Du, and G.-J. Ahn, "Uncovering the face of Android ransomware: Characterization and real-time detection," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 5, pp. 1286–1300, May 2018.
- [186] L. Bhagwat and B. Patil, "Ransomware API calls," *IEEE DataPort*, 2022, Accessed: Oct. 22, 2023, doi: [10.21227/9fke-w026](https://doi.org/10.21227/9fke-w026). [Online]. Available: <https://iee-dataport.org/documents/ransomware-api-calls>
- [187] J. A. Herrera-Silva and M. Hernández-Álvarez, "Dynamic feature dataset for ransomware detection using machine learning algorithms," *Sensors*, vol. 23, no. 3, p. 1053, Jan. 2023.
- [188] M. Hirano, R. Hodota, and R. Kobayashi, "RanSAP: An open dataset of ransomware storage access patterns for training machine learning models," *Forensic Sci. Int., Digit. Invest.*, vol. 40, Dec. 2021, Art. no. 301314.
- [189] K. Thummapudi, R. Boppana, and P. Lama. (2022). *IO_5Events_7Rounds—5_Events_7_Rounds*. Harvard Dataverse, doi: [10.7910/DVN/R9FYPL](https://doi.org/10.7910/DVN/R9FYPL). [Online]. Available: [10.7910/DVN/R9FYPL](https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/R9FYPL)
- [190] K. Thummapudi, R. Boppana, and P. Lama. (2022). *HPC_5Events_7Rounds—5_Events_7_Rounds*, doi: [10.7910/DVN/YAYWOJ](https://doi.org/10.7910/DVN/YAYWOJ). [Online]. Available: [Hpc_5events_7rounds.10.7910/DVN/YAYWOJ](https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/YAYWOJ)
- [191] K. Thummapudi, R. Boppana, and P. Lama. (2022). *IO_5Events_7Rounds—5_Events_7_Rounds*, doi: [10.7910/DVN/R9FYPL](https://doi.org/10.7910/DVN/R9FYPL). [Online]. Available: [Hpc_41events_5rounds.10.7910/DVN/MA5UPP](https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/R9FYPL)
- [192] K. Thummapudi, R. Boppana, and P. Lama. (2022). *IO_41Events_5Rounds—41_Events_5_Rounds*, doi: [10.7910/DVN/GHJFUT](https://doi.org/10.7910/DVN/GHJFUT). [Online]. Available: [10.7910/DVN/GHJFUT](https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/GHJFUT)
- [193] A. Singh, "PE entropy," *IEEE DataPort*, 2023, doi: [10.21227/0656-4g20](https://doi.org/10.21227/0656-4g20). [Online]. Available: <https://iee-dataport.org/documents/pe-entropy>
- [194] C. Chew, V. Kumar, P. Patros, and R. Malik, "Android crypto ransomware system call dataset," *IEEE DataPort*, 2024, doi: [10.21227/95y1-5h19](https://doi.org/10.21227/95y1-5h19). [Online]. Available: <https://iee-dataport.org/documents/android-crypto-ransomware-system-call-dataset>
- [195] C. Moreira, D. Moreira, and C. S. Jr., "Ransomware combined structural feature dataset," Mendeley Data, Tech. Rep., 2024, doi: [10.17632/yzhcvm7sj53](https://doi.org/10.17632/yzhcvm7sj53). [Online]. Available: <https://data.mendeley.com/datasets/yzhcvn7sj53>
- [196] C. C. Moreira, D. C. Moreira, and C. Sales, "A comprehensive analysis combining structural features for detection of new ransomware families," *J. Inf. Secur. Appl.*, vol. 81, Mar. 2024, Art. no. 103716.
- [197] M. Hirano and R. Kobayashi, "RanSMAP: Open dataset of ransomware storage and memory access patterns for creating deep learning based ransomware detectors," *Comput. Secur.*, vol. 150, Mar. 2025, Art. no. 104202.
- [198] R. S. de Zaldivar, A. H. Celdrán, J. von der Assen, P. M. S. Sánchez, G. Bovet, G. M. Pérez, and B. Stiller, "MalwSpecSys: A dataset containing syscalls of an IoT spectrum sensor affected by heterogeneous malware," *IEEE DataPort*, 2022, doi: [10.21227/nvmb-eg69](https://doi.org/10.21227/nvmb-eg69). [Online]. Available: <https://iee-dataport.org/documents/malwspecsys-dataset-containing-syscalls-iot-spectrum-sensor-affected-heterogeneous-malware>
- [199] (2023). *Thezoo - a Live Malware Repository*. Accessed: Oct. 22, 2023. [Online]. Available: <https://thezoo.morirt.com>
- [200] (2023). *Hybrid Analysis File Collections*. Accessed: Nov. 26, 2023. [Online]. Available: <https://hybrid-analysis.com>
- [201] (2023). *VirusTotal's*. Accessed: Nov. 26, 2023. [Online]. Available: <https://VirusTotal.com>
- [202] *Malwarebazaar*. Accessed: Nov. 15, 2024. [Online]. Available: <https://bazaar.abuse.ch/browse>
- [203] *Free Malware Feeds - Virussign*. Accessed: Nov. 15, 2024. [Online]. Available: <https://www.virussign.com/free-downloads>
- [204] M. Sewak, S. K. Sahay, and H. Rathore, "Deep counterstrike: Counter adversarial deep reinforcement learning for defense against metamorphic ransomware swarm attack," in *Broadband Communications, Networks, and Systems*, W. Wang and J. Wu, Eds., Berlin, Germany: Springer, 2023, pp. 31–50.
- [205] A. Adamov and A. Carlsson, "Reinforcement learning for anti-ransomware testing," in *Proc. IEEE East-West Design Test Symp. (EWDTS)*, Sep. 2020, pp. 1–5.
- [206] Z. Zhou, G. Liu, and Y. Tang, "Multiagent reinforcement learning: Methods, trustworthiness, applications in intelligent vehicles, and challenges," *IEEE Trans. Intell. Vehicles*, early access, Jun. 3, 2024, doi: [10.1109/TIV.2024.3408257](https://doi.org/10.1109/TIV.2024.3408257).
- [207] T. Dichpally, Y. Wutla, and S. Jayachandran, "Reinforcement learning in healthcare: Applications and challenges," in *Cognitive Analytics and Reinforcement Learning: Theories, Techniques and Applications*. Beverly, MA, USA: Scrivener Publishing LLC, 2024, pp. 283–315, doi: [10.1002/9781394214068.ch15](https://doi.org/10.1002/9781394214068.ch15).
- [208] A. Wong, T. Bäck, A. V. Kononova, and A. Plaat, "Deep multiagent reinforcement learning: Challenges and directions," *Artif. Intell. Rev.*, vol. 56, no. 6, pp. 5023–5056, Oct. 2022.
- [209] A. Nowé, "Trustworthy reinforcement learning: Opportunities and challenges," in *Proc. 23rd Int. Conf. Auto. Agents Multiagent Syst.*, 2024, p. 1.
- [210] C. O. Retzlaff, S. Das, C. Wayllace, P. Mousavi, M. Afshari, T. Yang, A. Saranti, A. Angerschmid, M. E. Taylor, and A. Holzinger, "Human-in-the-loop reinforcement learning: A survey and position on requirements, challenges, and opportunities," *J. Artif. Intell. Res.*, vol. 79, pp. 359–415, Jan. 2024.
- [211] S. Asiri, Y. Xiao, and S. Alzahrani, "Towards improving phishing detection system using human in the loop deep learning model," in *Proc. ACM Southeast Conf.*, Apr. 2024, pp. 77–85.
- [212] E. Kim, S.-J. Park, D.-K. Chae, S. Choi, and S.-W. Kim, "A human-in-the-loop approach to malware author classification," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2020, pp. 3289–3292.
- [213] S. Kumar, S. Datta, V. Singh, D. Datta, S. K. Singh, and R. Sharma, "Applications, challenges, and future directions of human-in-the-loop learning," *IEEE Access*, vol. 12, pp. 75735–75760, 2024.
- [214] P. Thakur, V. Kansal, and V. Rishiwal, "Hybrid deep learning approach based on LSTM and CNN for malware detection," *Wireless Pers. Commun.*, vol. 136, no. 3, pp. 1879–1901, Jun. 2024.
- [215] S. Venkatraman, M. Alazab, and R. Vinayakumar, "A hybrid deep learning image-based analysis for effective malware detection," *J. Inf. Secur. Appl.*, vol. 47, pp. 377–389, Aug. 2019.
- [216] S. Yoo, S. Kim, S. Kim, and B. B. Kang, "AI-HydRa: Advanced hybrid approach using random forest and deep learning for malware classification," *Inf. Sci.*, vol. 546, pp. 420–435, Feb. 2021.
- [217] T. Lu, Y. Du, L. Ouyang, Q. Chen, and X. Wang, "Android malware detection based on a hybrid deep learning model," *Secur. Commun. Netw.*, vol. 2020, pp. 1–11, Aug. 2020.
- [218] H.-J. Zhu, L.-M. Wang, S. Zhong, Y. Li, and V. S. Sheng, "A hybrid deep network framework for Android malware detection," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 12, pp. 5558–5570, Dec. 2022.
- [219] F. Teichmann, "Ransomware attacks in the context of generative artificial intelligence—An experimental study," *Int. Cybersecurity Law Rev.*, vol. 4, no. 4, pp. 399–414, 2023.
- [220] Y. S. Sun, Z.-K. Chen, Y.-T. Huang, and M. C. Chen, "Unleashing malware analysis and understanding with generative AI," *IEEE Secur. Privacy*, vol. 22, no. 3, pp. 12–23, May 2024.
- [221] U. Ahmed, J. C.-W. Lin, and G. Srivastava, "Mitigating adversarial evasion attacks of ransomware using ensemble learning," *Comput. Electr. Eng.*, vol. 100, May 2022, Art. no. 107903.
- [222] P. Wang, J.-H. Chen, H.-C. Lin, and W.-H. Lin, "A novel method for ransomware family detection on deep learning-based models using adversarial examples," in *Proc. IEEE 5th Eurasia Conf. IoT, Commun. Eng. (ECICE)*, Oct. 2023, pp. 294–300.
- [223] E. Idrizovic, B. Jung, D. Raygoza, and S. Hughes. (2022). *Navigating the Vast Ocean of Sandbox Evasions*. Accessed: Jun. 12, 2024. [Online]. Available: <https://unit42.paloaltonetworks.com/sandbox-evasion-memory-detection>

- [224] T. R. McIntosh, T. Sušnjak, T. Liu, D. Xu, P. Watters, D. Liu, Y. Hao, A. Ng, and M. N. Halgamuge, "Ransomware reloaded: Re-examining its trend, research and mitigation in the era of data exfiltration," *ACM Comput. Surv.*, vol. 57, no. 1, Aug. 2024, Art. no. 18, doi: 10.1145/3691340.
- [225] L. Svet, A. Brightwell, A. Wildflower, and C. Marshwood, "Unveiling zero-space detection: A novel framework for autonomous ransomware identification in high-velocity environments," 2025, *arXiv:2501.12811*.
- [226] N. Ouerdi, B. Mejjout, K. Laaroussi, and M. A. Kasmi, "Ransomware detection and classification using machine learning and deep learning," in *Advances in Smart Medical, IoT & Artificial Intelligence*, M. Serhini and K. Ghomid, Eds., Berlin, Germany: Springer, 2024, pp. 195–202.
- [227] M. S. Shifa, M. Hasan, M. Jakir Hossain, T. I. Tasin, M. R. Sarker, and M. Islam, "Ransomware attacks and detection mechanisms: A systematic literature review," in *Cyber Intelligence and Information Retrieval*, S. Dutta, A. Bhattacharya, C. Shahnaz, and S. Chakrabarti, Eds., Singapore: Springer, pp. 65–71.
- [228] L. Chen, C.-Y. Yang, A. Paul, and R. Sahita, "Towards resilient machine learning for ransomware detection," 2018, *arXiv:1812.09400*.
- [229] S. Wang, F. Dong, H. Yang, J. Xu, and H. Wang, "CanCal: Towards real-time and lightweight ransomware detection and response in industrial environments," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 2326–2340.
- [230] F. S. Prity, M. S. Islam, E. H. Fahim, M. M. Hossain, S. H. Bhuiyan, M. A. Islam, and M. Raquib, "Machine learning-based cyber threat detection: An approach to malware detection and security with explainable AI insights," *Hum.-Intell. Syst. Integr.*, vol. 6, no. 1, pp. 61–90, Dec. 2024.



SALEH ALZHRANI received the bachelor's degree in information systems from King Khalid University, Abha, Saudi Arabia, and the master's degree in computer information systems from St. Mary's University, San Antonio, TX, USA, in 2018. He is currently pursuing the Ph.D. degree in computer science with The University of Alabama, Tuscaloosa, AL, USA. His current research interests include computer security, cyber security, and malware analysis and detection.



YANG XIAO (Fellow, IEEE) received the B.S. and M.S. degrees in computational mathematics from Jilin University, Changchun, China, in 1989 and 1991, respectively, and the M.S. and Ph.D. degrees in computer science and engineering from Wright State University, Dayton, OH, USA, in 2000 and 2001, respectively.

He is currently a Full Professor with the Department of Computer Science, The University of Alabama, Tuscaloosa, AL, USA. He directed more than 20 Ph.D. dissertations and supervised more than 20 M.S. theses/projects. He has published more than 600 articles (more than 300 SCI-indexed journal articles (including more than 70 IEEE/ACM TRANSACTIONS) and more than 300 conference papers and book chapters) with more than 29653 citations and H-index=86 by Google Scholar. He is ranked #188 and #481 in USA (#362 and #808 in the world) in electronics and electrical engineering (EEE) and computer science (CS), respectively, among the Best Scientists for 2024 based on Research.com. The scientists of Stanford University identified him as one of the world's Top 2% of Scientists. His research interests include cyber-physical systems, the Internet of Things, security, wireless networks, smart grids, and telemedicine. He was a Voting Member of the IEEE 802.11 Working Group, from 2001 to 2004, involving the IEEE 802.11 (Wi-Fi) standardization work. He is a fellow of IET, AAIA, AIIA, and ACIS. He holds the Certificate of Senior Software Engineer issued by the State Council of China, in 1991. He serves/served as a member for the Technical Program Committee for more than 300 conferences. He received the IEEE Transactions on Network Science and Engineering Excellent

Editor Award, in 2022 and 2023. He served as a Guest Editor 39 times for different international journals, including IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS (JSAC), in 2022 and 2023, IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING (TNSE), in 2021, IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING, in 2021, IEEE NETWORK, in 2007, IEEE WIRELESS COMMUNICATIONS, in 2006 and 2021, IEEE Communications Standards Magazine, in 2021, and Mobile Networks and Applications (MONET) (ACM/Springer), in 2008. He also serves as the Editor-in-Chief for Cyber-Physical Systems, International Journal of Sensor Networks (IJSNet), and International Journal of Security and Networks (IJSN). He has been serving as an Editorial Board Member and an Associate Editor for 20 international journals, including IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING, since 2022, IEEE TRANSACTIONS ON CYBERNETICS, since 2020, IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS, from 2014 to 2015, IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, from 2007 to 2009, and IEEE COMMUNICATIONS SURVEYS AND TUTORIALS, from 2007 to 2014.



SULTAN ASIRI received the B.S. degree in computer information systems from King Khalid University, Saudi Arabia, the master's degree in computer information science from Gannon University, Erie, PA, USA, and the Ph.D. degree in computer science from The University of Alabama, Tuscaloosa, AL, USA.

Currently, he is an Assistant Professor with King Khalid University. He is a Consultant with King Khalid University Center of Artificial Intelligence.

His research interests include applied natural language processing (NLP), security (including social engineering and adversarial attacks), deep learning, AI security and challenges, and geospatial analysis. In addition to his research, he actively participates in the academic community by serving as a reviewer for prestigious conferences, such as ACM conferences and journals, including IEEE and MDPI.



JIANYING ZHENG received the B.S. degree from the University of Science and Technology of China, in 2004, and the Ph.D. degree from Shenyang Institute of Automation, Chinese Academy of Sciences, in 2010. He is currently a Professor with the School of Intelligent Manufacturing and Smart Transportation, Suzhou City University. His research interests include wireless sensor networks, intelligent transportation systems, and multi-agent systems.



TIESHAN LI (Senior Member, IEEE) received the B.S. degree in ocean fisheries engineering from the Ocean University of China, Qingdao, China, in 1992, and the Ph.D. degree in vehicle operation engineering from Dalian Maritime University (DMU), China, in 2005.

He was a Lecturer with DMU, from September 2005 to June 2006, an Associate Professor, from 2006 to 2011, and a Ph.D. Supervisor, since 2009. Since July 2011, he has been a Full Professor with DMU and the Chaired Professor, since 2021. Currently, he is a tenured Professor with the University of Electronic Science and Technology of China. From March 2007 to April 2010, he was a Postdoctoral Scholar with the School of Naval Architecture, Ocean and Civil Engineering, Shanghai Jiao Tong University. From November 2008 to February 2009, and from December 2014 to March 2015, he visited the City University of Hong Kong as a Senior Research Associate (SRA). In September 2013, he visited the University of Macau as a Visiting Scholar many times. His research interests include intelligent learning and control for nonlinear systems, multi-agent systems, and their applications to marine vehicle control.

...